

FROM C TO JAVA · PRODUCTION-READY REFERENCE

Java 使用大全

面向 C 语言学习者的 14 讲系统进阶

覆盖 14 大主题:数据类型 → 运算符 → 流程控制 → 函数 → 内存管理 → 面向对象
文件I/O → 网络 → 并发异常 → 泛型与元编程 → 反射 → 注解装饰器 → 模块系统 → 类型系统进阶

第1章 数据类型与变量 · 第2章 运算符与表达式 · 第3章 流程控制 · 第4章 函数与方法
第5章 内存管理 · 第6章 面向对象 · 第7章 文件I/O · 第8章 网络通信
第9章 并发与异常 · 第10章 泛型与元编程 · 第11章 反射与动态特性 · 第12章 注解与装饰器
第13章 模块系统 · 第14章 类型系统进阶

知识导图 × C 语言对照 × 例题精讲 × 章末实验

第1章 数据类型与变量

建议学时:3-5 小时 | 难度:★★☆☆☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 掌握 Java 全部 8 种基本类型,能说出与 C 的 `char/int/long/float/double` 的对应关系与差异
- 理解 JVM 与字节码:"一次编译、处处运行"的机制,以及"int 不是对象"的含义
- 掌握变量声明、初始化与默认值规则,并会使用 `var` 局部类型推导
- 掌握字面量写法(二进制/下划线/后缀/文本块)与 `final` 常量的语义(与 C 的 `const` 对比)
- 掌握类型转换规则,避开截断、溢出等常见坑;理解包装类与自动装箱的 `==` 陷阱
- 掌握字符串的不可变性与常用 API、数组的基础操作与类型安全的枚举

💡 从 C 到 Java:本章主题如何迁移

如果你会用 C 写 `int n = 0; char *s = "hi";`,那么 Java 的语法你几乎已经会了——但有三件"世界观"上的事必须先转换。**第一,程序不再直接跑在操作系统上,而是跑在 JVM 里。**C 编译器把源码编译成机器码交给 CPU;Java 编译器把源码编译成**字节码**,交给 JVM 解释/即时编译执行。代价是冷启动稍慢,收益是同一份字节码在 Windows、Linux、macOS 上行完全一致——这是生产环境"编译一次、处处运行"的基础。你的 `main` 是"JVM 调用的入口",所以签名固定为 `public static void main(String[] args)`。

第二,基本类型是"值",其他一切都是"引用"。C 里数组名是"退化的指针",Java 里数组是对象;C 里结构体可以整体拷贝,Java 里对象只能通过引用操作。`int、double` 等 8 种基本类型像 C 的值类型,其余一切(数组、字符串、对象)都是引用——引用类似 C 的指针,但没有指针运算、不会越界、不可能悬垂。

第三,变量有"默认值",内存由 JVM 管。C 的局部变量不初始化是未定义行为;Java 规定成员变量自动获得零值,局部变量则必须先初始化才能用,否则编译直接报错——把"未定义行为"变成"编译期错误"。`malloc/free` 彻底消失,取而代之的是自动垃圾回收(第5章详解)。

📦 前置知识自查

- C 基本类型的字节数与范围(`sizeof`);变量声明与初始化;局部变量与全局变量(`static`)的区别
- C 的整型字面量(0x 十六进制、0 八进制)与转义序列(`\n、\t`);隐式转换(整型提升)与强制转换(截断)
- C 的字符串(`char[]/char*`)与数组操作(`strlen/strcpy`、下标访问)
- C 的 `enum` 枚举与 `#define` 宏常量的局限

🚀 本章学习路线

1. **第一阶段(约 1.5 小时,建立世界观):**通读 §1.1–§1.3,理解 JVM、8 种基本类型与变量规则,对照"C 写法 | Java 写法"表格体会差异
2. **第二阶段(约 1.5 小时,动手写):**运行 §1.4–§1.8 的示例代码,重点实验 §1.6 的 `==` 陷阱与 §1.7 的字符串 API
3. **第三阶段(实验,约 1 小时):**独立完成章末实验"学生成绩登记与统计",用自测题验收
4. **验收标准:**不看资料能说出 8 种基本类型、成员变量与局部变量初始化的区别、包装类 `==` 陷阱的完整机制

本章知识导图

- §1.1 Java 平台与第一个程序(JVM / 字节码 / 编译运行流程 / printf)
- §1.2 基本类型总览(8 种类型表 / 与 C 一一对照 / boolean 与 char / 编码)
- §1.3 变量:声明、初始化、默认值与 var(成员变量零值 / 局部变量必须初始化)
- §1.4 字面量与常量(进制 / 下划线 / 后缀 / 文本块 / final 与 C 的 const)
- §1.5 类型转换(隐式拓宽 / 显式收窄 / 截断与除法丢精度)
- §1.6 包装类与自动装箱(装箱拆箱 / 缓存 / == 陷阱)
- §1.7 字符串与字符(不可变性 / API / 长度与字节 / == 与 equals)
- §1.8 数组入门(声明初始化 / length / 增强 for / 二维数组)
- §1.9 枚举:从 C 的 enum 到类型安全(带字段与方法 / values / valueOf)

§1.1 Java 平台与第一个程序

1.1.1 从"C 编译"到"Java 编译 + 运行"

C 的构建链是:源码 → 编译器 → 机器码 → 链接器 → 可执行文件,产物与 CPU、操作系统绑定。Java 的构建链是:源码(.java) → javac 编译器 → 字节码(.class) → JVM 解释或即时编译(JIT)执行。运行时代码路径是"先解释执行,热点代码被 JIT 编译成本地码"——所以 Java 程序有"预热"过程,性能测试必须考虑预热。

★ 重点:main 方法签名是固定的

入口方法必须精确写作 `public static void main(String[] args)`:`public` 让 JVM 能访问;`static` 使入口不依赖对象;`void` 表示返回给 JVM 的只有退出码;参数 `args` 对应 C 的 `argc/argv`。文件名必须与 `public` 类名一致。

示例 1-1 第一个 Java 程序(对照 C 的 hello.c)

```
// HelloWorld.java — 文件主名必须与 public 类名一致
public class HelloWorld {
    public static void main(String[] args) {
        // 编译: javac HelloWorld.java -> 生成 HelloWorld.class
        // 运行: java HelloWorld          (不带 .class 后缀)
        System.out.println("你好,Java!");
    }
}
```

⚠ 易错点 1:用 "java HelloWorld.class" 运行

`java` 命令的参数是类名而非文件名:应写 `java HelloWorld`。写了 `java HelloWorld.class` 会报 `Error: Could not find or load main class`。`javac` 要带 `.java` 后缀, `java` 不能带后缀——两者恰好相反。

1.1.2 输出与格式化:printf 的 Java 版本

Java 提供三层输出: `System.out.print` (不换行)、`System.out.println` (换行)、`System.out.printf` (格式化,语法与 C 的 `printf` 几乎相同)。区别:Java 用 `%n` 表示跨平台换行;字符串拼接用 `+` 运算符(任意类型都能拼)。

示例 1-2 printf 与字符串拼接

```

public class PrintDemo {
    public static void main(String[] args) {
        int n = 42;
        double pi = 3.1415926;
        String name = "张三";
        System.out.println("n = " + n);           // 拼接:任何类型 + 字符串
        System.out.printf("n = %d, pi = %.2f\n", n, pi); // 保留 2 位小数
        System.out.printf("姓名: %s,十六进制: %x,八进制: %o\n", name, n, n);
        System.out.printf("左对齐: %-8d| 补零: %05d\n", n, 7);
        System.out.printf("科学计数: %e\n", pi);
    }
}

```

§1.2 基本类型总览:与 C 一一对照

1.2.1 8 种基本类型

Java 只有 8 种基本类型,没有任何 unsigned 类型,也没有"int 大小由平台决定"这类 C 问题——所有类型的大小、范围由语言规范硬性规定。同样的 int 在 32 位与 64 位机器上都是 4 字节。

类型	字节	取值范围	C 对照
byte	1	-128 ~ 127	signed char
short	2	-32768 ~ 32767	short
int	4	约 ±21 亿	int(最常见)
long	8	约 ±9.2×10 ¹⁸	long long
float	4	约 ±3.4×10 ³⁸ (约 7 位有效数字)	float
double	8	约 ±1.8×10 ³⁰⁸ (约 15-16 位有效数字)	double
char	2	0 ~ 65535(UTF-16 码元)	char(1 字节,ASCII)
boolean	未规定	true / false	C 没有真正的 bool

三个关键差异:① **boolean 是独立类型**——C 用 0/非0 表示真假,Java 的 if/while 条件必须是 boolean,写 if (1) 直接编译错误,消灭了一大类"把赋值当比较"的 C 经典 bug;② **char 是 2 字节**——16 位无符号,存 UTF-16 编码单元,可直接写 '中';③ **没有 unsigned**——需要无符号语义时用更大的类型(如 long)或 Integer.toUnsignedString 等工具方法。

例题 1.1 检查你的 C 直觉:这行代码错在哪?

```

public class TypeDemo {
    public static void main(String[] args) {
        char ch = '中';           // 合法:char 是 16 位,容纳汉字
        boolean ok = 1;           // 编译错误:1 不能自动转成 boolean
        byte b = 128;             // 编译错误:128 超出 byte 范围(字面量默认 int)
        long big = 3000000000L;    // 合法:加 L 后缀才是 long 字面量
        System.out.println(ch + " " + big);
    }
}

```

1.2.2 字符与编码:从 ASCII 到 Unicode

C 时代一个字符一个字节,处理中文要靠 setlocale 与多字节编码"体操"。Java 出生就是 Unicode:char 是 UTF-16 编码单元,字符串内部是 UTF-16 序列,源码本身也允许中文标识符(但不建议)。生产要点:文件读写一律显式指定 UTF-8,避免 Windows 默认编码(GBK)导致的乱码——第7章详解。

§1.3 变量:声明、初始化与默认值

1.3.1 声明语法与 C 的差异

Java 声明变量与 C 几乎一致: 类型 名字;。但有两差异: ① 数组声明必须写作 `int[] a ( int a[]` 虽兼容但不建议); ② 没有全局变量——所有变量要么是类成员(静态/实例字段), 要么是局部变量, "全局"由类的 `public static` 字段承担。

★ 重点:默认值的完整规则

- **成员变量(字段):**自动获得零值——数值 0/0.0, boolean 为 false, char 为 `'\u0000'`, 引用类型为 `null`
- **局部变量:**没有任何默认值, 使用前必须显式初始化, 否则编译报错 "variable might not have been initialized"
- **数组元素:**如同成员变量, 自动获得零值(`new int[5]` 得到 5 个 0)

示例 1-3 默认值实验

```
public class DefaultValues {
    // 成员变量:自动零值
    static int i;
    static double d;
    static boolean flag;
    static String name;

    public static void main(String[] args) {
        System.out.println(i);    // 0
        System.out.println(d);    // 0.0
        System.out.println(flag); // false
        System.out.println(name); // null
        // 局部变量:必须先初始化才能读
        int x = 0;
        System.out.println(x);
        // int y; System.out.println(y); // 编译错误:可能未初始化
    }
}
```

1.3.2 var:局部变量类型推导(Java 10+)

C 没有类型推导; Java 10 引入 `var`, 让编译器从初始化表达式推断类型。 `var` 只适用于局部变量(含增强 for 循环变量、lambda 参数), 不能用于字段、方法参数与返回值——公共 API 必须显式标注类型, 这是工程约定。

示例 1-4 var 的使用边界

```
import java.util.ArrayList;

public class VarDemo {
    // var name = "x"; // 编译错误:var 不能用于字段
    public static void main(String[] args) {
        var count = 10;           // 推断为 int
        var name = "Java";        // 推断为 String
        var list = new ArrayList<String>(); // 推断为 ArrayList<String>
        for (var i = 0; i < 3; i++) { // 循环变量也可以用
            System.out.println(i);
        }
        // var x; // 编译错误:必须初始化
        // var y = null; // 编译错误:null 无法推断类型
        System.out.println(count + name + list.getClass());
    }
}
```

工程建议:什么时候用 var

生产惯例: 类型"又长又明显"时用 var(如 `var map = new HashMap<String, List<Integer>>()`),类型不直观或影响可读性时显式写出。公共 API 的签名永不使用 var。这是与 C 最大的风格差异之一,团队规范里通常会写明。

§1.4 字面量与常量

1.4.1 字面量写法:比 C 更丰富

Java 支持 C 的全部整型字面量风格(十进制、0x 十六进制、0 八进制),并新增**二进制字面量(0b)**与**下划线分隔符**。浮点字面量默认是 double,要 float 必须加 F 后缀;long 加 L 后缀(大写 L 避免与 1 混淆)。字符转义与 C 相同(`\n \t \\ \' \"`),另有 Unicode 转义 `\uXXXX`。

示例 1-5 字面量全集

```
public class LiteralDemo {
    public static void main(String[] args) {
        int hex = 0x1F;           // 十六进制:31
        int oct = 075;            // 八进制(前缀 0):61
        int bin = 0b1010;         // 二进制:10
        long big = 100_000_000_000L; // 下划线分隔,long 后缀 L
        double pi = 3.141_592_653; // 浮点下划线
        float f = 3.14F;          // float 必须加 F
        double sci = 1.5e3;        // 科学计数法:1500.0
        char c = '中';             // Unicode 转义:中
        char tab = '\t';           // 转义序列与 C 相同
        System.out.println(hex + " " + oct + " " + bin);
        System.out.println(big + " " + pi + " " + f + " " + sci);
    }
}
```

1.4.2 文本块(Java 15+):告别 "\n" 地狱

C 里写多行字符串要手动拼接 `"line1\n" "line2\n"`;Java 15 起可用**文本块**直接书写多行字符串,生产中最常见的用途是内嵌 SQL、JSON、HTML 模板。

示例 1-6 文本块

```
public class TextBlockDemo {
    public static void main(String[] args) {
        // 三个双引号开始,内容原样保留(无需 \n 转义)
        String sql = """
            SELECT id, name, score
            FROM student
            WHERE score > 60
            ORDER BY score DESC
            """;
        System.out.println(sql); // 文本块内无需 \n 转义,原样输出
    }
}
```

1.4.3 常量:final 与 C 的 const

C 的 `const int MAX = 100;` 是"只读变量",仍可能被指针绕过;Java 的 `final` 更严格: **final 变量一旦赋值,任何代码都不能再修改它**。约定俗成:类级常量写作 `public static final`,命名全大写加下划线。

示例 1-7 final 常量(对照:C 的 `#define MAX 100` 是宏、`const int n = 10` 是只读变量)

```

public class ConstDemo {
    // 类级常量:全大写 + 下划线
    public static final double PI = 3.14159265358979;
    private static final int MAX_RETRY = 3;

    public static void main(String[] args) {
        final int local = 100;
        // local = 200;    // 编译错误:final 变量不能重新赋值
        System.out.println(PI + " " + MAX_RETRY + " " + local);
        // final 引用:引用不可变,但指向的对象内容可变
        final StringBuilder sb = new StringBuilder("a");
        sb.append("b");    // 合法:对象内容可以变
        System.out.println(sb);
        // sb = new StringBuilder(); // 编译错误:引用不能重新指向
    }
}

```

⚠ 易错点 2:final 引用 ≠ 内容不可变

`final` 修饰引用类型时,保证的是"引用不能再指向别的对象",不保证对象内部状态不变。`final StringBuilder sb` 照样能 `sb.append()`。真正不可变的是 `String`、`Integer` 这类"不可变类",与 `final` 是两回事。

§1.5 类型转换:隐式与显式

1.5.1 隐式转换:只有"拓宽"是自动的

与 C 一样,Java 的隐式转换遵循"小 → 大"拓宽方向: `byte` → `short` → `int` → `long` → `float` → `double`,以及 `char` → `int`。任何"大 → 小"的收窄都必须显式强转。与 C 不同,Java 规则统一、无实现差异,不会出现"同一段代码不同编译器行为不同"。

1.5.2 显式强转与截断

示例 1-8 转换的规则与截断

```

public class CastDemo {
    public static void main(String[] args) {
        // 隐式拓宽:自动完成
        int i = 100;
        long l = i;
        double d = l;

        // 显式收窄:必须强转,可能丢失信息
        long big = 3_000_000_000L;
        int n = (int) big;    // 超出 int 范围:结果被截断,不可预料
        double pi = 3.99;
        int p = (int) pi;    // 截断:3(向零取整,与 C 相同)
        char ch = (char) 97;    // 'a'
        System.out.println(n + " " + p + " " + ch);

        // 经典坑:两个 int 相除,小数被丢弃
        int a = 7, b = 2;
        double r = a / b;    // 3.0!先整数除法得 3,再转 double
        double r2 = (double) a / b;    // 3.5,先把 a 转 double 再除
        System.out.println(r + " " + r2);
    }
}

```

⚠ 易错点 3:除法丢精度与强转截断

两个经典事故:① `double r = 7 / 2;` 得到 3.0 而不是 3.5——整数除法先执行,小数在转 `double` 之前就丢了;正确写法是 `(double) 7 / 2`。② 大 `long` 强转成 `int` 不报错,而是按位截断出"看似合理实则错误"的数,生产上应在转换前做范围检查(如 `if (big > Integer.MAX_VALUE)`)。

🚀 工程建议:金额永远用整数或 BigDecimal

float/double 是二进制浮点, $0.1 + 0.2$ 不等于 0.3 。涉及金额、税率的计算一律不用浮点:用 long (以分为单位)或 BigDecimal。这与 C 的教训一致,Java 只是提供了更顺手的工具。

§1.6 包装类与自动装箱:== 陷阱

1.6.1 为什么需要包装类

基本类型不是对象——数组、集合、泛型(第10章)要求元素是对象。于是每个基本类型都有对应**包装类**:byte→Byte、short→Short、int→Integer、long→Long、float→Float、double→Double、char→Character、boolean→Boolean。Java 5 起编译器自动在两者间转换,称为**自动装箱/拆箱**。

示例 1-9 装箱、拆箱与 == 陷阱(必背)

```
public class BoxDemo {
    public static void main(String[] args) {
        // 自动装箱:Integer.valueOf(int) 的语法糖
        Integer a = 100;        // 等价于 Integer a = Integer.valueOf(100);
        Integer b = 100;
        System.out.println(a == b);    // true:100 在 -128~127 缓存区间内

        Integer c = 200;
        Integer d = 200;
        System.out.println(c == d);    // false!超出缓存区间,两个不同对象
        System.out.println(c.equals(d)); // true:比较值,永远的正确姿势

        // 自动拆箱:Integer 参与算术时自动转 int
        Integer x = 10;
        int y = x + 1;            // 拆箱后相加:11
        System.out.println(y);

        // 拆箱空指针陷阱
        Integer z = null;
        // int w = z;            // 运行时 NullPointerException:null 不能拆箱
    }
}
```

⚠ 易错点 4:Integer == 的缓存区间 -128~127

装箱时,JVM 对 -128~127 之间的 Integer 复用缓存对象,所以 $a == b$ 是 true;超过这个区间每次装箱都新建对象, $c == d$ 是 false。同一段代码,改个数值结果反转——这是生产中最隐蔽的 bug 来源之一。铁律:包装类比较一律用 equals。另外 == 比较 Integer 与 int 时会发生拆箱,按值比较,行为正常。

§1.7 字符串与字符

1.7.1 字符串不可变:与 C 的 char[] 对照

C 的字符串是 char[],内容随时可改;Java 的 String 是**不可变对象**:任何"修改"操作(拼接、替换、截取)都返回新对象,原字符串不变。好处:线程安全、可放心共享、可作 HashMap 的键;代价:循环里拼接必须用 StringBuilder,否则会产生大量临时对象。

C 的写法	Java 的写法
<code>char s[] = "hello" (可变)</code>	<code>String s = "hello" (不可变)</code>
<code>strlen(s) (字节数)</code>	<code>s.length() (UTF-16 码元数)</code>
<code>strcpy(dst, s)</code>	<code>s2 = s (引用赋值即可)</code>
<code>strcat(dst, s)</code>	<code>s1 + s 或 s1.concat(s)</code>
<code>strcmp(a, b) (按字节)</code>	<code>a.equals(b) (按值比较!)</code>
<code>sprintf(buf, "%d", n)</code>	<code>String.format("%d", n) 或 Integer.toString(n)</code>

示例 1-10 字符串常用 API

```
public class StringDemo {
    public static void main(String[] args) {
        String s = "Java 教程";
        System.out.println(s.length());           // 6(UTF-16 码元数,不是字节数)
        System.out.println(s.charAt(0));           // 'J'
        System.out.println(s.substring(0, 4));     // "Java"(左闭右开)
        System.out.println(s.indexOf("教程"));     // 5
        System.out.println(s.contains("va"));      // true
        System.out.println(s.replace("教程", "使用大全")); // 返回新串,原串不变

        // 切分与拼接
        String[] parts = "a,b,c".split(",");       // -> [a, b, c]
        String joined = String.join("-", parts);    // -> a-b-c

        // 频繁拼接:用 StringBuilder
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < 1000; i++) {
            sb.append(i).append(',');
        }
        System.out.println(sb.length());           // 3890,而不是 1000 个新对象

        // 数值与字符串互转
        int n = Integer.parseInt("42");            // 字符串 -> 数值
        String s2 = Integer.toString(42);          // 数值 -> 字符串

        // == 与 equals:字符串常量池
        String t1 = "abc";
        String t2 = "abc";
        String t3 = new String("abc");
        System.out.println(t1 == t2);              // true(编译期常量入常量池)
        System.out.println(t1 == t3);              // false(不同对象)
        System.out.println(t1.equals(t3));         // true(按值比较,正确姿势)
    }
}
```

⚠ 易错点 5:字符串比较用 == 的后果

"abc" == "abc" 是 true(编译期常量进常量池复用),但 new String("abc") == "abc" 是 false, scanner.next() == "abc" 也几乎必然是 false——凡是运行时构造出来的字符串,== 都不可靠。生产铁律:字符串比较一律 equals;忽略大小写用 equalsIgnoreCase。C 用 strcmp,Java 用 equals,就是把"按值比较"从函数提升为方法。

§1.8 数组入门

1.8.1 声明、创建与 length

Java 数组与 C 数组的核心差异:数组是对象。声明 `int[] a` 只是声明引用;必须 `new int[5]` 才分配空间,元素自动获得零值。访问越界时 JVM 抛出 `ArrayIndexOutOfBoundsException`,而不是 C 的"静默越界"未定义行为。数组有 `length` 属性

——C 没有,C 程序员必须自己记大小,这是 C 数组最大的痛点。

示例 1-11 数组基础

```
import java.util.Arrays;

public class ArrayDemo {
    public static void main(String[] args) {
        int[] scores = new int[5];           // 5 个 0
        int[] primes = {2, 3, 5, 7, 11};     // 初始化(仅声明时可用)
        int[] more = new int[]{1, 2, 3};     // 匿名数组,可随时用

        System.out.println(primes.length);   // 5:length 是属性,不是函数
        System.out.println(scores[0]);       // 0(默认零值)

        for (int p : primes) {               // 增强 for:C 里没有
            System.out.print(p + " ");
        }
        System.out.println();

        Arrays.sort(primes);                 // 排序(原地)
        Arrays.fill(scores, 60);             // 全部填 60
        int[] copy = Arrays.copyOf(primes, 10); // 扩容拷贝,多余补 0
        System.out.println(Arrays.toString(primes)); // 转字符串方便打印

        // 二维数组:数组的数组,每行长度可以不同
        int[][] matrix = new int[3][4];      // 3 行 4 列
        int[][] triangle = {{1}, {1, 1}, {1, 2, 1}}; // 三角阵(每行不等长)
        System.out.println(matrix.length + " 行," + matrix[0].length + " 列");
    }
}
```

⚠ 易错点 6:数组是引用,拷贝要用 `copyOf`

`int[] b = a;` 只是让两个引用指向同一块内存——修改 `b` 会改 `a`,这与 C 的“数组名是首元素指针”本质相似。真正拷贝用 `Arrays.copyOf(a, len)` 或 `a.clone()`。生产上把数组传给方法也要小心:方法内修改会直接影响原数组(传引用,见第4章)。

§1.9 枚举:从 C 的 `enum` 到类型安全

1.9.1 C 的 `enum` 之痛

C 的 `enum {RED, GREEN, BLUE};` 本质是整型常量:名字全局可见、可以与其他整数混用(`int x = RED;` 随意)、没有字符串对应、不能携带附加信息。Java 的 `enum` 是一个完整的类:每个枚举值是它的实例,类型安全(`Season.SPRING` 不能赋给 `int`),可以带字段、构造器与方法,天然支持 `switch`,还有 `values()` / `valueOf()` / `ordinal()` 内置工具。

示例 1-12 带字段与方法的枚举


```

public enum Season {
    // 枚举值:每个都是 Season 的一个实例,可带参数
    SPRING("春", 1), SUMMER("夏", 2), AUTUMN("秋", 3), WINTER("冬", 4);

    private final String name;    // 附加字段
    private final int index;

    Season(String name, int index) {    // 构造器(自动为每个枚举值调用)
        this.name = name;
        this.index = index;
    }

    public String getName() { return name; }
    public int getIndex() { return index; }

    public static void main(String[] args) {
        Season s = Season.SPRING;
        System.out.println(s.getName() + " " + s.getIndex()); // 春 1
        for (Season x : Season.values()) {    // values():全部枚举值
            System.out.println(x.name() + ":" + x.ordinal());
        }
        Season parsed = Season.valueOf("SUMMER"); // 字符串 -> 枚举
        System.out.println(parsed.getName());    // 夏
    }
}

```



工程建议:用枚举替代魔法数字

生产代码规范普遍要求: 状态、类型、等级等有限取值集合一律用 enum,禁止裸用 int 和字符串。好处:编译期检查(拼错常量名直接编译失败)、switch 穷尽性检查(第3章)、携带业务信息(名称、颜色、权重)。这是与 C 的 #define 宏枚举完全不同的体验。

本章速查表

主题	要点 / 写法
基本类型	byte short int long float double char boolean,共 8 种,大小平台无关
boolean	只接受 true/false,if 条件必须是 boolean(C 的 0/非0 失效)
char	16 位无符号,UTF-16 码元,可直接写 '中'
成员变量	自动零值:0 / 0.0 / false / '\u0000' / null
局部变量	必须初始化才能使用,否则编译错误
var	仅局部变量(Java 10+);字段/参数/返回值不可用;必须初始化
字面量	0b1010 二进制、1_000 下划线、100L、3.14F、1.5e3
文本块	""" 多行字符串 """(Java 15+),适合 SQL/JSON
常量	public static final 全大写加下划线;final 引用 ≠ 内容不可变
隐式转换	只允许拓宽:byte→short→int→long→float→double
显式转换	(int) x 收窄强转;截断向零取整;7/2 得 3,要 3.5 先转 double
包装类	Integer/Character 等;自动装箱拆箱;-128~127 有缓存,== 有坑
比较铁律	包装类、String 一律 equals;字符串常量池使 == 结果"看运气"
String 不可变	拼接产生新对象;循环拼接用 StringBuilder
字符串互转	Integer.parseInt("42") / Integer.toString(42) / String.valueOf(x)
数组	int[] a = new int[5];元素零值;length 属性;越界抛异常
数组工具	Arrays.sort / fill / copyOf / toString / binarySearch
枚举	完整类:带字段/方法;values() / valueOf() / ordinal();类型安全

最小必会清单

- 能默写 8 种基本类型及字节数,并指出与 C 的对应关系(含 long long、无 unsigned 的差异)
- 能解释 JVM 与字节码:为什么同一个 .class 文件可以跨平台运行
- 能说出成员变量与局部变量初始化规则的差异,以及数组元素的默认值
- 会写 var 并说出使用边界(仅局部变量、必须初始化、公共 API 不用)
- 会写二进制/下划线/后缀字面量与文本块,能处理其中的转义
- 能说清 final 与 C const 的异同,以及"final 引用 ≠ 内容不可变"
- 能完整复述 Integer 缓存陷阱:-128~127 与 == 的关系,并坚持用 equals
- 能写出字符串常用操作并解释 length() 是码元数、String 不可变
- 会声明数组、用 length、增强 for、Arrays 工具,并知道"等号只拷贝引用"
- 能写出带字段与方法的枚举,并用 values()/valueOf() 操作

自测题

Q 1. 下列代码哪些行会编译错误? byte b = 128; long l = 3000000000L; boolean ok = 1; char c = '中'; double d = 3.14F;

 参考答案

`byte b = 128` 编译错误(128 是 int 字面量,超出 byte 范围,且 int 不能隐式收窄为 byte); `boolean ok = 1` 编译错误(boolean 不接受整数)。其余合法:long 带 L 后缀合法;char 是 16 位可放汉字;float 字面量可以自动拓宽为 double。

Q 2. `int a = 7, b = 2; double r = a / b;` 结果是多少?如何得到 3.5?

✅ 参考答案

r 是 3.0:先做整数除法 $7/2=3$,再隐式转成 double。要 3.5 应写 `double r2 = (double) a / b;` (强转优先于除法),或 `a / 2.0`。

Q 3. 解释下面的输出,并给出修正: `Integer x = 200, y = 200; System.out.println(x == y);`

✅ 参考答案

输出 false。200 超出 Integer 缓存区间 -128~127,自动装箱各创建新对象,== 比较引用。改成 `x.equals(y)` 即 true。若两数是 127 则 == 为 true,同一段逻辑换数值结果反转,这正是它危险的原因。

Q 4. 结果分别是什么? `String a = "ab"; String b = "a" + "b"; String c = new String("ab");` (提示:考虑编译期常量折叠与常量池)

✅ 参考答案

`a == b` 是 true("a"+"b" 是编译期常量,折叠为 "ab" 入常量池); `a == c` 是 false(new 必出新对象)。生产结论:除了常量折叠的少数情况,字符串比较一律用 equals。

Q 5. 枚举 `Direction { UP, DOWN, LEFT, RIGHT }`,如何用字符串 "UP" 得到枚举值?如何遍历?

✅ 参考答案

`Direction.valueOf("UP")` 返回 UP(字符串不存在时抛 `IllegalArgumentException`);遍历用 `for (Direction d : Direction.values())`。`ordinal()` 返回声明序号,但生产上不要依赖 ordinal 做业务逻辑(枚举顺序调整会破坏它)。

Q 6. `final StringBuilder sb = new StringBuilder("a"); sb.append("b"); sb = new StringBuilder();` 哪一行编译错误?为什么?

✅ 参考答案

第三行 `sb = new StringBuilder()` 编译错误:final 保证引用不能重新指向,但对象内部状态可变,所以 `sb.append("b")` 合法。这就是"final 引用 ≠ 内容不可变"的现场演示。

章末实验题:学生成绩登记与统计

实验 1:学生成绩登记与统计

实验目标:编写一个控制台程序,登记学生信息并统计成绩,覆盖本章全部知识点。

实验要求:

- 任务 1:定义常量 MAX_STUDENTS 与及格线(覆盖 §1.4 常量/final)
- 任务 2:定义成绩等级枚举,带中文名(覆盖 §1.9 枚举)
- 任务 3:定义 Student 类,字段覆盖整型/浮点/布尔/字符/字符串/枚举(覆盖 §1.2 基本类型)
- 任务 4:控制台循环输入,输入 q 结束(覆盖 §1.3 变量、§1.7 parseInt 字符串互转)
- 任务 5:统计平均/最高/最低/各等级人数,格式化输出(覆盖 §1.1 printf、§1.8 数组)
- 任务 6:按成绩降序排序并打印成绩单(覆盖 §1.8 数组拷贝与排序)

实验环境:JDK 17+;命令: javac GradeBook.java 编译, java GradeBook 运行。

第一步 写"骨架":main + 常量 + 枚举 + Student 类,先编译通过再填逻辑

第二步 现输入循环:用 Scanner 读取, Integer.parseInt / Double.parseDouble 转换——输入非法字符会抛异常,先不管(第9章学异常后回来加固)

第三步 现统计与排序:复制数组用普通 for 循环;排序用 C 程序员最熟悉的选择排序,逐行翻译成 Java 语法

参考答案要点

```

import java.util.Scanner;

/** 学生成绩登记与统计 — 第1章综合实验参考答案 */
public class GradeBook {

    // 常量:final + static,全大写命名(对应 C 的 #define)
    public static final int MAX_STUDENTS = 100;
    private static final double PASS_LINE = 60.0;

    // 枚举:从 C 的整型枚举升级为带中文名的类型安全枚举
    enum Grade {
        EXCELLENT("优秀"), GOOD("良好"), PASS("及格"), FAIL("不及格");

        private final String text;

        Grade(String text) {
            this.text = text;
        }

        public String text() {
            return text;
        }
    }

    // 学生类:字段覆盖数值/布尔/字符/字符串/枚举全部类型
    static class Student {
        String name;        // 字符串:姓名
        int id;              // 整型:学号
        double score;        // 浮点:成绩
        boolean passed;      // 布尔:是否及格
        char gender;         // 字符:性别 M/F
        Grade grade;         // 枚举:等级

        Student(String name, int id, double score, char gender) {
            this.name = name;
            this.id = id;
            this.score = score;
            this.gender = gender;
            this.passed = score >= PASS_LINE;
            this.grade = toGrade(score);
        }

        static Grade toGrade(double s) {
            if (s >= 90) return Grade.EXCELLENT;
            if (s >= 75) return Grade.GOOD;
            if (s >= PASS_LINE) return Grade.PASS;
            return Grade.FAIL;
        }
    }

    public static void main(String[] args) {
        Student[] students = new Student[MAX_STUDENTS]; // 数组:默认 null
        int count = 0;
        Scanner in = new Scanner(System.in);

        System.out.println("=== 学生成绩登记(输入 q 结束) ===");
        while (count < MAX_STUDENTS) {
            System.out.print("姓名: ");
            String name = in.nextLine();
            if (name.equals("q")) { break; } // 字符串比较必须用 equals
            System.out.print("学号: ");
            int id = Integer.parseInt(in.nextLine()); // 字符串 -> int
            System.out.print("成绩: ");
            double score = Double.parseDouble(in.nextLine()); // 字符串 -> double
            System.out.print("性别(M/F): ");
            char gender = in.nextLine().charAt(0); // 取首字符
            students[count] = new Student(name, id, score, gender);
            count++;
        }
    }
}

```

```

    }
    in.close();

    // 统计:总分 / 平均 / 最高 / 最低 / 各等级人数
    double sum = 0, max = 0, min = 100;
    int[] gradeCount = new int[Grade.values().length]; // 默认全 0
    for (int i = 0; i < count; i++) {
        Student s = students[i];
        sum += s.score;
        max = Math.max(max, s.score);
        min = Math.min(min, s.score);
        gradeCount[s.grade.ordinal()]++; // 用序号做数组下标
    }
    double avg = (count == 0) ? 0.0 : sum / count;

    // 格式化输出:%n 是跨平台换行
    System.out.printf("=== 统计结果(共 %d 人) ===%n", count);
    System.out.printf("平均分: %.1f 最高: %.1f 最低: %.1f%n", avg, max, min);
    System.out.printf("优秀 %d 人 良好 %d 人 及格 %d 人 不及格 %d 人%n",
        gradeCount[0], gradeCount[1], gradeCount[2], gradeCount[3]);

    // 拷贝数组(等号只是引用拷贝,必须逐元素复制)
    Student[] sorted = new Student[count];
    for (int i = 0; i < count; i++) {
        sorted[i] = students[i];
    }
    // 选择排序:按成绩降序(与 C 完全一致的算法,只是换 Java 语法)
    for (int i = 0; i < count - 1; i++) {
        int k = i;
        for (int j = i + 1; j < count; j++) {
            if (sorted[j].score > sorted[k].score) {
                k = j;
            }
        }
        Student t = sorted[i];
        sorted[i] = sorted[k];
        sorted[k] = t;
    }

    System.out.println("=== 成绩单(降序) ===");
    for (int i = 0; i < count; i++) {
        Student s = sorted[i];
        System.out.printf("%2d. %-8s %c %6.1f %s%n",
            i + 1, s.name, s.gender, s.score, s.grade.text());
    }
}
}

```

设计说明:枚举 Grade 同时承载"等级判断"与"中文名"两类信息;passed 与 grade 由构造器统一推导,避免数据不一致;排序刻意用选择排序而非 Arrays.sort,便于与 C 实现逐行对照。扩展方向:用 HashMap 按姓名查学生、用 try-catch 处理非法输入(第9章)。

下一章预告:第2章 运算符与表达式。有了类型与变量,接下来就是"操作"——Java 的运算符与 C 表面相似,但溢出语义、除法、短路、== 比较等细节完全不同,处处是坑。

第2章 运算符与表达式

建议学时:3-5 小时 | 难度:★★★☆☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 掌握 Java 算术运算与 C 的三点根本差异:有符号溢出是"回绕"而非 UB、整数除零抛异常、浮点除零得 Infinity/NaN
- 掌握 / 与 % 的取整方向,会用 Math.floorDiv/floorMod 获得向下取整语义
- 掌握复合赋值运算符的"隐藏强转",避免 `s += 1` 与 `s = s + 1` 的迷惑
- 掌握全套位运算与三种移位(<< >> >>>),会用标志位技术
- 掌握 `&&/||` 的短路求值与"操作数必须是 boolean"的强约束
- 掌握 `==` 的值/引用语义、浮点比较的正确姿势、instanceof 模式匹配
- 掌握完整优先级表与"操作数从左到右求值"的规定(C 的求值顺序是 UB!)

💡 从 C 到 Java:本章主题如何迁移

运算符是 C 程序员最有"熟悉感"也最容易翻车的一章:符号几乎全认识,语义却在几处关键点上完全不同。

第一,溢出的处理哲学不同。C 里 `int` 溢出是未定义行为,编译器可以"做任何事";Java 把行为固化下来:有符号整数溢出是 **静默回绕** (wrap-around),整数除零是抛异常,浮点除零是 Infinity。看似少了"自由",实则是生产安全的巨大胜利——行为可预测、可测试。

第二,真假值的强约束。C 的 `if (x)` 接受任意整数;Java 的 `&&/||/!` 只接受 boolean。`if (n = 5)` 这种"赋值当比较"的经典 C 事故,在 Java 里直接编译失败。代价是写法上必须显式比较;收益是一类 bug 从根上消失。

第三,求值顺序被规定死了。C 里 `f() + g()` 先调谁未定义;Java 规定操作数一律从左到右求值。本章的优先级表与 C 几乎相同,但"同一表达式不同编译器结果不同"这种事不再发生。

本章策略:先过一遍与 C 相同的部分(快速),然后把精力集中在上述差异与 `>>>`、`instanceof`、`Math` 工具这些 C 没有的东西上。

📚 前置知识自查

- C 的算术运算符、自增自减、位运算(`& | ^ ~ << >>`)、逻辑运算与短路
- C 的运算符优先级与结合性(口诀:单目 > 算术 > 移位 > 关系 > 位 > 逻辑 > 三目 > 赋值)
- C 的整数除法截断与取模符号规则;浮点精度问题
- C 的 `printf` 占位符与条件表达式 `?:`

🚀 本章学习路线

1. **第一阶段(约 1.5 小时,扫差异):**重点阅读 §2.1 溢出/除零/取模与 §2.4 短路,与 C 逐条对照;其余快速过
2. **第二阶段(约 1.5 小时,动手写):**运行 §2.2-§2.6 的全部示例,亲手验证每个输出,把"猜答案→对答案"做成小测验
3. **第三阶段(实验,约 1 小时):**独立完成章末实验"表达式计算器",用自测题验收
4. **验收标准:**不看资料能预测:溢出结果、取模符号、除零行为、短路是否执行右侧、复合赋值是否截断

本章知识导图

- §2.1 算术运算:溢出回绕 / 除零 / 除法与取模 / Math.floorDiv
- §2.2 自增自减与复合赋值(隐藏强转陷阱)
- §2.3 位运算与移位(>>> 逻辑右移 / 移位距离取模 / 标志位场景)
- §2.4 逻辑运算与短路(boolean 强约束 / 判空模式)
- §2.5 比较与相等(== 值/引用 / 浮点比较姿势 / instanceof 模式匹配)
- §2.6 三元、优先级与求值顺序(完整优先级表 / 从左到右求值)
- §2.7 字符串拼接 + 与 Math 工具速览

§2.1 算术运算:溢出、除零与取模

2.1.1 溢出:C 的 UB 与 Java 的回绕

C 标准规定有符号整数溢出是未定义行为,编译器可能优化出任何结果;Java 规定:有符号整数运算按补码回绕,即 `Integer.MAX_VALUE + 1` 恒等于 `Integer.MIN_VALUE`。行为确定、可测试,但不等于“不会错”——回绕值常常静默地通过后续检查,生产上应在运算前做范围检查或用 `Math.addExact` 等精确运算方法(溢出时抛异常)。

示例 2-1 溢出与除零

```
public class ArithDemo {
    public static void main(String[] args) {
        // 1) 整型溢出:C 是 UB,Java 是“静默回绕”
        int max = Integer.MAX_VALUE;
        System.out.println(max + 1);           // -2147483648:回绕
        System.out.println(max + 1L);          // 2147483648L:先提升为 long,安全
        System.out.println(Math.addExact(max, 1)); // 抛 ArithmeticException

        // 2) 整数除以零:抛异常(C 是 UB/直接崩溃)
        // int x = 1 / 0; // ArithmeticException: / by zero

        // 3) 浮点除以零:不抛异常,得无穷或 NaN
        System.out.println(1.0 / 0);           // Infinity
        System.out.println(-1.0 / 0);          // -Infinity
        System.out.println(0.0 / 0);           // NaN

        // 4) NaN 不等于任何数,包括自己
        double nan = 0.0 / 0.0;
        System.out.println(nan == nan);        // false!
        System.out.println(Double.isNaN(nan)); // true:唯一的正确判断方式
    }
}
```

⚠ 易错点 1:回绕值“看起来合理”

`Integer.MAX_VALUE + 1` 得到 -2147483648,如果代码接下来用 `if (sum > 0)` 判断正负,这个负数会静默通过边界检查,产生“查不出来的 bug”。生产防护:金额与计数器用 `long`;必须 `int` 且可能溢出时用 `Math.addExact/subtractExact/multiplyExact` (溢出即抛异常,至少能在测试期暴露)。

2.1.2 除法与取模:与 C 完全一致,但多了工具

Java 的 `/` 与 `%` 对整数的语义与 C 完全一致:向零取整,取模符号跟随被除数。C 没有的工具是 `Math.floorDiv / Math.floorMod`:提供“向下取整”语义(Python 风格),循环索引、环形缓冲等场景常用。

示例 2-2 除法与取模

```
public class DivModDemo {
    public static void main(String[] args) {
        // Java 的 / 和 %:向零取整(与 C 完全一致)
        System.out.println(7 / 2);           // 3
        System.out.println(-7 / 2);          // -3(向零,不是向下)
        System.out.println(7 % 2);           // 1
        System.out.println(-7 % 2);          // -1(符号跟随被除数,与 C 一致)
        System.out.println(7 % -2);          // 1

        // Math.floorMod/floorDiv:向下取整语义(相当于 Python 的 // 与 %)
        System.out.println(Math.floorDiv(-7, 2)); // -4(向下)
        System.out.println(Math.floorMod(-7, 2)); // 1(恒为非负)

        // 浮点取模也可用 %
        System.out.println(7.5 % 2);          // 1.5
    }
}
```

语义	C 的写法	Java 的写法
有符号整数溢出	未定义行为(编译器任意发挥)	补码回绕,行为确定;可用 Math.addExact 检测
整数除零	UB(通常崩溃)	抛 ArithmeticException,可捕获处理
浮点除零	实现定义(通常 ±Inf)	明确为 Infinity / NaN
整除与取模	向零取整,符号跟随被除数	相同;另有 floorDiv/floorMod 提供向下取整
求幂	pow(x, y)(double)	Math.pow(double, double);整型幂需自写或 BigInteger

 **工程建议:取模的正确用法**

环形队列、哈希桶、时间换算等场景中,需要"结果恒为非负"时用 `Math.floorMod`,而不是 `%`。例如按 24 小时制换算昨天:今天是 3 点, `(3 - 5) % 24` 得 -2,而 `Math.floorMod(3 - 5, 24)` 得 22——后者才是"昨天 22 点"。

§2.2 自增自减与复合赋值

2.2.1 前置与后置:与 C 相同,但要小心表达式

`++i / --i` (先变后用)与 `i++ / i--` (先用后变)的语义与 C 完全一致,而且同一表达式内的求值顺序是确定的(从左到右),不会出现 C 的 `i = i++ + ++i` 那种 UB。Java 风格建议:自增自减主要用于独立语句与循环,避免塞进复杂表达式——这是团队规范与 C 老手的共同经验。

2.2.2 复合赋值的隐藏强转(经典陷阱)

示例 2-3 复合赋值陷阱

```

public class CompoundDemo {
    public static void main(String[] args) {
        short s = 1;
        s += 1;           // 合法! s += 1 等价于 s = (short)(s + 1), 自带强转
        // s = s + 1;      // 编译错误: int 不能赋给 short

        int i = 5;
        i += 0.9;          // 合法: 复合赋值做隐式收窄, i 变成 5 (小数被截断)
        System.out.println(s + " " + i);    // 2 5

        // 自增自减: 与 C 相同, 先变后用 / 先用后变
        int a = 5;
        int b = a++;        // b = 5, a = 6
        int c = ++a;        // c = 7, a = 7
        System.out.println(a + " " + b + " " + c);    // 7 5 7

        // 浮点累加陷阱: 不要用 == 判断浮点结束
        double x = 0;
        // for (x = 0; x != 1; x += 0.1) {} // 0.1 不精确, 可能死循环
        for (x = 0; x < 1; x += 0.1) {}           // 用 < 而不是 !=
        System.out.println(x);                     // 1.0999999999999999
    }
}

```

⚠ 易错点 2: `i += 0.9` 不报错, 结果却是 5

复合赋值 `E1 op= E2` 等价于 `E1 = (T)(E1 op E2)`, 其中 T 是 E1 的类型——**自带一次强转**。所以 `int i = 5; i += 0.9;` 编译通过且结果为 5 (截断), 而 `i = i + 0.9` 编译错误。生产上遇到“浮点混进整数累加”要特别警惕: 类型检查放过了它, 精度却丢了。

§2.3 位运算与移位

2.3.1 全套位运算: C 有的都有, C 没有的也有

Java 支持 C 的全部位运算符: `&` (与)、`|` (或)、`^` (异或)、`~` (取反), 以及 `<<` (左移)、`>>` (算术右移, 补符号位)。C 没有的是 `>>>` (**无符号右移, 高位补 0**)——因为 C 没有 unsigned 与 signed 之争以外的第三种选择, Java 的 `int` 全是有符号, `>>>` 让“只关心低位”的场景不必再折腾符号位。

示例 2-4 位运算与三种移位

```

public class BitDemo {
    public static void main(String[] args) {
        int a = 0b1100;    // 12
        int b = 0b1010;    // 10
        System.out.println(a & b);    // 8 (1000)按位与
        System.out.println(a | b);    // 14 (1110)按位或
        System.out.println(a ^ b);    // 6 (0110)按位异或
        System.out.println(~a);       // -13(按位取反, 含符号位)

        // 移位: << 左移(等效乘 2^n); >> 算术右移(补符号位); >>> 逻辑右移(补 0)
        int n = 1;
        System.out.println(n << 4);    // 16: 左移 4 位 = 乘 16
        System.out.println(-16 >> 2);  // -4: 符号位保留
        System.out.println(-16 >>> 2); // 1073741820: 高位补 0

        // 移位距离按 32(int)/64(long) 取模: 1 << 32 等于 1 << 0
        System.out.println(1 << 32);   // 1(不是 0!)
        System.out.println(1 << 33);   // 2(等价 1 << 1)
    }
}

```

⚠ 易错点 3:移位距离取模

Java 规定 `int` 的移位距离按 32 取模、`long` 按 64 取模：`1 << 32` 实际是 `1 << 0`，结果是 1。C 中 `1 << 32` 是 UB；Java 里它是“确定但违反直觉”的值。循环中用 `i < 32` 做掩码生成时，`1 << i` 在 `i=32` 时悄悄回到 1——建议循环上限写成 `i < 32` 且掩码用 `1L << i` (64 位)或用 `Math.floorMod` 显式控制。

2.3.2 生产场景:标志位

示例 2-5 权限标志位(C 老手艺的 Java 版)

```
public class FlagDemo {
    // 用一个 int 的多个位保存多个开关:只读/可写/可执行
    static final int READ = 1 << 0;    // 0001
    static final int WRITE = 1 << 1;    // 0010
    static final int EXEC = 1 << 2;     // 0100

    public static void main(String[] args) {
        int perm = READ | WRITE;        // 置位:读 + 写 = 0011
        System.out.println((perm & READ) != 0); // true:检查某位
        System.out.println((perm & EXEC) != 0); // false
        perm |= EXEC;                   // 追加执行权限:0111
        perm &= ~WRITE;                 // 清除写权限:0101
        System.out.println(Integer.toBinaryString(perm)); // 101

        // 交换两个数(教学演示,生产中直接用临时变量更清晰)
        int x = 3, y = 5;
        x ^= y; y ^= x; x ^= y;
        System.out.println(x + " " + y); // 5 3
    }
}
```

🚀 工程建议:位运算优先用于“底层与性能敏感”代码

标志位、哈希、掩码、颜色通道(ARGB)是位运算的用武之地;但业务代码中能用枚举与布尔字段表达就优先用枚举——第1章讲过,可读性与类型安全更重要。位运算出现在:协议解析、位图索引、权限位、整数哈希。

§2.4 逻辑运算与短路

2.4.1 操作数必须是 boolean

`&&` (短路与)、`||` (短路或)、`!` (非)与 C 的语义相同,但要求两侧是 `boolean`。`if (x)` (`x` 为整数)在 Java 中编译失败,必须写成 `if (x != 0)`。这不是麻烦,而是把“非 0 即真”的隐式转换从语言里拿掉了——C 程序里 `if (ptr = malloc(...))` 这类 bug 在 Java 里不可能发生。

示例 2-6 短路求值与判空模式

```

public class ShortCircuitDemo {
    public static void main(String[] args) {
        // 短路:左侧已决定结果时,右侧不执行
        int n = 0;
        boolean ok = (n != 0) && (10 / n > 1); // 左侧 false,不会除零
        System.out.println(ok);                // false

        // 经典判空模式:C 里是 if (s != NULL && strlen(s) > 3)
        String s = null;
        if (s != null && s.length() > 3) {
            System.out.println("长度大于 3");
        }

        // || 同样短路:右侧函数不会执行
        boolean r = true || expensive();
        System.out.println(r);

        // 非短路版本 & | 也支持(不常用):两侧都会执行
        // boolean bad = false & expensive(); // 两侧都算
    }

    static boolean expensive() {
        System.out.println("不会执行到这里");
        return false;
    }
}

```

⚠ 易错点 4:把 & 当 && 用

& 对 boolean 也合法,但不短路: `s != null & s.length() > 3` 在 `s` 为 `null` 时照样调用 `s.length()`,直接 `NullPointerException`。判空 + 后续操作必须用 `&&`;同理 `|` 与 `||`。生产上 `&/|` 只用于位运算(整数),不要用来做逻辑判断。

§2.5 比较与相等:== 的两种世界

2.5.1 值比较与引用比较

Java 的 `==` 语义取决于操作数类型:基本类型比较值,引用类型比较"是否指向同一对象"。C 程序员最容易踩的坑:用 `==` 比较 `String` 或包装类(第1章已详述),这里补充浮点比较的正确姿势——`==` 对浮点"精确到每一位",而 `0.1 + 0.2` 这种运算结果与字面量 `0.3` 的二进制表示不同,直接 `==` 是 `false`。

示例 2-7 浮点比较的正确姿势

```

public class FloatCompareDemo {
    public static void main(String[] args) {
        // 0.1 + 0.2 != 0.3: 二进制浮点不精确(与 C 相同,但 Java 更常踩中)
        double a = 0.1 + 0.2;
        System.out.println(a);           // 0.30000000000000004
        System.out.println(a == 0.3);    // false!

        // 生产姿势一: 误差范围(epsilon)比较
        double eps = 1e-9;
        System.out.println(Math.abs(a - 0.3) < eps); // true

        // 生产姿势二: 金额用整数(以分为单位)比较,天然精确
        long centsA = 10, centsB = 20, centsSum = 30;
        System.out.println(centsA + centsB == centsSum); // true

        // 特殊值: NaN 不等于任何数; 正负零相等
        System.out.println(Double.NaN == Double.NaN);    // false
        System.out.println(0.0 == -0.0);                // true
        System.out.println(Double.POSITIVE_INFINITY > 1e300); // true
    }
}

```

2.5.2 instanceof: 类型判断与模式匹配

C 用 `switch` 配合枚举或函数指针表模拟"运行时类型分派"; Java 有 `instanceof` 运算符。Java 16 起支持**模式匹配**: `o instanceof String s` 在判断为真的同时声明变量 `s`, 省去一次强转——这是生产代码中最常用的新写法之一。

示例 2-8 instanceof 模式匹配

```

public class InstanceofDemo {
    public static void main(String[] args) {
        Object o = "hello";
        System.out.println(o instanceof String); // true

        // 旧写法: 判断 + 强转(C 风格)
        if (o instanceof String) {
            String s = (String) o;           // 强转
            System.out.println(s.toUpperCase());
        }

        // 新模式匹配(Java 16+): 判断即声明, 无需强转
        if (o instanceof String s && s.length() > 3) {
            System.out.println(s.toUpperCase()); // HELLO
        }

        // 注意: null instanceof 任何类型都是 false, 天然防空
        System.out.println(null instanceof String); // false
    }
}

```

§2.6 三元、优先级与求值顺序

2.6.1 三元运算符 ?:

语法与 C 相同: 条件 ? 表达式A : 表达式B, **只计算选中的分支**。Java 对两个分支的类型有统一要求(自动找公共类型), `int ? int : double` 的 `int` 会被提升为 `double`, 不要惊讶于类型变化。

2.6.2 完整优先级表(从高到低)

优先级	运算符	结合性
1	() [] . 方法调用	左
2	后置 ++ --	左
3	一元 + - ! ~ 前置 ++ -- (T)强转	右
4	* / %	左
5	+ -(含字符串拼接)	左
6	<< >> >>>	左
7	< <= > >= instanceof	左
8	== !=	左
9	&	左
10	^	左
11		左
12	&&	左
13		左
14	?:	右
15	= += -= *= /= %= <<= >>= >>>= &= ^= =	右

与 C 的差异:新增 >>> (在 6 级)、instanceof(在 7 级)、+ 可用于字符串拼接。结论:把 C 的优先级表平移过来,99% 情况正确。工程惯例:复杂表达式加括号,优先级表只用于读代码。

2.6.3 求值顺序:从左到右,写进规范

示例 2-9 求值顺序演示

```
public class OrderDemo {
    public static void main(String[] args) {
        int r = 2 + 3 * 4;           // 14:优先级决定"先乘后加"
        int r2 = (2 + 3) * 4;        // 20:括号改变结合
        boolean b = 1 < 2 && 3 < 4; // true:比较优先于逻辑
        System.out.println(r + " " + r2 + " " + b);

        // 求值顺序:操作数从左到右求值(C 中是未定义行为!)
        int x = f() + g() * h();      // 输出 f g h:一定从左到右
        System.out.println(x);       // 1 + 2*3 = 7
    }
    static int f() { System.out.print("f "); return 1; }
    static int g() { System.out.print("g "); return 2; }
    static int h() { System.out.print("h "); return 3; }
}
```

⚠ 易错点 5:把 C 的 UB 习惯带进 Java
C 里 `a[i++] = i++` 是 UB;Java 里它是定义良好的(从左到右求值),但结果仍是反直觉的。规范规定求值顺序≠鼓励你写这种代码。生产铁律:一个表达式里最多一次自增/自减,副作用(赋值、调用)独立成句,这是 C 与 Java 通用的代码规范。

§2.7 字符串拼接与 Math 工具速览

2.7.1 "+" 的字符串魔法

Java 的 `+` 遇到字符串时执行拼接, 两侧任意类型都会被转成字符串; 拼接顺序从左到右, `1 + 2 + "3"` 是 `"33"`, `"1" + 2 + 3` 是 `"123"` ——这个差异是面试必考题。循环内大量拼接必须用 `StringBuilder`(第1章已述)。

2.7.2 Math 工具:比 C 的 `math.h` 更顺手

`Math.abs` (注意 `Integer.MIN_VALUE` 取绝对值仍是负数, 回绕!)、`Math.max/min` (C 要手写宏)、`Math.pow/sqrt/exp/log`、`Math.floor/ceil/round` (`round` 是"四舍五入"但 `Math.round(-1.5)` 得 `-1`, 向正无穷)、`Math.random()` (`0~1`)。整数运算专用 `Math.floorDiv/floorMod/addExact` 已在 §2.1 讲过。

示例 2-10 Math 速览与字符串拼接

```
public class MathDemo {
    public static void main(String[] args) {
        // 字符串拼接顺序陷阱
        System.out.println(1 + 2 + "3");    // 33:先算 1+2=3,再拼
        System.out.println("1" + 2 + 3);    // 123:从左到右都是拼

        // Math 常用
        System.out.println(Math.max(3, 5));    // 5
        System.out.println(Math.abs(-7));    // 7
        System.out.println(Math.abs(Integer.MIN_VALUE)); // -2147483648(回绕!)
        System.out.println(Math.pow(2, 10));    // 1024.0(double)
        System.out.println(Math.floor(-1.5));    // -2.0(向下)
        System.out.println(Math.round(-1.5));    // -1(向正无穷,注意)
        System.out.println((int) (Math.random() * 6) + 1); // 1~6 随机数
    }
}
```

工程建议:随机数用 `ThreadLocalRandom`

生产代码中 不要用 `Math.random()`: 它在多线程下有竞争且不支持指定范围。用 `ThreadLocalRandom.current().nextInt(1, 7)` (生成 `1~6`, 左闭右开)。需要密码学安全的随机用 `java.security.SecureRandom` ——这是安全规范的红线。

本章速查表

主题	要点 / 写法
int 溢出	补码回绕(不是 UB);long 参与运算避免;Math.addExact 检测溢出
整数除零	抛 ArithmeticException;浮点除零得 Infinity/NaN
/ 与 %	向零取整;符号跟随被除数(-7%2 = -1);floorDiv/floorMod 向下取整
自增自减	i++ 先用后变 / ++i 先变后用;一个表达式内最多用一次
复合赋值	i += x 自带强转:s += 1 合法,s = s + 1 编译错误;i += 0.9 截断为 5
位运算	& ^ ~ 与 C 相同;只能用于整数
移位	<< 左移;>> 算术右移(补符号位);>>> 逻辑右移(补 0);距离按 32/64 取模
逻辑运算	&& 短路;操作数必须是 boolean;! 取反;判空用 && 不用 &
比较	基本类型比值;引用比对象身份;字符串/包装类用 equals
浮点比较	0.1+0.2 != 0.3;用 a-b < eps 或转整数(分)比较;NaN 用 isNaN
instanceof	o instanceof String s:模式匹配,判断即声明;null 恒为 false
三元 ?:	只算选中的分支;两分支类型会自动找公共类型
求值顺序	操作数从左到右(C 是 UB);副作用独立成句
字符串 +	两侧任意类型转字符串;1+2+"3"="33","1"+2+3="123"
Math	max/min/pow/floor/ceil/round;abs(MIN_VALUE) 仍是负数;random 用 ThreadLocalRandom

最小必会清单

- 能解释“溢出回绕”与 C UB 的区别,并说出两种防护手段(long、Math.addExact)
- 能预测:整数除零、浮点除零、0.0/0.0 的三种行为
- 能说出 -7/2、-7%2、Math.floorDiv(-7,2)、Math.floorMod(-7,2) 四个结果
- 能解释 s += 1 与 s = s + 1 一个合法一个报错的原因
- 能写全三种移位并说出 >>> 的场景;知道 1 << 32 == 1
- 能用 && 写判空模式;知道 & 与 && 在 boolean 上的行为差异
- 能写出浮点比较的两种生产姿势(epsilon、整数化)
- 会写 instanceof 模式匹配并知道 null 的处理
- 能默写优先级表中 && 与 == 与 + 的顺序;能说出求值顺序的规定
- 能预测 1+2+"3" 与 "1"+2+3 的结果

自测题

Q 1. 写出输出: `int max = Integer.MAX_VALUE; System.out.println(max + 1); System.out.println(max + 1L);`

✓ 参考答案

第一行 -2147483648(int 回绕);第二行 2147483648(int 与 long 相加时 int 提升为 long,不溢出)。若想检测溢出,用 `Math.addExact(max, 1)` ,它抛 `ArithmeticException`。

Q 2. 求值: `System.out.println(-7 / 2); System.out.println(-7 % 2); System.out.println(Math.floorMod(-7, 2));`

✓ 参考答案

-3(向零取整,不是向下);-1(符号跟随被除数,与 C 一致);1(floorMod 结果恒为非负)。生产上需要"非负余数"时用 floorMod,如环形索引 `Math.floorMod(i, size)`。

Q 3. 下面的代码编译通过吗?结果是什么? `short s = 1; s += 1; s = s + 1; int i = 5; i += 0.9;`

✓ 参考答案

`s += 1` 编译通过(自带强转), `s = s + 1` 编译失败(int 不能隐式收窄为 short); `i += 0.9` 编译通过且 i 变为 5(小数截断)。复合赋值 = 先运算 + 隐式强转,这是它"放过类型错误"的原因。

Q 4. 求值: `1 << 32`、`-16 >> 2`、`-16 >>> 2`

✓ 参考答案

1(移位距离按 32 取模,等价 `1 << 0`);-4(算术右移保留符号位);1073741820(逻辑右移高位补 0,即 0x3FFFFFFC)。记忆:>> 是"带符号除",>>> 是"无符号位搬运"。

Q 5. 判断对错: `double a = 0.1 + 0.2; a == 0.3` 是 true 还是 false?如何正确比较?

✓ 参考答案

false:0.1 与 0.2 的二进制近似相加得到 0.30000000000000004,与 0.3 字面量的二进制表示不同。正确做法: `Math.abs(a - 0.3) < 1e-9` (误差范围),或金额场景全部用 long 分为单位计算。

Q 6. 下面代码会除零崩溃吗? `int n = 0; boolean ok = (n != 0) && (10 / n > 1);` 如果把 && 换成 & 呢?

✓ 参考答案

用 && 不崩溃:左侧 `n!=0` 为 false,短路,右侧不执行。换成 & 会崩溃:& 不短路,两侧都执行, `10 / 0` 抛 `ArithmeticException`。判空 + 后续访问必须用 &&。

Q 7. 写出 `1 + 2 + "3"` 与 `"1" + 2 + 3` 的结果并解释。

✓ 参考答案

前者 "33":从左到右,先算 `1+2=3`(数值加),再与 "3" 拼接;后者 "123":第一个 + 遇到字符串就开始拼接,后续都是拼。记忆:拼接从"第一次遇到字符串"开始生效。

章末实验题:整数表达式计算器

实验 2:整数表达式计算器

实验目标:实现一个命令行表达式计算器,支持全章运算符并按 Java 优先级求值,正确处理短路。

实验要求:

- 任务 1:支持 $+$ $-$ $*$ $/$ $\%$ 四则与取模,遵循"乘除先于加减"(覆盖 §2.1 算术、§2.6 优先级)
- 任务 2:支持 $<<$ $>>$ 移位与 $\&$ $|$ $\wedge$ $\sim$ 位运算(覆盖 §2.3 位运算)
- 任务 3:支持 $==$ $!=$ $<$ $<=$ $>$ $>=$ 比较,结果为 0/1(覆盖 §2.5 比较)
- 任务 4:支持 $\&\&$ $||$ 逻辑运算,必须真正短路:右侧不再计算(覆盖 §2.4 短路)——注意 Java 的操作数必须是 boolean,这是"从 C 到 Java"的现场练习
- 任务 5:支持一元 $!$ $\sim$ 与括号(覆盖 §2.6 优先级、§2.2 单目结合性)
- 任务 6:输入 exit 退出;除零等错误用 try-catch 捕获并提示(覆盖 §2.1 除零语义)

实验环境:JDK 17+; `javac Calculator.java` 编译, `java Calculator` 运行。测试用例: $2+3*4=14$ 、 $(2+3)*4=20$ 、 $1<<4=16$ 、 $1+2==3 \ \&\& \ 5>3=1$ 、 $0 \ \&\& \ 1/0$ 不应报错。

第一步 实现"词法+语法"骨架: `expr/term/factor` 三层递归下降,只支持加减乘除与括号,跑通 $2+3*4$ 与 $(2+3)*4$

第二步 在正确的层级插入移位、位运算、比较与逻辑:优先级从低到高为 $|| \rightarrow \&\& \rightarrow | \rightarrow \wedge \rightarrow \& \rightarrow == \ != \rightarrow$ 关系 $\rightarrow$ 移位 $\rightarrow$ 加减 $\rightarrow$ 乘除 $\rightarrow$ 一元 $\rightarrow$ 括号,每一层一个方法

第三步 路实现:逻辑层用 `v != 0 && and() != 0 ? 1 : 0` 的写法——Java 的 `&&` 自带短路,左侧为假时右侧递归根本不会执行;用 `0 && 1/0` 验证

参考答案要点

```

import java.util.Scanner;

/**
 * 整数表达式计算器 — 第2章综合实验参考答案
 * 覆盖:算术/取模、位运算、移位、比较、逻辑短路、一元运算、优先级与括号
 * 真值规则:按 C 风格,非 0 即真,比较与逻辑的结果为 0 或 1
 */
public class Calculator {
    private final String src;
    private int pos;

    public Calculator(String src) { this.src = src; }

    private int expr() { return or(); } // 顶层

    // 优先级从低到高,每层一个方法(递归下降)
    private int or() {
        int v = and();
        while (match("||")) {
            v = (v != 0 || and() != 0) ? 1 : 0; // 短路:左侧真则右侧不执行
        }
        return v;
    }

    private int and() {
        int v = bitOr();
        while (match("&")) {
            v = (v != 0 && bitOr() != 0) ? 1 : 0; // 短路:左侧假则右侧不执行
        }
        return v;
    }

    private int bitOr() {
        int v = bitXor();
        while (match("|")) { v = v | bitXor(); }
        return v;
    }

    private int bitXor() {
        int v = bitAnd();
        while (match("^")) { v = v ^ bitAnd(); }
        return v;
    }

    private int bitAnd() {
        int v = equality();
        while (match("&")) { v = v & equality(); }
        return v;
    }

    private int equality() {
        int v = relational();
        while (true) {
            if (match("==")) { v = (v == relational()) ? 1 : 0; }
            else if (match("!=")) { v = (v != relational()) ? 1 : 0; }
            else { return v; }
        }
    }

    private int relational() {
        int v = shift();
        while (true) {
            if (match("<=")) { v = (v <= shift()) ? 1 : 0; }
            else if (match(">=")) { v = (v >= shift()) ? 1 : 0; }
            else if (match("<")) { v = (v < shift()) ? 1 : 0; }
            else if (match(">")) { v = (v > shift()) ? 1 : 0; }
            else { return v; }
        }
    }
}

```

```

}

private int shift() {
    int v = additive();
    while (true) {
        if (match("<<")) { v = v << additive(); }
        else if (match(">>")) { v = v >> additive(); }
        else { return v; }
    }
}

private int additive() {
    int v = multiplicative();
    while (true) {
        if (match("+")) { v = v + multiplicative(); }
        else if (match("-")) { v = v - multiplicative(); }
        else { return v; }
    }
}

private int multiplicative() {
    int v = unary();
    while (true) {
        if (match("*")) { v = v * unary(); }
        else if (match("/")) { v = v / unary(); } // 除零 -> ArithmeticException
        else if (match("%")) { v = v % unary(); }
        else { return v; }
    }
}

private int unary() {
    if (match("-")) { return -unary(); } // 一元负号(右结合)
    if (match("!")) { return (unary() == 0) ? 1 : 0; } // 逻辑非
    if (match("~")) { return ~unary(); } // 按位取反
    return primary();
}

private int primary() {
    if (match("(")) {
        int v = expr();
        match(")"); // 忽略不匹配的右括号
        return v;
    }
    return number();
}

private int number() {
    int v = 0;
    while (pos < src.length() && Character.isDigit(src.charAt(pos))) {
        v = v * 10 + (src.charAt(pos) - '0');
        pos++;
    }
    return v;
}

// 尝试匹配运算符;匹配成功则前进并返回 true
private boolean match(String op) {
    if (src.startsWith(op, pos)) {
        pos += op.length();
        return true;
    }
    return false;
}

public static void main(String[] args) {
    Scanner in = new Scanner(System.in);
    System.out.println("整数表达式计算器(输入 exit 退出)");
    System.out.println("支持: + - * / % << >> & | ^ ~ == != < <= > >= && || ! 括号");
    while (true) {

```

```

        System.out.print("> ");
        String line = in.nextLine().trim();
        if (line.equals("exit")) { break; }
        try {
            Calculator c = new Calculator(line);
            System.out.println("=" + c.expr());
        } catch (ArithmeticException e) {
            System.out.println("错误:整数除零抛 ArithmeticException(C 中这是 UB)");
        } catch (Exception e) {
            System.out.println("语法错误:" + e);
        }
    }
    in.close();
}
}

```

设计说明:递归下降是"优先级"最直接的代码化:每个优先级一层方法,方法体是"先解析上一层,再循环匹配本层运算符"。短路的关键在 `v != 0 && and() != 0`:Java 的 `&&` 保证左侧为假时右侧不执行,于是 `0 && 1/0` 不会除零——这同时演示了 §2.4 的"操作数必须是 boolean"这一 C 到 Java 的核心差异。除零由 `v / unary()` 抛出 `ArithmeticException`,在 `main` 中捕获提示。扩展方向:支持变量、支持 `double`、加入幂运算符与函数调用。

下一章预告:第3章 流程控制。表达式算清楚了,就要谈"程序怎么走"——Java 的 `if` 条件、`switch` 的新旧两代语法、三种循环与带标签的 `break/continue`。

第3章 流程控制

建议学时:3-5 小时 | 难度:★★☆☆☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 掌握 if/else 的 boolean 强约束,理解"悬挂 else"规则与 C 的差异
- 掌握 switch 的两代语法:传统 fallthrough 版与箭头版,以及 C 的 switch 三个缺陷如何被修复
- 掌握 switch 表达式与 yield,学会用表达式替换 if-else 链
- 掌握 while/do-while/for 三种循环与增强 for,能与 C 的三种循环逐条对照
- 掌握 break/continue 与带标签跳转,知道 Java 的 goto 是保留字
- 能写出菜单驱动的控制台程序,正确组合循环与跳转
- 理解循环变量捕获(effectively final)约束,避免匿名类与 lambda 的编译错误

💡 从 C 到 Java:本章主题如何迁移

流程控制在 Java 里"形似而神不似":语句长相几乎相同,但约束与陷阱完全不同。

第一,条件必须是 boolean。 C 的 `if (x)` 接受任意表达式,0 为假、非 0 为真;Java 只接受 boolean。`if (x = 5)` 在 C 里是经典 bug,在 Java 里是编译错误。代价:写比较时多敲两下;收益:"赋值当比较"这类事故从语言层面灭绝。

第二,switch 被彻底重做。 C 的 switch 有三大缺陷:只能整型、默认 fallthrough、无作用域隔离(变量声明容易串)。Java 首先放开类型(字符串、枚举),然后用箭头语法 `case x ->` 消灭 fallthrough,最后把 switch 升级为"表达式"(可以给变量赋值,还能 yield 返回值)——现代 Java 的 switch 已经是一种**模式匹配**工具(第14章),与 C 的"跳转表"完全是两个物种。

第三,goto 的态度。 C 的 goto 用得好可以简化错误处理,用不好就是意大利面;Java 干脆把 `goto` 定为保留字禁止使用,用**带标签的 break/continue**覆盖"跳出多层循环"这一 goto 的正当用途。想做的事一样,工具换成了安全的。

本章策略:C 有的先对照(if/while/for 语法几乎零成本迁移),然后重点掌握 switch 的新语法——这是本章最值钱的部分。

📦 前置知识自查

- C 的 if/else、while、do-while、for 语法与 break/continue
- C 的 switch-case 与 fallthrough 行为(忘记 break 的后果)
- C 的三目表达式与常见循环模式(计数器、哨兵值、遍历数组)
- 第1章知识:boolean 类型、枚举 enum、数组与 length 属性

🚀 本章学习路线

1. **第一阶段(约 1 小时,快扫):**§3.1 与 §3.3 与 C 几乎相同,对照表格过一遍即可,重点看"差异"列
2. **第二阶段(约 1.5 小时,攻坚):**§3.2 的 switch 三讲(传统版、箭头版、表达式版)逐段运行,对比 C 的写法;§3.4 的标签跳转做一遍二维查找
3. **第三阶段(实验,约 1 小时):**独立完成章末实验"菜单驱动的数据统计程序",用自测题验收
4. **验收标准:**能不看资料写出:箭头版 switch、switch 表达式(含 yield)、带标签 break 的二维查找

本章知识导图

- §3.1 if/else:boolean 强约束 / 悬挂 else / if-else 链
- §3.2 switch:从 C 的缺陷到 Java 的新语法(传统版 / 箭头版 / switch 表达式 / 字符串与枚举)
- §3.3 三种循环:while / do-while / for / 增强 for / 循环变量捕获
- §3.4 break、continue 与带标签跳转(goto 保留字 / 跳出多层循环)
- §3.5 无限循环写法与生产实践(菜单驱动 / 分支策略选型)

§3.1 if/else:条件必须是 boolean

3.1.1 与 C 的对照

语法与 C 完全相同: `if (条件) 语句; else 语句;`。唯一差异:条件必须是 `boolean` 表达式。`if (1)`、`if (ptr)`、`if (n)` 在 Java 里全部编译失败,必须显式写成 `if (n != 0)`、`if (ptr != null)`。这带来一个连锁好处:`if (n = 5)` 不可能通过编译——因为 `n = 5` 的类型是 `int` 不是 `boolean`,编译器直接拦下。

示例 3-1 if/else 与 boolean 条件

```
public class IfDemo {
    public static void main(String[] args) {
        int score = 75;
        // C: if (score) 合法;Java:必须显式比较
        if (score >= 60) {
            System.out.println("及格");
        } else {
            System.out.println("不及格");
        }
        // if (score) {}           // 编译错误:不是 boolean
        // if (score = 60) {}      // 编译错误:赋值表达式是 int

        // 三目与 if 等价(C 程序员熟知的写法)
        String level = score >= 60 ? "及格" : "不及格";
        System.out.println(level);
    }
}
```

3.1.2 悬挂 else 与 if-else 链

悬挂 `else`(dangling else)规则与 C 相同:`else` 与最近的未配对 `if` 结合。Java 的缩进规范(官方风格 4 空格)要求 `if/else` 必须带花括号,从格式上消灭了悬挂 `else` 的阅读歧义。多分支时用 `if-else` 链;分支多(≥ 4)且是"单值分派"时,应该改用 §3.2 的 `switch`——这是本章最重要的工程判断。

主题	C 的写法	Java 的写法
if 条件	任意数值,0 为假非 0 为真	必须 <code>boolean</code> ; <code>if (x)</code> 报编译错误
赋值当比较	<code>if (n = 5)</code> 合法,经典 bug	编译错误,无法写出
悬挂 else	与最近 <code>if</code> 结合(需小心)	规则相同,但规范强制花括号
多分支	<code>if-else</code> 链或 <code>switch</code>	<code>if-else</code> 链、 <code>switch</code> 或 <code>switch</code> 表达式

⚠ 易错点 1:else 就近配对

`if (a) if (b) x(); else y();` 中 `else` 属于内层 `if (b)` 还是外层? 属于内层 `if (b)` (就近配对),与 C 相同。想要 `else` 属于外层,必须写 `if (a) { if (b) x(); } else y();`。Java 官方风格强制花括号,这类歧义在规范代码中不存在,但阅读别人的“紧凑代码”时仍要警惕。

§3.2 switch:从 C 的缺陷到 Java 的新语法

3.2.1 回顾 C 的 switch 三大缺陷

C 的 `switch` 是跳转表:① 表达式只能是整型(不能是字符串);② **fallthrough**——漏写 `break` 会一路执行下去;③ `case` 里的变量声明作用域混乱。Java 的演进分三步:先放开类型(Java 5 枚举、Java 7 字符串),再引入箭头语法消灭 `fallthrough`(Java 14),最后升级为表达式(Java 14,支持 `yield` 与模式匹配)。

3.2.2 传统写法(与 C 对照)

示例 3-2 传统 `switch`:必须 `break`(与 C 相同)

```
public class SwitchOldDemo {
    public static void main(String[] args) {
        int day = 3;
        String name;
        switch (day) {
            case 1:
                name = "周一";
                break;           // 漏掉 break 就 fallthrough(与 C 相同)
            case 2:
                name = "周二";
                break;
            case 3:
                name = "周三";
                break;
            default:
                name = "未知";
        }
        System.out.println(name);    // 周三

        // 字符串与枚举也可以 switch(Java 7/5 起)
        String fruit = "apple";
        switch (fruit) {             // 内部用 equals 比较
            case "apple":
                System.out.println("苹果");
                break;
            case "banana":
                System.out.println("香蕉");
                break;
            default:
                System.out.println("其他水果");
        }
        // 枚举 switch:case 不用写枚举名
        // switch (season) { case SPRING: ... }
    }
}
```

⚠ 易错点 2:忘记 break 的 fallthrough

C 的经典事故在 Java 里同样存在(传统写法下): `case` 执行完没有 `break`,会继续执行下一个 `case`。生产代码建议:新代码一律用下面的箭头语法,编译器会强制你为每个 `case` 写语句,fallthrough 从机制上消失。

3.2.3 箭头语法(Java 14+):推荐写法

示例 3-3 箭头版 switch:无 fallthrough

```
public class SwitchNewDemo {
    public static void main(String[] args) {
        int day = 3;
        String name;
        switch (day) {
            case 1 -> name = "周一";           // 箭头后是单条语句,无需 break
            case 2 -> name = "周二";
            case 3 -> name = "周三";
            default -> name = "未知";
        }
        System.out.println(name);

        // 多值标签:一个 case 匹配多个值(逗号分隔)
        String season = "夏";
        switch (season) {
            case "春", "夏" -> System.out.println("温暖");
            case "秋", "冬" -> System.out.println("凉爽");
            default -> System.out.println("未知季节");
        }
        // 箭头后可以是代码块(需要多条语句时)
        switch (day) {
            case 1 -> {
                System.out.println("周一");
                System.out.println("新的一周");
            }
            default -> System.out.println("其他");
        }
    }
}
```

3.2.4 switch 表达式:switch 可以有返回值

示例 3-4 switch 表达式与 yield

```

public class SwitchExprDemo {
    public static void main(String[] args) {
        int day = 3;
        // switch 表达式:整体是一个值,可以直接赋值
        String name = switch (day) {
            case 1 -> "周一";
            case 2 -> "周二";
            case 3 -> "周三";
            default -> "未知";
        };
        System.out.println(name);    // 周三

        // 需要多条语句的分支:用 yield 返回值
        int month = 2;
        int days = switch (month) {
            case 1, 3, 5, 7, 8, 10, 12 -> 31;
            case 4, 6, 9, 11 -> 30;
            case 2 -> {
                int leap = (month % 4 == 0) ? 29 : 28;    // 简化,勿用于真实历法
                yield leap;                            // yield 返回值
            }
            default -> -1;
        };
        System.out.println(days);    // 28

        // 枚举 switch 表达式:穷尽所有枚举值时可以省略 default
        // Season s = Season.SPRING;
        // String desc = switch (s) {
        //     case SPRING -> "春";
        //     case SUMMER -> "夏";
        //     case AUTUMN -> "秋";
        //     case WINTER -> "冬";
        // };    // 枚举分支齐全,无需 default,编译器能验证穷尽性
    }
}

```

★ 重点:switch 表达式的三条铁律

- 表达式版本 必须穷尽 :所有可能值都有分支,否则加 default(枚举全列完可省略,编译器可验证)
- 每个分支必须"产生一个值":箭头后是表达式,或代码块以 yield 结尾
- 箭头版不会 fallthrough: case 之间不需要 break,多标签用逗号 ——与 C 的心智模型完全不同

🚀 工程建议:分支选型三问

写分支前问三个问题: ① 是"单值分派"吗?(按一个变量的取值走不同逻辑)→ 用 switch; ② 是"范围/复合条件"吗?(score >= 60 这种)→ 用 if-else 链; ③ 是"类型分派"吗?(按对象类型走不同逻辑)→ 用 instanceof 模式匹配(第2章)或第14章的模式匹配 switch。选对工具比写对语法更重要。

§3.3 三种循环:while、do-while 与 for

3.3.1 与 C 逐条对照

Java 的三种循环语法与 C 几乎逐字相同: while (条件)、do 语句 while (条件);、for (初始化; 条件; 更新)。差异有三: ① 条件必须是 boolean; ② for 的循环变量可以声明为 var; ③ 新增增强 for(遍历数组与集合)。do-while 在 Java 里仍然存在(区别于 Go/Rust 等去掉了它的语言)。

场景	C 的写法	Java 的写法
计数循环	for (i = 0; i < n; i++)	for (int i = 0; i < n; i++)(条件须 boolean)
条件循环	while (n > 0)	while (n > 0)(相同)
至少执行一次	do { } while (...)	do { } while (...)(相同)
遍历数组	for (i = 0; i < N; i++) 用 a[i]	for (int x : a)(增强 for,无需下标)
遍历需要下标	同上	用传统 for 或 IntStream.range(第4章)

示例 3-5 三种循环与增强 for

```
public class LoopDemo {
    public static void main(String[] args) {
        // while:经典"哨兵值"模式(C 相同)
        int sum = 0, n = 1;
        while (n <= 100) {
            sum += n;
            n++;
        }
        System.out.println("1..100 之和 = " + sum);    // 5050

        // do-while:至少执行一次(用户输入校验)
        int input;
        do {
            System.out.print("请输入 1~7: ");
            input = 7;           // 真实代码里从控制台读
        } while (input < 1 || input > 7);
        System.out.println("OK: " + input);

        // for:与 C 完全一致;变量可声明为 var
        for (int i = 0; i < 5; i++) {
            System.out.print(i + " ");
        }
        System.out.println();

        // 增强 for:遍历数组(不需要下标时首选)
        int[] scores = {85, 92, 78, 66, 99};
        int total = 0;
        for (int s : scores) {
            total += s;
        }
        System.out.println("平均分 = " + (double) total / scores.length);
    }
}
```

3.3.2 循环变量捕获:effectively final

在 C 里,循环变量可以随便被内层函数取地址;Java 的 lambda 与匿名类(第4、6章)捕获局部变量时有一个约束: 被捕获的变量必须是 final 或"实际上的 final"(赋值后不再改变)。经典写法是把循环变量复制一份:

示例 3-6 循环变量捕获

```
import java.util.ArrayList;
import java.util.List;

public class CaptureDemo {
    public static void main(String[] args) {
        List<Runnable> tasks = new ArrayList<>();
        for (int i = 0; i < 3; i++) {
            int copy = i;                // effectively final 的副本
            tasks.add(() -> System.out.print(copy + " "));
        }
        for (Runnable t : tasks) {
            t.run();                    // 0 1 2
        }
        System.out.println();
        // 若直接捕获 i:编译错误 "local variables referenced from
        // a lambda expression must be final or effectively final"
    }
}
```

⚠ 易错点 3:lambda 捕获循环变量

很多语言(如 C 的闭包扩展、JS 的 var)在循环里捕获循环变量会得到"全部是最后一个值";Java 的解法更彻底:捕获必须 **effectively final**,非法的直接编译失败。惯例:循环内定义 `int copy = i;` 再捕获。这个约束也顺带防止了"共享可变量"这一并发 bug 温床。

§3.4 break、continue 与带标签跳转

3.4.1 单层跳转:与 C 相同

`break` 跳出当前循环或 `switch`; `continue` 跳过本轮进入下一轮——语义与 C 完全一致。Java 的 `continue` 不能用于 `switch`(会报错,因为 `switch` 不是循环)。

3.4.2 带标签的 break/continue:goto 的替代品

`goto` 是 Java 的保留字,不可使用。跳出多层嵌套循环的正当需求由**带标签的 break/continue** 承担:给外层循环起名 `outer:`,内层写 `break outer;` 直接跳出整个外层循环;`continue` 带标签则跳到外层循环的下一轮。经典场景:二维数组查找。

示例 3-7 二维查找与带标签 break

```

public class LabelDemo {
    public static void main(String[] args) {
        int[][] grid = {
            {1, 2, 3},
            {4, 5, 6},
            {7, 8, 9}
        };
        int target = 5;
        boolean found = false;

        outer:                // 标签:给外层循环命名
        for (int i = 0; i < grid.length; i++) {
            for (int j = 0; j < grid[i].length; j++) {
                if (grid[i][j] == target) {
                    System.out.println("找到 " + target + " 在 (" + i + ", " + j + ")");
                    found = true;
                    break outer;        // 跳出两层循环
                }
            }
        }
        if (!found) {
            System.out.println("未找到");
        }
        // 普通 break 只能跳出一层,内层循环结束后外层还会继续
        // 需要"跳过外层本轮剩余部分"时用 continue outer;
    }
}

```

★ 重点:标签跳转的使用纪律

带标签跳转只用于"跳出多层嵌套"这一种场景,且标签名建议用英文小写(如 `outer:`、`search:`)。不要用它实现"任意跳转"——那等于复活 `goto`,可读性灾难。多层嵌套本身也说明函数太长,重构(提取方法 + 提前 `return`)往往是更优解。

§3.5 无限循环与生产实践

3.5.1 无限循环写法对照

风格	C 的写法	Java 的写法
经典无限循环	<code>while (1) { }</code>	<code>while (true) { }</code> (或 <code>for(;;)</code>)(1 不是 boolean!)
服务主循环	<code>for (;;) { ... }</code>	<code>while (true) { ... }</code> (等价)
退出方式	<code>break / goto / return</code>	<code>break / return / 抛异常</code> (第9章)

3.5.2 菜单驱动的控制台程序:流程控制的集大成

生产中最常见的"流程控制全家桶"就是菜单程序:`while` 包住整个交互循环,`switch` 分派菜单项,`do-while` 校验输入,`for`/增强 `for` 处理数据,`break`/`return` 退出。第1章的 `Scanner` 用法在这里继续使用。

示例 3-8 菜单程序骨架(实验题的最小版本)

```
import java.util.Scanner;

public class MenuSkeleton {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        while (true) { // 主循环:菜单反复出现
            System.out.println("1. 录入 2. 统计 3. 退出");
            System.out.print("请选择: ");
            int choice = Integer.parseInt(in.nextLine());
            switch (choice) {
                case 1 -> System.out.println("录入功能");
                case 2 -> System.out.println("统计功能");
                case 3 -> {
                    System.out.println("再见");
                    return; // 结束整个程序
                }
                default -> System.out.println("无效选项");
            }
        }
    }
}
```

工程建议:交互输入与健壮性

真实生产里交互程序注意三点: ① 解析输入必须处理异常 (Integer.parseInt 遇到非数字会抛异常,第9章学习完整处理); ② 不要用 Scanner 混用 nextLine 与 nextInt (换行符残留问题),统一用 nextLine 再解析; ③ 循环必须有明确的退出路径,菜单程序用 return/break,服务程序用关闭标志位——无限循环本身不是问题,没有出口才是。

本章速查表

主题	要点 / 写法
if 条件	必须 boolean;if (x) 编译错误,写 if (x != 0)
悬挂 else	else 就近配对;官方规范强制花括号
传统 switch	case 后必须 break,否则 fallthrough(与 C 相同);可 switch 字符串/枚举
箭头 switch	case x -> 语句;无 fallthrough;多标签 case 1, 2 -> ...(Java 14+)
switch 表达式	switch (x) { ... } 整体是值;多语句分支用 yield;必须穷尽
枚举 switch	case 不写枚举名;分支全列完可省略 default(编译器验证穷尽)
while	与 C 相同;条件必须 boolean
do-while	至少执行一次,与 C 相同(区别于 Go/Rust/Python)
for	for (int i = 0; i < n; i ++);循环变量可 var
增强 for	for (int x : arr) 遍历数组/集合;需要下标时用传统 for
break/continue	单层语义与 C 相同;continue 不能用在 switch
标签跳转	outer: 标签 + break outer;;continue outer;;goto 是保留字
循环捕获	lambda 捕获的变量必须 effectively final;用副本 int copy = i
无限循环	while (true) 或 for(;;);必须有退出路径
分支选型	单值分派用 switch;范围条件用 if-else;类型分派用 instanceof/模式匹配

最小必会清单

- 能解释 if 的 boolean 强约束如何消灭"赋值当比较"bug,并写出等价写法

- 能默写箭头版 switch 与 switch 表达式(含 yield 与多标签)
- 能说出 C 的 switch 三大缺陷及 Java 的修复方式
- 能写出三种循环与增强 for,并说明 do-while 的存在价值(至少执行一次)
- 能写出带标签 break 的二维查找;能说出 goto 为什么不能用
- 能解释 lambda 捕获循环变量的 effectively final 约束与副本技巧
- 能写出 while(true) + switch 的菜单骨架,并设计明确的退出路径
- 能根据"单值/范围/类型"三种场景选择 if、switch 或 instanceof

自测题

Q 1. 下列哪些行能通过编译? if (1) { }, if (flag) { }, if (s.length() > 0) { }, while (n) { }

✅ 参考答案

第二、三行通过; if (1) 与 while (n) 编译失败(1 和 n 不是 boolean)。正确写法 if (n != 0)。这正是"把 C 的真值规则删掉"的结果。

Q 2. 下面的输出是什么? int day = 2; switch (day) { case 1 -> System.out.print("A"); case 2 -> System.out.print("B"); default -> System.out.print("C"); }

✅ 参考答案

只输出 "B"。箭头语法没有 fallthrough,每个分支执行完自动结束,无需 break。若换成传统写法且 case 2 后漏写 break,会输出 "BC"。

Q 3. 用 switch 表达式写出:输入数字 1~3,返回 "低/中/高",其他返回 "未知"。

✅ 参考答案

```
String level = switch (n) {
    case 1 -> "低";
    case 2 -> "中";
    case 3 -> "高";
    default -> "未知";
};
```

Q 4. 这段代码能编译吗?为什么? for (int i = 0; i < 3; i++) { tasks.add(() -> System.out.println(i)); }

✅ 参考答案

不能编译: i 在 lambda 中被引用但会变化,不是 effectively final。修正: int copy = i; 后捕获 copy。这是 Java 与 C(闭包扩展)最大的不同:编译期就禁止捕获可变循环变量。

Q 5. 下面的代码找得到 8 吗? int[][] g = {{1,2},{3,4}}; outer: for (int i = 0; i < 2; i++) { for (int j = 0; j < 2; j++) { if (g[i][j] == 8) break outer; } } 如果找不到,输出什么?

✅ 参考答案

找不到(数组中根本没有 8),但代码不会报错:两层循环都正常结束,break outer 从未执行。若找到则立即跳出两层——带标签 break 只做"跳转",不做"是否存在"的判断,所以需要 found 标志配合,如示例 3-7。

Q 6. 如何判断一个循环是不是"死循环"?给出两个能保证退出的循环写法。

✅ 参考答案

循环体内没有任何 break/return/异常出口,且条件恒真,就是死循环。保证退出的写法: `while (true) { if (exit) break; ... }` (标志位 + break);或 `for (;;) { if (cond) return; }` (直接返回)。菜单程序用 return 退出主循环是最清晰的。

章末实验题:菜单驱动的数据统计程序

📁 实验 3:菜单驱动的数据统计程序

实验目标:编写一个菜单驱动的控制台程序:录入整数、素数筛选、统计汇总,覆盖本章全部流程控制知识点。

实验要求:

- 任务 1:主菜单用 `while (true)` 无限循环 + 箭头版 `switch` 分派,含 default 与退出分支(覆盖 §3.5 无限循环、§3.2 箭头 switch)
- 任务 2:录入功能用 `do-while` 至少读一次,遇到哨兵值 -1 用 `break` 结束(覆盖 §3.3 do-while、§3.4 break)
- 任务 3:素数筛选用双层 `for` (试除法),内层用 `continue` 跳过非候选、`break` 提前结束(覆盖 §3.3 for、§3.4 continue/break)
- 任务 4:统计功能用增强 for 遍历,遇到负数用带标签 `break outer` 中止并提示(覆盖 §3.3 增强 for、§3.4 标签跳转)
- 任务 5:if-else 做输入校验与边界判断(空数据、无效选项)(覆盖 §3.1 if/else)
- 任务 6:退出菜单用 `return`,录入与统计共用同一个数组(覆盖 §3.4 return、数组与枚举基础)

实验环境:JDK 17+; `javac MenuApp.java` 编译, `java MenuApp` 运行。

第一步 搭骨架:main 里的 `while(true)` + `switch`,四个菜单项各一个静态方法,编译通过再填空

第二步 现录入:do-while + 哨兵;注意数组容量上限(`count < data.length` 也要检查)

第三步 现素数筛选与统计:先写普通版本跑通,再为统计加上带标签 `break` 的"遇到负数中止"逻辑,验证标签跳转的效果

✅ 参考答案要点

```

import java.util.Scanner;

/**
 * 菜单驱动的数据统计程序 — 第3章综合实验参考答案
 * 覆盖:while(true)、箭头 switch、do-while、for、增强 for、
 * break、continue、带标签 break、if-else、return
 */
public class MenuApp {
    private static final Scanner in = new Scanner(System.in);
    private static final int MAX = 100;

    public static void main(String[] args) {
        int[] data = new int[MAX];
        int count = 0;

        while (true) {
            // §3.5:主菜单无限循环
            System.out.println("==== 数据统计菜单 =====");
            System.out.println("1. 录入数据(以 -1 结束)");
            System.out.println("2. 素数筛选");
            System.out.println("3. 统计汇总");
            System.out.println("4. 退出");
            System.out.print("请选择(1-4): ");
            int choice = readInt(); // 无效输入返回 -1

            if (choice == -1) {
                // §3.1:输入非法,重新显示菜单
                System.out.println("输入无效,请输入数字 1-4");
                continue; // §3.4:continue 回到菜单
            }
            switch (choice) {
                // §3.2:箭头 switch,无 fallthrough
                case 1 -> count = inputData(data);
                case 2 -> printPrimes(data, count);
                case 3 -> printStats(data, count);
                case 4 -> {
                    System.out.println("再见!");
                    return; // §3.4:return 结束整个程序
                }
                default -> System.out.println("选项无效(1-4)");
            }
        }
    }

    // 读取一个整数;非法输入返回 -1(简化:异常先不处理,第9章详讲)
    static int readInt() {
        try {
            return Integer.parseInt(in.nextLine().trim());
        } catch (NumberFormatException e) {
            return -1;
        }
    }

    // §3.3 do-while:至少执行一次;哨兵 -1 用 break 结束
    static int inputData(int[] data) {
        int count = 0;
        int n;
        do {
            System.out.print("整数(输入 -1 结束): ");
            n = readInt();
            if (n == -1) {
                break; // §3.4:break 跳出 do-while
            }
            data[count++] = n; // 容量检查简化,MAX=100
        } while (count < MAX);
        System.out.println("已录入 " + count + " 个数");
        return count;
    }

    // §3.3 for + §3.4 continue/break:试除法筛素数
    static void printPrimes(int[] data, int count) {

```

```

    if (count == 0) { // §3.1 边界判断
        System.out.println("没有数据");
        return;
    }
    boolean found = false;
    for (int i = 0; i < count; i++) {
        int v = data[i];
        if (v < 2) {
            continue; // 跳过非候选:0、1、负数
        }
        boolean prime = true;
        for (int d = 2; d * d <= v; d++) { // 试除到 sqrt(v) 即可
            if (v % d == 0) {
                prime = false;
                break; // 已确认合数,提前退出内层
            }
        }
        if (prime) {
            System.out.print(v + " ");
            found = true;
        }
    }
    System.out.println(found ? "(以上为素数)" : "(没有素数)");
}

// §3.3 增强 for + §3.4 带标签 break:统计遇到负数立即中止
static void printStats(int[] data, int count) {
    if (count == 0) {
        System.out.println("没有数据");
        return;
    }
    int sum = 0, max = Integer.MIN_VALUE, min = Integer.MAX_VALUE;
    int skipped = 0;
    outer: // §3.4: 标签跳转(goto 的替代)
    for (int v : data) { // 增强 for:遍历整个数组
        if (v < 0) {
            skipped++;
            continue; // 负数跳过统计,继续下一轮
        }
        sum += v;
        max = Math.max(max, v);
        min = Math.min(min, v);
        if (skipped > 3) {
            System.out.println("负数过多,统计中止");
            break outer; // 跳出整个循环
        }
    }
    if (skipped > 3) {
        System.out.println("本次统计无效,请检查数据");
        return;
    }
    int valid = count - skipped;
    System.out.printf("有效数据 %d 个:和=%d 平均=%.1f 最大=%d 最小=%d\n",
        valid, sum, (double) sum / valid, max, min);
}
}

```

设计说明:每个菜单项对应一个方法,main 只做"循环 + 分派",这是菜单程序的标准结构。switch (choice) 用箭头语法,四个分支互不干扰;输入非法时 continue 直接回到菜单顶部,不进入 switch。素数筛选体现"continue 跳过 + break 提前终止"的组合;统计功能用带标签 break 演示"数据异常时整体中止"——这正是 C 里 goto 的经典用途,Java 用标签实现了它。扩展方向:数据存文件(第7章)、多线程统计(第9章)。

下一章预告:第4章 函数与方法。流程控制让程序"走起来",方法让程序"组织起来"——Java 的方法与 C 函数有何异同、参数到底怎么传、lambda 与 Stream 如何改写你的编程习惯。

第4章 函数与方法

建议学时:4-6 小时 | 难度:★★★☆☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 掌握方法声明与 C 函数的对照关系,理解 static 方法与实例方法的区别
- 彻底理解"Java 只有值传递":基本类型与引用类型如何传参,为什么 swap 写不出来
- 掌握方法重载与可变参数 varargs,理解重载解析规则
- 掌握递归与调用栈的关系,会写终止条件清晰的递归
- 掌握 lambda 表达式与函数式接口,会用方法引用简化代码
- 掌握 Stream API 的基本用法:过滤、映射、归约
- 建立"小函数、纯函数"的工程习惯,为第5章内存管理铺路

💡 从 C 到 Java:本章主题如何迁移

C 的函数是"全局自由函数",Java 的方法必须活在类里——这是语法差异,不是哲学差异:你照样可以把类当成"装函数的盒子"(工具类模式),把 static 方法当成 C 函数用。

真正的分水岭是参数传递。C 里传数组等于传指针(退化),传结构体等于复制;Java 里"数组和对象"永远传引用、基本类型永远传值,而且——引用也是按值传的。理解"传引用但是值传递"这句话,是理解整个 Java 参数体系的关键。

重载是 C 没有的能力。C 的 `printf` 用格式串魔法解决"多种参数形态";Java 用重载在编译期解决:同名方法、不同参数列表,编译器按参数精确选择。`printf` 的魔法在 Java 里变成了 `System.out.printf`,但生产代码更常用重载。

函数式编程是 C 程序员最大的增量。C 有函数指针,但回调写起来繁琐;Java 8 的 lambda 让"把行为当参数传递"成为日常,Stream API 则把"循环遍历 + 条件判断"改写成一行声明式代码。本章目标是:能读、能写、能辨别何时用 Stream。

学习顺序建议:先对照 C 的语法差异(\$4.1-4.3),再理解递归与调用栈(\$4.4),最后攻克 lambda 与 Stream(\$4.5-4.6)——前 60% 内容是"老朋友新语法",后 40% 才是 Java 的独家武器。

📁 前置知识自查

- C 函数:声明、定义、形参实参、返回值、递归、函数指针
- C 数组退化与指针传参的语义(实参是数组时到底传了什么)
- 第1章知识:基本类型、数组、字符串;第2章:运算符与表达式
- 第3章:流程控制(循环、分支)——Stream 之前必须熟练

🚀 本章学习路线

1. **第一阶段(约 1.5 小时):**\$4.1-4.3 与 C 对照着读,重点做"swap 陷阱"实验,彻底理解值传递
2. **第二阶段(约 1.5 小时):**\$4.4 递归 + \$4.5 lambda,每个示例敲一遍,体会"行为参数化"
3. **第三阶段(约 1.5 小时):**\$4.6 Stream 对照"循环写法"和"流式写法"逐段运行,再完成章末实验
4. **验收标准:**不看资料写出:swap 失败的完整解释、用 lambda 排序、用 Stream 统计平均分

本章知识导图

- §4.1 方法与 C 函数:声明对照 / static / main 签名
- §4.2 值传递:基本类型传值 / 引用传值 / swap 陷阱
- §4.3 重载与可变参数:重载解析 / varargs 规则
- §4.4 递归与调用栈:终止条件 / 栈帧概念
- §4.5 lambda 与方法引用:函数式接口 / 捕获
- §4.6 Stream API 入门:创建 / 中间操作 / 终止操作
- §4.7 方法设计工程实践:小函数 / 纯函数

§4.1 方法与 C 函数:声明对照

4.1.1 语法对照

主题	C 的写法	Java 的写法
顶层函数	<code>int add(int a, int b) { ... }</code>	必须放类里; <code>static int add(int a, int b)</code>
返回 void	<code>void f() { }</code>	<code>void f() { }</code> (相同)
声明位置	头文件声明 + 源文件定义	方法都定义在类体里,没有头文件
调用	<code>f(x)</code>	<code>f(x)</code> (同包/同文件);其他类用 类名.f(x)
程序入口	<code>int main(int argc, char* argv[])</code>	<code>public static void main(String[] args)</code>
参数个数可变	<code>printf(...)</code> 用变参宏 <code>va_list</code>	<code>varargs:int... nums</code> (语法糖)
同名函数	不允许同名(除非宏)	允许重载:同参数名不同参数列表

示例 4-1 工具类模式:把 C 函数装进类里

```
public class MathUtils {
    // static 方法:相当于 C 的全局函数,不需要对象即可调用
    static int add(int a, int b) {
        return a + b;
    }

    static int max3(int a, int b, int c) {
        return Math.max(Math.max(a, b), c); // 标准库 Math 也是工具类
    }

    public static void main(String[] args) {
        System.out.println(MathUtils.add(2, 3)); // 类名.方法名
        System.out.println(max3(1, 9, 4));       // 同类内可直接调用
    }
}
```

4.1.2 main 的签名

入口方法必须一字不差:`public static void main(String[] args)`。与 C 的 `main(int argc, char* argv[])` 对照:`args` 就是 `argv`, `args.length` 相当于 `argc`(不含程序名),参数从 `args[0]` 开始。`public` 表示 JVM 可以从外部调用它;`static` 表示无需创建对象。

⚠ 易错点 1:main 写错签名的表现

签名写错(比如漏 `static`、把 `args` 写成别的)不会编译报错——JVM 只是找不到入口,运行时报 `Error: Main method not found`。C 里签名写错是编译警告 + 运行崩溃,Java 是"编译通过、运行报错",更隐蔽。写模板时逐字对照:`public static void main(String[] args)`。

§4.2 值传递:Java 最重要的参数语义

4.2.1 一句话结论

Java 只有值传递(`pass-by-value`):实参的值会被复制一份给形参。基本类型复制数值,引用类型复制"引用的拷贝"(两个引用指向同一个对象)。C 程序员最容易误解的点:传对象进去"改得动"(因为引用指向同一对象),但这不叫引用传递——形参本身还是实参的拷贝,在方法里给形参重新赋值,影响不到实参。

示例 4-2 值传递演示:swap 为什么失败

```
public class PassByValueDemo {
    public static void main(String[] args) {
        int a = 3, b = 7;
        swap(a, b);           // 传的是数值拷贝
        System.out.println(a + ", " + b);    // 3, 7:没有交换!

        int[] arr = {10, 20};
        swapArr(arr, 0, 1);    // 数组是引用,能改数组内容
        System.out.println(arr[0] + ", " + arr[1]); // 20, 10:交换了

        String s = "hello";
        changeRef(s);          // 形参被重新赋值,实参不变
        System.out.println(s); // hello:引用拷贝,改形参不影响实参
    }

    // C 里要传 int* 才能交换;Java 基本类型无法交换,只能返回或包装
    static void swap(int x, int y) {
        int t = x;
        x = y;
        y = t;                 // 只改了形参的拷贝
    }

    // 数组是引用:直接修改元素,影响"外面的"数组
    static void swapArr(int[] arr, int i, int j) {
        int t = arr[i];
        arr[i] = arr[j];
        arr[j] = t;
    }

    static void changeRef(String str) {
        str = "world";         // 形参指向新字符串,实参纹丝不动
    }
}
```

传参形态	C 的语义	Java 的语义
基本类型	传值(拷贝)	传值(拷贝,相同)
数组	退化为指针,传地址;可改内容	传引用(引用的拷贝),可改内容;形参重新赋值不影响实参
结构体/对象	默认传值(整个复制);传指针才可改	永远传引用拷贝;可改字段,不能换引用
交换两个变量	传指针 swap(int* a, int* b)	无法实现(只能包装进数组/类或用返回值)
NULL/空值	指针可为 NULL	引用可为 null;使用前判空

★ 重点:一张图理解引用传递

把"引用"想成一根线:线头(变量)和线尾(对象)。传参 = 复制一根新线头,两端都拴着同一个对象。用新线头"拉回一个对象"(重新赋值),老线头不受影响;用新线头"改造对象"(改字段/改数组元素),对象变了,老线头看到的是同一个被改过的对象。这个心智模型可以解释 90% 的传参困惑。

⚠ 易错点 2:把"对象可修改"误读为"引用传递"

看到 `f(p)` 里改了 `p` 的字段、外面也变了,就以为 Java 是引用传递——错。测试法:在方法里执行 `p = new ...;`,如果外面跟着变,才是引用传递;Java 外面不变,所以是值传递(传的是引用的拷贝)。面试高频题,务必能说清。

§4.3 方法重载与可变参数

4.3.1 重载:同名不同参

同一类中可以有多个同名方法,参数列表不同(数量、类型、顺序)即构成**重载**。编译器根据实参类型在编译期选择目标方法。**返回值类型不参与重载判定**(只改返回值不叫重载,是编译错误)。常见用途:printf 风格的多形态输出、构造器重载(第6章)。

示例 4-3 重载示例

```
public class OverloadDemo {
    static void report(String name) {           // 一个参数
        System.out.println("学生:" + name);
    }

    static void report(String name, int score) { // 两个参数:重载
        System.out.println("学生:" + name + ",分数:" + score);
    }

    static void report(String name, double avg) { // 类型不同:重载
        System.out.println("学生:" + name + ",平均:" + avg);
    }

    // 返回类型不同不算重载!下面这行编译错误:
    // static int report(String name) { return 0; }

    public static void main(String[] args) {
        report("张三");           // 选 1 参版本
        report("李四", 88);       // 选 int 版本
        report("王五", 92.5);     // 选 double 版本
    }
}
```

4.3.2 可变参数 varargs

`int... nums` 表示"0 到任意个 int",底层是一个数组:`int... nums` 等价于 `int[] nums` (调用处自动包装)。规则:可变参数必须是最后一个参数;重载时可变参数是"兜底",编译器优先选精确匹配。

示例 4-4 varargs 示例

```
public class VarargsDemo {
    // 求任意个数的最大值:int... 等价于 int[]
    static int maxOf(int... nums) {
        if (nums.length == 0) {
            throw new IllegalArgumentException("至少需要一个数");
        }
        int max = nums[0];
        for (int i = 1; i < nums.length; i++) {
            if (nums[i] > max) {
                max = nums[i];
            }
        }
        return max;
    }

    // 可变参数必须是最后一个
    static void log(String tag, String... items) {
        System.out.print "[" + tag + " ] ";
        for (String item : items) {
            System.out.print(item + " ");
        }
        System.out.println();
    }

    public static void main(String[] args) {
        System.out.println(maxOf(3, 9, 1)); // 9
        System.out.println(maxOf()); // 抛异常(0 个参数)
        log("INFO", "启动", "连接", "完成"); // 传 3 个
        log("WARN"); // 传 0 个
        // 也可以直接传数组:
        int[] arr = {5, 6, 7};
        System.out.println(maxOf(arr)); // 7
    }
}
```

⚠ 易错点 3:varargs 与重载的"歧义陷阱"

`f(int...)` 与 `f(int[])` 同时存在会编译失败(相同签名); `f(int)` 与 `f(int...)` 同时存在时, `f(3)` 优先选精确的 `f(int)` ——但阅读代码的人常误以为会进 `varargs`。生产建议: `varargs` 版本只用于"确定是可变场景",不要与同语义的单参版本并存。

§4.4 递归与调用栈

4.4.1 递归与栈帧

递归与 C 完全相同:方法调用自己,每次调用在调用栈上分配一个**栈帧**(局部变量、参数、返回地址)。递归必须满足两件事:①有终止条件(基准情形);②每次递归都向基准情形靠近。缺一条就是栈溢出(StackOverflowError,第5章详讲内存)。

示例 4-5 递归:二分查找与斐波那契


```

public class RecursionDemo {
    // 二分查找(递归版):要求 arr 升序;返回下标,找不到返回 -1
    static int binarySearch(int[] arr, int target, int lo, int hi) {
        if (lo > hi) {
            return -1; // 终止条件:区间为空
        }
        int mid = lo + (hi - lo) / 2; // 防溢出的经典写法
        if (arr[mid] == target) {
            return mid; // 命中
        }
        if (arr[mid] > target) {
            return binarySearch(arr, target, lo, mid - 1); // 向左半区递归
        }
        return binarySearch(arr, target, mid + 1, hi); // 向右半区递归
    }

    // 斐波那契(递归版,指数复杂度,仅演示;生产用迭代)
    static long fib(int n) {
        if (n <= 1) {
            return n; // 基准情形
        }
        return fib(n - 1) + fib(n - 2);
    }

    public static void main(String[] args) {
        int[] a = {1, 3, 5, 7, 9, 11};
        System.out.println(binarySearch(a, 7, 0, a.length - 1)); // 3
        System.out.println(binarySearch(a, 8, 0, a.length - 1)); // -1
        System.out.println(fib(10)); // 55
        // fib(50) 会非常慢:每层递归分裂成两个调用,总调用数接近 2^n
    }
}

```

★ 重点:递归改迭代的判定

递归深度与数据规模成正比时,生产代码优先改迭代(尾递归在 Java 中无优化)。判定方法:递归深度是 $O(n)$ 且 n 可能上万 → 改迭代;深度是 $O(\log n)$ (二分、快速排序) → 递归放心用。栈大小默认约 512KB~1MB,几万帧就能打满(第5章)。

§4.5 lambda 表达式与方法引用

4.5.1 函数式接口:lambda 的"类型"

lambda 必须挂在[函数式接口](#)上——只有一个抽象方法的接口(接口概念第6章详讲,现在只需会用)。Java 自带一批常用函数式接口: `Runnable` (无参无返回)、`Consumer<T>` (一个参数无返回)、`Function<T,R>` (一进一出)、`Predicate<T>` (返回 boolean)、`Supplier<T>` (无参出值)。

示例 4-6 lambda 的四种形态

```

import java.util.function.Consumer;
import java.util.function.Function;
import java.util.function.Predicate;

public class LambdaDemo {
    public static void main(String[] args) {
        // 形态 1:无参数,一句话,等价于 Runnable
        Runnable task = () -> System.out.println("任务执行");

        // 形态 2:一个参数,省略括号和类型
        Consumer<String> printer = name -> System.out.println("你好," + name);

        // 形态 3:多语句用花括号,必须写 return
        Function<Integer, String> describe = n -> {
            if (n >= 60) {
                return "及格";
            }
            return "不及格";
        };

        // 形态 4:Predicate 返回 boolean,常用于过滤
        Predicate<Integer> isEven = n -> n % 2 == 0;

        task.run();
        printer.accept("张三");
        System.out.println(describe.apply(75));
        System.out.println(isEven.test(8));    // true

        // 用 lambda 给数组排序(Comparator 是函数式接口)
        String[] words = {"pear", "apple", "banana", "kiwi"};
        java.util.Arrays.sort(words, (x, y) -> x.length() - y.length());
        System.out.println(java.util.Arrays.toString(words));
        // [kiwi, pear, apple, banana] 按长度升序
    }
}

```

4.5.2 方法引用:已有方法的"简写指针"

lambda 体若只是"调用一个现有方法",可以缩写为**方法引用**: `类::静态方法`、`对象::实例方法`、`类::实例方法`、`类::new` (构造器引用)。它相当于 C 里"函数指针的地址",但类型安全、由编译器检查签名。

示例 4-7 方法引用示例

```

import java.util.Arrays;

public class MethodRefDemo {
    static int byLength(String a, String b) {
        return a.length() - b.length();
    }

    public static void main(String[] args) {
        String[] names = {"Tom", "Alice", "Bob", "Carolyn"};
        // lambda 写法:
        Arrays.sort(names, (a, b) -> byLength(a, b));
        // 方法引用写法(完全等价):
        Arrays.sort(names, MethodRefDemo::byLength);

        System.out.println(Arrays.toString(names));
        // 输出流式打印:println 是 System.out 的实例方法
        Arrays.asList(names).forEach(System.out::println);
    }
}

```

工程建议:lambda 的可读性红线

lambda 体超过 3 行就提取成具名方法,再使用方法引用。好处:方法有名字(自文档)、可单测、可复用。一行 lambda 写清,两行勉强,三行以上是坏味道。这与 C 里"回调逻辑复杂就写具名函数"是同一纪律。

§4.6 Stream API 入门

4.6.1 三件套:创建 → 中间操作 → 终止操作

Stream 是对数据集合的声明式流水线:① 创建(从数组/集合/范围);② 中间操作(过滤 filter、映射 map、排序 sorted,惰性、可链式);③ 终止操作(collect、forEach、count,触发真正计算)。没有终止操作,中间操作什么都不做。Stream 不修改原数据,返回新流。

示例 4-8 循环写法 vs 流式写法

```
import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;

public class StreamDemo {
    public static void main(String[] args) {
        List<Integer> scores = Arrays.asList(45, 88, 62, 91, 55, 73, 100);

        // ---- C 程序员习惯的循环写法 ----
        int sum = 0, count = 0;
        for (int s : scores) {           // 遍历
            if (s >= 60) {               // 过滤
                sum += s;                // 映射(累加)
                count++;
            }
        }
        double avg = count == 0 ? 0 : (double) sum / count;

        // ---- Stream 写法:同样的逻辑,声明式 ----
        double avg2 = scores.stream()    // 创建
            .filter(s -> s >= 60)        // 中间:过滤及格
            .mapToInt(s -> s)            // 中间:转 int 流
            .average()                   // 终止:求平均
            .orElse(0);                  // 空流时给默认值

        System.out.println(avg + " vs " + avg2);    // 78.5 vs 78.5

        // 常用组合:过滤 + 映射 + 收集成新列表
        List<String> top = scores.stream()
            .filter(s -> s >= 90)
            .sorted((a, b) -> b - a)      // 降序
            .map(s -> "优:" + s)
            .collect(Collectors.toList());
        System.out.println(top);          // [优:100, 优:91]

        // 范围流:IntStream.range 是"带下标的增强 for"的替代
        int total = java.util.stream.IntStream.range(1, 101).sum();
        System.out.println(total);        // 5050
    }
}
```

★ 重点:惰性求值

中间操作不会立刻执行,只有终止操作被调用时才"拉取"元素逐个流过流水线——所以 `stream().filter(...).map(...)` 不接终止操作时,日志里一个元素都不会处理。这个"声明而不执行"的模型,是函数式编程与命令式循环最大的思维差异。

4.6.2 何时用 Stream:两条经验法则

- 处理"整个集合"且逻辑是"过滤-变换-汇总"三步之内 → Stream,代码更短、意图更显式
- 需要中途 break/带下标/多循环变量 → 传统循环,Stream 的短路(anyMatch、findFirst)不够直白时别硬拗

⚠ 易错点 4:Stream 不能复用

Stream 是"一次性"的:终止操作执行后流就关闭了,再调用会抛 `IllegalStateException: stream has already been operated upon`。需要再遍历就重新创建流。这与 C 里"数组可以反复遍历"的习惯不同,是新手最常见的 Stream 运行时错误。

§4.7 方法设计工程实践

📄 生产级方法设计清单

- **小而专**:一个方法只做一件事(通常 5-20 行);超过 40 行必须拆分。C 的"巨型函数"坏味道在 Java 里同样成立
- **纯函数优先**:不修改参数、不依赖全局可变状态,同样的输入必有同样的输出——便于测试与并行(第9章)
- **命名动词化**:calculateAverage、findById、isValid;boolean 返回用 is/has/can 前缀
- **参数宁少勿多**:超过 3 个参数考虑"参数对象"(第6章 records 是完美载体)
- **防御式返回**:返回 null 前想清楚:调用方 10 行外就要判空(第14章 Optional 解决)
- **文档注释**:public 方法写 javadoc 说明意图与参数约束,不是复述代码

本章速查表

主题	要点 / 写法
方法声明	static 返回类型 名(参数列表) { };工具类模式 = 类装 static 方法
入口	public static void main(String[] args);args 即 argv,args.length 即 argc
值传递	只有值传递;基本类型传拷贝,引用传引用的拷贝;方法内重新赋值不影响实参
swap	Java 无法交换两个基本类型变量;用返回值或包装数组
重载	同名不同参(数量/类型/顺序);返回值不参与判定
varargs	int... nums ≡ int[] nums;必须放最后;空参数时 length 为 0
递归	必须满足:终止条件 + 向基准靠近;深度 O(n) 且 n 大时改迭代
函数式接口	只有一个抽象方法的接口;Runnable/Consumer/Function/Predicate/Supplier
lambda	(参数) -> 表达式 或 { 多语句 };捕获变量须 effectively final
方法引用	类::静态方法 / 对象::方法 / 类::实例方法 / 类::new
Stream 三件套	创建 stream() → 中间 filter/map/sorted → 终止 collect/forEach/average
惰性求值	没有终止操作,中间操作不执行;流一次性,不能复用
循环 vs Stream	整集合"过滤-变换-汇总"用 Stream;需要 break/下标用循环
方法设计	小而专 / 纯函数 / 动词命名 / 参数 ≤ 3 / javadoc

最小必会清单

- 能逐字写出 main 签名并解释 public/static/void/args 的含义
- 能画图解释"值传递":swap 失败、数组内容可改、形参重赋值无效
- 能写出 2 个重载方法并说明重载判定规则(返回值不参与)

- 能写出 `varargs` 方法并说出它与数组的关系
- 能写出递归二分查找,并能判断何时递归该改迭代
- 能写出 `lambda` 的四种形态并说出函数式接口的定义
- 能写出"过滤及格→求平均"的 `Stream` 流水线,并解释惰性求值
- 能说出方法引用的四种形式并举例

自测题

Q 1. 下面的输出是什么? `int a = 1; f(a); System.out.println(a);` 其中 `static void f(int x) { x = 99; }`

✓ 参考答案

输出 1。基本类型按值传递,x = 99 只改形参拷贝。想改实参只能用返回值接收、包装进数组/类。

Q 2. 判断对错:Java 是引用传递,因为方法里修改对象字段,调用方能看到变化。

✓ 参考答案

错。那是"引用指向同一对象"的效果。验证法:方法里写 `p = new Point(...)` 后调用方不受影响,说明传的是引用的拷贝——值传递。

Q 3. 以下哪个是合法重载?① `int f(int x)` 与 `double f(int x)`;② `void f(int x)` 与 `void f(long x)`;③ `void f(int... a)` 与 `void f(int[] a)`

✓ 参考答案

只有 ② 合法:参数类型不同。① 只改返回值,非法;③ `int...` 与 `int[]` 签名相同,非法(编译错误:duplicate method)。

Q 4. 递归求 $1+2+\dots+n$,写出来;并说明 $n = 100000$ 时会发生什么。

✓ 参考答案

```
static long sum(int n) {
    if (n <= 0) return 0;           // 终止条件
    return n + sum(n - 1);         // 向基准靠近
}
```

$n = 100000$ 时递归深度 10 万层,超出默认栈大小,抛 `StackOverflowError`。改用 `for` 或公式 $n*(n+1)/2$ 。判定口诀:深度 $O(n)$ 且 n 可能很大 → 迭代。

Q 5. 把这段循环改写成 `Stream`:统计数组里正数的个数。 `int c = 0; for (int x : arr) if (x > 0) c++;`

✓ 参考答案

```
long c = Arrays.stream(arr).filter(x -> x > 0).count();
```

先 `stream()` 创建,filter 中间操作筛正数,count 终止操作计数。注意 `count` 返回 `long`。

Q 6. 为什么 `stream().filter(...).map(...)` 没有输出?如何让它工作?

✓ 参考答案

缺少终止操作,中间操作惰性执行、什么都不做。加一个终止操作,如 `.collect(Collectors.toList())` 或 `.forEach(System.out::println)`,流水线才真正运行。流是一次性的,第二次用要重新创建。

章末实验题:成绩处理系统

实验 4:成绩处理系统

实验目标:编写一个成绩处理程序,综合运用本章全部知识点:方法定义与调用、值传递、重载、varargs、递归、lambda、方法引用、Stream。

实验要求:

- 任务 1:定义 `readScores()` 读取成绩存入数组,主方法调用它(覆盖 §4.1 方法定义与调用)
- 任务 2:定义 `swapScore(int[] arr, int i, int j)` 交换数组元素,并在方法里演示"试图直接交换两个 int 局部变量失败"(覆盖 §4.2 值传递)
- 任务 3:重载 `report(String name)` 与 `report(String name, double avg)` (覆盖 §4.3 重载)
- 任务 4:定义 `maxOf(int... nums)` 求最高分,以 varargs 方式调用(覆盖 §4.3 varargs)
- 任务 5:用递归实现二分查找,在排序后的成绩里找目标分(覆盖 §4.4 递归)
- 任务 6:用 lambda 实现降序排序、用方法引用打印及格名单(覆盖 §4.5 lambda/方法引用)
- 任务 7:用 Stream 计算及格平均分,并筛选出前 3 名(覆盖 §4.6 Stream)
- 任务 8:main 中按顺序调用各方法,控制台输出完整报告(覆盖 §4.7 方法组织)

实验环境:JDK 17+; javac GradeSystem.java 编译, java GradeSystem 运行。

第一步 写 main 骨架:数组 + 读入 + 各任务方法占位,编译通过再逐个实现

第二步 任务 2 的"swap 失败演示"与"数组元素交换成功"对照打印,加深值传递理解

第三步 任务 6-7 对照着写:先写循环版,再改写 Stream 版,体会两者等价

参考答案要点

```

import java.util.Arrays;

/** 成绩处理系统 — 第4章综合实验参考答案
 * 覆盖:方法定义/调用、值传递、重载、varargs、递归、lambda、方法引用、Stream
 */
public class GradeSystem {
    private static final int[] scores = {45, 88, 62, 91, 55, 73, 100, 68, 59, 82};

    public static void main(String[] args) {
        // 任务 2:值传递演示
        int a = 1, b = 2;
        swap(a, b); // 失败:基本类型传值
        System.out.println("swap(a,b) 后: a=" + a + ", b=" + b); // a=1, b=2
        swapScore(scores, 0, 1); // 成功:数组是引用,可改内容
        System.out.println("交换后前两项: " + scores[0] + ", " + scores[1]);

        // 任务 3:重载
        report("张三");
        report("张三", average(scores));

        // 任务 4:varargs
        System.out.println("最高分 = " + maxOf(scores));
        System.out.println("varargs 单独调用 = " + maxOf(3, 9, 1, 7));

        // 任务 5:递归二分(先排序)
        int[] sorted = scores.clone();
        Arrays.sort(sorted);
        System.out.println("二分查找 68 的下标 = " + binarySearch(sorted, 68, 0, sorted.length - 1));

        // 任务 6:lambda 排序 + 方法引用打印
        Integer[] sArr = Arrays.stream(scores).boxed().toArray(Integer[]::new);
        Arrays.sort(sArr, (x, y) -> y - x); // lambda:降序
        System.out.print("降序: ");
        Arrays.asList(sArr).forEach(System.out::print); // 方法引用
        System.out.println();

        // 任务 7:Stream 统计
        double passAvg = Arrays.stream(scores)
            .filter(s -> s >= 60)
            .average()
            .orElse(0);
        System.out.printf("及格平均分 = %.1f%n", passAvg);
        System.out.println("前 3 名: " + Arrays.stream(scores)
            .boxed()
            .sorted((x, y) -> y - x)
            .limit(3)
            .toList());
    }

    // 任务 2:两个 int 无法交换(值传递,只改形参拷贝)
    static void swap(int x, int y) {
        int t = x;
        x = y;
        y = t;
    }

    // 数组元素可以交换:引用指向同一数组
    static void swapScore(int[] arr, int i, int j) {
        int t = arr[i];
        arr[i] = arr[j];
        arr[j] = t;
    }

    // 任务 3:重载(参数个数不同)
    static void report(String name) {
        System.out.println("学生:" + name);
    }
}

```

```

static void report(String name, double avg) {
    System.out.printf("学生:%s,平均分:%.1f%n", name, avg);
}

// 任务 4:varargs 求最大值
static int maxOf(int... nums) {
    int max = nums[0];
    for (int i = 1; i < nums.length; i++) {
        if (nums[i] > max) {
            max = nums[i];
        }
    }
    return max;
}

// 任务 5:递归二分查找(数组升序)
static int binarySearch(int[] arr, int target, int lo, int hi) {
    if (lo > hi) {
        return -1;
    }
    int mid = lo + (hi - lo) / 2;
    if (arr[mid] == target) {
        return mid;
    }
    if (arr[mid] > target) {
        return binarySearch(arr, target, lo, mid - 1);
    }
    return binarySearch(arr, target, mid + 1, hi);
}

// 辅助:平均值(空数组返回 0)
static double average(int[] arr) {
    return arr.length == 0 ? 0 : (double) Arrays.stream(arr).sum() / arr.length;
}
}

```

设计说明:任务 2 把"失败的 swap"与"成功的数组交换"并列打印,一次看懂值传递;任务 6 故意对照 lambda 与方法引用;任务 7 的 Stream 全部是标准 API,没有手写循环。扩展方向:成绩来自控制台输入(第3章菜单)、异常处理(第9章)、按学生对象分组(第6章)。

下一章预告:第5章 内存管理。方法在调用栈上执行,对象在堆上生存——JVM 的内存区域划分、垃圾回收原理、内存泄漏与调优,是"生产级"Java 的必修课。

第5章 内存管理

建议学时:4-6 小时 | 难度:★★★★☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 掌握 JVM 内存区域的五大划分,并与 C 的栈/堆/静态区对照
- 理解对象在堆上的生命周期与"逃逸"概念
- 理解垃圾回收的基本原理:可达性分析、分代收集、Stop-The-World
- 掌握强/软/弱/虚四种引用,重点会用 WeakReference 与 WeakHashMap
- 识别生产环境四大内存泄漏源,会写防泄漏代码
- 掌握 try-with-resources 资源管理,理解它与 C RAII 的对应关系
- 会用 JVM 参数与诊断工具定位 OOM

💡 从 C 到 Java:内存管理的世界观转变

C 程序员的第一直觉是"内存必须自己管":malloc 配对 free,一个不配就是泄漏。Java 把这套全扔了——new 出来的对象没人负责回收,垃圾回收器(GC)自动处理。第一次写 Java 的人会恐慌:"那我的内存什么时候被释放?会不会到处泄漏?"

答案分两层:**浅层答案**——你不需要管,GC 会找到"再也够不着"的对象并回收,绝大多数内存问题自动消失;**深层答案**——GC 只能回收"不可达"的对象,如果你的代码把对象留着(静态集合、缓存、监听器),GC 永远不会动它,这就是 Java 的内存泄漏。C 的泄漏是"忘了释放",Java 的泄漏是"忘了放手"。

栈与堆的职责也变了。C 里结构体可以在栈上、可以复制;Java 里所有对象一律在堆上,栈上只放"引用"。你写 `Point p = new Point(1,2)`,p 在栈上,Point 对象在堆上——这和 `Point* p = malloc(sizeof(Point))` 同构,只是没有 free。

资源管理需要新工具。C 用 RAII(析构函数)保证文件、锁在退出作用域时释放;Java 没有析构函数,替代方案是 try-with-resources ——写出来像 C 的 RAII,但语法上是显式的 try 块。这是本章的生产力重点。

学习顺序:先看 JVM 内存地图(\$5.1-5.2),再理解 GC 机制(\$5.3-5.4),然后学防泄漏与资源管理(\$5.5-5.6),最后是诊断工具(\$5.7)。

📦 前置知识自查

- C 内存:malloc/free、栈帧、指针、悬垂指针、内存泄漏的概念
- 第1章:数组与字符串(对象本质);第4章:方法调用与栈帧
- 第3章:循环与分支(写实验需要)

🚀 本章学习路线

1. **第一阶段(约 1.5 小时):**\$5.1-5.2 对照 C 的进程内存模型,画一张"JVM 内存地图"
2. **第二阶段(约 1.5 小时):**\$5.3-5.4 GC 原理 + 弱引用,运行 WeakReference 示例观察回收
3. **第三阶段(约 1.5 小时):**\$5.5-5.6 内存泄漏与 try-with-resources,完成章末实验
4. **验收标准:**能解释"Java 内存泄漏=忘了放手";能写出防泄漏的缓存;能写出 try-with-resources

本章知识导图

- §5.1 JVM 内存区域:程序计数器 / 栈 / 堆 / 方法区
- §5.2 栈与堆的生命周期:栈帧 / 对象逃逸
- §5.3 垃圾回收:可达性分析 / 分代收集 / STW
- §5.4 四种引用:强 / 软 / 弱 / 虚引用
- §5.5 内存泄漏四大源头与防护
- §5.6 try-with-resources 资源管理
- §5.7 OOM 诊断与 JVM 参数

§5.1 JVM 内存区域:一张地图

5.1.1 五大区域

JVM 区域	C 的对应物	存什么 / 特点
程序计数器	(无直接对应)	当前执行到哪条字节码;线程私有,极小
虚拟机栈(Java 栈)	调用栈 + 局部变量	每线程一个;栈帧存局部变量、操作数;方法调用→压帧,返回→弹帧
本地方法栈	调用栈	执行 native 方法(如 System.currentTimeMillis 底层)
堆(Heap)	malloc 的堆	所有 new 出来的对象;GC 的主战场;线程共享;最大的一块
方法区/元空间	静态区 (.data/.bss)	类元数据、静态变量、常量池(字符串字面量);JDK8+ 叫元空间,用本地内存

示例 5-1 栈深观察:递归到溢出

```
public class StackDepthDemo {
    static int depth = 0;

    static void dive() {
        depth++;           // 每层递归压一个栈帧
        dive();            // 永不返回:栈帧无限增长
    }

    public static void main(String[] args) {
        try {
            dive();
        } catch (StackOverflowError e) {
            // 注意:Error 不是 Exception,生产代码通常不 catch;这里仅演示
            System.out.println("栈溢出,最大深度约 " + depth);
        }
    }
}
```

// 输出类似:栈溢出,最大深度约 12000(与 -Xss 有关)

⚠ 易错点 1:OOM 与栈溢出是不同的 Error

StackOverflowError:递归过深,栈区耗尽;**OutOfMemoryError**:堆(或元空间)耗尽。两者都是 Error 不是 Exception(第9章详讲)。区分诊断:StackOverflowError 先看递归终止条件;OOM 先看是"Java heap space"还是"Metaspace"还是"unable to create native thread"——三种成因完全不同。

§5.2 栈与堆的生命周期

5.2.1 对象逃逸:栈上的引用,堆上的对象

规则: 局部变量在栈帧里,对象在堆上,栈帧弹出时,引用消失,对象变成"不可达"候选,等待 GC。唯一例外是"局部变量是基本类型"(整个值都在栈上)。"对象被方法返回/存入字段/存入集合,导致它的生命周期超过定义它的方法"——这叫**逃逸**。

示例 5-2 对象生命周期演示

```
public class LifecycleDemo {
    // 堆上对象计数:构造器里 ++,演示"谁还活着"很难靠计数,要靠可达性
    static int alive = 0;

    static class BigData {
        byte[] payload = new byte[1024 * 1024]; // 每个对象占 1MB 堆
        BigData() { alive++; }
        @Override protected void finalize() { alive--; } // 仅演示,生产禁用!
    }

    static BigData make() {
        BigData d = new BigData(); // 对象在堆上,d 是栈上的引用
        return d; // 引用返回:对象逃逸出 make 的栈帧
    }

    public static void main(String[] args) throws Exception {
        System.out.println("初始存活: " + alive);
        BigData keep = make(); // keep 持有引用:对象继续活
        System.out.println("make 后存活: " + alive);

        for (int i = 0; i < 20; i++) {
            make(); // 返回值丢弃:20 个对象立刻不可达
        }
        System.gc(); // 建议 JVM 执行 GC(不保证立即执行)
        Thread.sleep(100); // 给 GC 一点时间
        System.out.println("丢弃后存活: " + alive); // 只剩 keep 那 1 个
    }
}
// 输出趋势:初始 0 → make 后 1 → 丢弃后 1(finalize 已不推荐,结果可能为 2,原理见下)
```

★ 重点:finalize 已废弃,别用

C 的析构函数在 Java 里没有对应物。finalize() 是历史遗留:调用时机不确定、影响 GC 性能, JDK9 起标记废弃,生产代码一律不用。资源释放的正确姿势:try-with-resources(§5.6)。上面示例用 finalize 只为演示"对象消亡"——正常代码统计存活对象应该用分析工具,而不是钩子。

§5.3 垃圾回收:可达性分析

5.3.1 原理:根可达,其余皆垃圾

GC 从 **GC Roots** 出发做可达性分析:栈上的局部变量、静态字段、活跃线程、JNI 引用是根;从根能引用到的对象都"可达、活着";不可达的对象被标记为垃圾,统一回收。这正是"引用计数"方案的替代——C++ 的 shared_ptr 用引用计数,循环引用会泄漏;Java 的可达性分析天然解决循环引用(两个互相引用但外部够不着的对象,一起被回收)。

5.3.2 分代收集:新生代与老年代

- **新生代(Young)**:新对象都先进来;大部分对象"朝生暮死";Minor GC 频繁且快(Eden + 两个 Survivor 区)
- **老年代(Old)**:熬过多次 Minor GC 的对象晋升进来;Major GC 慢、少
- **Stop-The-World(STW)**:GC 线程要暂停业务线程才能安全移动对象;停顿时间 = GC 调优的核心指标

生产结论: 别迷信"自己调 GC 参数",默认 GC(JDK17 默认 G1)对绝大多数应用已足够;你需要做的只是:别把对象留在老年代(防泄漏)、别创建海量短命大对象(给 GC 添乱)。

示例 5-3 观察 GC:内存曲线

```
public class GcDemo {
    public static void main(String[] args) throws Exception {
        Runtime rt = Runtime.getRuntime();
        long used = (rt.totalMemory() - rt.freeMemory()) / 1024 / 1024;
        System.out.println("JVM 总内存(约): " + rt.totalMemory() / 1024 / 1024 + " MB");
        System.out.println("当前已用(约): " + used + " MB");

        for (int round = 0; round < 3; round++) {
            for (int i = 0; i < 1000; i++) {
                byte[] junk = new byte[1024 * 100];    // 每个 100KB,共约 100MB
            }
            System.gc();          // 建议回收(生产不要主动调,这里用于教学观察)
            Thread.sleep(200);
            used = (rt.totalMemory() - rt.freeMemory()) / 1024 / 1024;
            System.out.println("第 " + (round + 1) + " 轮后已用(约): " + used + " MB");
        }
        // 观察点:每轮创建 100MB 垃圾后,已用内存没有累积到 300MB—GC 把它们回收了
    }
}
```

⚠ 易错点 2:System.gc() 只是"建议"

生产代码 不要调 System.gc():它只是向 JVM 提交请求,可能被忽略(比如 -XX:+DisableExplicitGC);即使执行,STW 停顿也会伤性能。教学观察除外。正确做法:设置堆大小上限(-Xmx),让 JVM 自己按水位线回收,并用工具观察。

§5.4 四种引用:从强到弱

引用类型	回收条件	典型用途
强引用(默认)	永不回收,直到不可达	日常所有 new
软引用 SoftReference	内存不足时才回收	图片/数据缓存(内存敏感)
弱引用 WeakReference	下一次 GC 必回收	缓存、观察者、WeakHashMap
虚引用 PhantomReference	对象回收后收到通知	直接内存回收、对象消亡跟踪(高级)

示例 5-4 WeakReference:让缓存"漏得掉"

```

import java.lang.ref.WeakReference;
import java.util.WeakHashMap;

public class WeakRefDemo {
    public static void main(String[] args) throws Exception {
        // ---- 弱引用:GC 后引用自动变 null ----
        BigObject obj = new BigObject();
        WeakReference<BigObject> ref = new WeakReference<>(obj);
        System.out.println("GC 前 get(): " + (ref.get() != null));    // true

        obj = null;          // 唯一强引用断开,对象只剩弱引用
        System.gc();
        Thread.sleep(100);
        System.out.println("GC 后 get(): " + (ref.get() != null));    // false

        // ---- WeakHashMap:键是弱引用,键被回收时条目自动消失 ----
        WeakHashMap<String, String> cache = new WeakHashMap<>();
        String key = new String("config");    // 不要用字面量,否则常量池强引用它
        cache.put(key, "v1");
        key = null;          // 键的强引用断开
        System.gc();
        Thread.sleep(100);
        System.out.println("缓存条目数: " + cache.size());           // 0
    }

    static class BigObject {
        byte[] payload = new byte[1024 * 1024];
    }
}

```

★ 重点:缓存的正确姿势

缓存是弱引用的经典场景:缓存数据的生命周期应该"跟着使用它的业务对象走"。强引用缓存 = 静态持有 = 永不回收 = 内存泄漏(见 §5.5);WeakHashMap 让条目在"业务键"消失后自动蒸发。生产级别的缓存框架(Caffeine、Redis)有更复杂的淘汰策略,但原理起点就是这里。

§5.5 内存泄漏四大源头

5.5.1 Java 内存泄漏 = 忘了放手

对象明明没用了,但代码里还留着到它的引用链,GC 认为它可达,永远不回收。四大高频源头:

1. **静态集合无限增长**: `static List<Session> sessions` 只 add 不 remove,或"全局缓存无上限"
2. **长生命周期对象持有短生命周期对象**:全局单例持有请求对象、监听器注册后从不注销
3. **资源未关闭**:文件/连接/流没 close,底层 native 资源泄漏(栈内存也涨)
4. **ThreadLocal 不清除**:线程池里的线程长期存活,ThreadLocal 值被线程强引用,形成泄漏(第9章)

示例 5-5 泄漏演示与修复

```

import java.util.ArrayList;
import java.util.List;

public class LeakDemo {
    // ---- 泄漏版:静态集合只进不出 ----
    static final List<byte[]> CACHE = new ArrayList<>();

    static void leak(int n) {
        for (int i = 0; i < n; i++) {
            CACHE.add(new byte[1024 * 1024]); // 1MB x n, 永远可达!
        }
    }

    // ---- 修复版:限长缓存,超出丢弃最旧 ----
    static void boundedCache(byte[] data) {
        if (CACHE.size() >= 10) {
            CACHE.remove(0); // 超过上限,移除最旧
        }
        CACHE.add(data);
    }

    public static void main(String[] args) throws Exception {
        // 模拟业务:每次请求缓存一个快照,快照用完应丢弃
        for (int req = 0; req < 1000; req++) {
            byte[] snapshot = new byte[1024 * 1024];
            // leak(1) 会累积 1000MB;boundedCache 只留 10MB
            boundedCache(snapshot);
            snapshot = null; // 用完就放手
        }
        System.gc();
        Thread.sleep(200);
        long used = (Runtime.getRuntime().totalMemory()
            - Runtime.getRuntime().freeMemory()) / 1024 / 1024;
        System.out.println("处理后已用内存(约): " + used + " MB");
        System.out.println("缓存条目: " + CACHE.size()); // 10
    }
}

```

⚠ 易错点 3:把对象赋 null 不是好习惯

看到"C 程序员式清理" `obj = null` ——多数时候没必要:局部变量随栈帧弹出自然消失。但 长生命周期容器里"用完了"的对象,显式置 `null` 或 `remove` 是有意义的(帮助 GC 尽早回收)。判断标准:这个引用会不会活得比对象久?会 → 放手;不会 → 别画蛇添足。

§5.6 try-with-resources:Java 的 RAII

5.6.1 语法与语义

try-with-resources:在 `try` 括号里声明资源,代码块结束时(无论正常、`return`、异常)自动按声明逆序调用 `close()`。资源类必须实现 `AutoCloseable` (其子接口 `Closeable` 的 `close` 不抛普通异常)。对照 C:作用域结束自动调用析构函数——语义等价,语法更显式。

示例 5-6 自定义资源 + try-with-resources

```

import java.io.BufferedReader;
import java.io.FileReader;

public class TwrDemo {
    // 自定义资源:实现 AutoCloseable 就能进 try-with-resources
    static class LogFile implements AutoCloseable {
        private final String name;

        LogFile(String name) { this.name = name; System.out.println("打开 " + name); }

        void write(String line) { System.out.println(" [写入] " + name + ": " + line); }

        @Override
        public void close() {          // 必须实现;声明 throws Exception 也行
            System.out.println("关闭 " + name);    // 一定被执行
        }
    }

    public static void main(String[] args) throws Exception {
        // 多个资源:用分号分隔,按声明逆序关闭(先 b 后 a)
        try (LogFile a = new LogFile("a.log");
            LogFile b = new LogFile("b.log")) {
            a.write("hello");
            b.write("world");
        }                                     // 出块自动关闭,连 catch/finally 都不需要

        // 标准库示例:读文件,不用手写 finally { in.close(); }
        try (BufferedReader in = new BufferedReader(
            new FileReader("data.txt"))) {
            String line = in.readLine();
            System.out.println("首行: " + line);
        } // 文件句柄在这里被自动关闭;文件不存在时 catch 一下即可
    }
}

```



工程建议:资源处理三板斧

1. 凡是"打开就要关"的东西(流、连接、通道、锁),一律 try-with-resources ,不要手写 try/finally/close 三板斧——代码更短,且异常路径也不会漏关
2. 多资源按"先开的后关"写,括号里用分号分隔,别嵌套 try
3. close() 失败(比如网络断开)会让真正的业务异常被吞——需要时用 try {...} catch (Exception e) { e.addSuppressed(...) } 语义,JDK 已自动处理 suppressed 异常(可用 getSuppressed 查看)

§5.7 OOM 诊断与 JVM 参数

5.7.1 常用 JVM 启动参数

参数	含义	生产建议
-Xms256m	堆初始大小	与 -Xmx 相同,避免运行时扩容抖动
-Xmx1g	堆最大大小	必须设置上限,防止 OOM 拖垮整机
-Xss512k	单线程栈大小	递归深再调;默认约 512k-1m
-XX:+HeapDumpOnOutOfMemoryError	OOM 时自动导出堆转储	生产必开,配合 -XX:HeapDumpPath 指定路径
-XX:MaxMetaspaceSize	元空间上限	类加载过多(动态生成类)时保护

示例 5-7 生产启动参数示例(命令行)


```
# 生产启动示例:堆固定 512m,防止扩容抖动;OOM 自动导出堆转储
java -Xms512m -Xmx512m -XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/logs/dumps/ -jar app.jar

# 本地调试:调大栈空间,观察递归深度变化
java -Xss2m StackDepthDemo

# 查看 GC 频率(每秒一次):jstat 是定位 GC 风暴的第一把刀
jstat -gcutil 12345 1000
```

5.7.2 诊断工具清单

- **jps**:列出 JVM 进程;**jstat -gcutil <pid>**:每秒看 GC 频率与堆占用——先看这个,判断是不是 GC 风暴
- **jmap -dump**:手动导出堆转储(生产慎用,会 STW);OOM 自动转储后用 **Eclipse MAT / VisualVM** 分析"支配树",找谁占着内存、被谁引用
- **jcmd**:综合诊断命令, **jcmd <pid> GC.heap_info** 等

⚠ 易错点 4:OOM 处理流程别反

正确顺序:① 先确认 OOM 类型(heap?metaspace?thread?);② 用 jstat 看 GC 频率;③ 导出堆转储分析支配树找"大对象+引用链";④ 改代码(防泄漏/限长/分页),而不是盲目加 -Xmx。90% 的 OOM 是代码问题,10% 才是堆太小。

本章速查表

主题	要点
五大区域	程序计数器 / 虚拟机栈 / 本地方法栈 / 堆 / 元空间(JDK8+ 取代方法区)
栈 vs 堆	局部变量在栈帧;所有 new 的对象在堆;栈帧弹出→引用消失→对象可回收
逃逸	对象被返回/存入字段/集合 = 生命周期超过所在方法 = 逃逸
GC 原理	从 GC Roots 做可达性分析;不可达即回收;循环引用自动解决
分代	新生代(Eden+Survivor,频繁 Minor GC)→ 老年代(Major GC 慢);STW 暂停业务线程
System.gc()	只是建议,生产不要调;教学观察除外
四种引用	强(默认)/ 软(内存不足才回收,缓存)/ 弱(下次 GC 必回收)/ 虚(回收通知)
WeakHashMap	键用弱引用;键无强引用时条目自动消失;缓存首选
泄漏四源	静态集合只增不减 / 长生命周期持短生命 / 资源未关 / ThreadLocal 不清
finalize	已废弃,生产禁用;资源释放在 try-with-resources
try-with-resources	try (资源) { };自动 close,逆序关闭;资源须实现 AutoCloseable
JVM 参数	-Xmx 设上限;-XX:+HeapDumpOnOutOfMemoryError 必开;-Xss 管递归深度
诊断流程	jps → jstat -gcutil → 堆转储 + MAT 支配树 → 改代码,别盲目加堆
OOM 与栈溢出	OutOfMemoryError 是堆耗尽;StackOverflowError 是递归过深;都是 Error

最小必会清单

- 能画出 JVM 五大内存区域并说出各自存什么、对应 C 的什么
- 能解释"栈上的引用、堆上的对象"与对象逃逸
- 能解释可达性分析与分代收集,说出 STW 的含义
- 能说出四种引用的回收条件,并写出 WeakHashMap 缓存
- 能说出 Java 内存泄漏的定义与四大源头,并写出修复
- 能写出 try-with-resources 并解释它与 C RAII 的关系

- 能说出 OOM 诊断四步流程

自测题

Q 1. 递归调用 10 万层会发生什么?怎么解决?

✓ 参考答案

StackOverflowError(不是 OOM):每层方法调用压一个栈帧,默认栈大小约 512KB-1MB,装不下 10 万帧。解决:改迭代(尾递归 Java 不优化)、加深思量后调 -Xss,或根本不该用递归(深度 $O(n)$ 的算法)。

Q 2. 判断:C 的内存泄漏是"忘了释放",Java 的内存泄漏是"忘了放手"。请解释"放手"指什么。

✓ 参考答案

C 的泄漏:malloc 后没有 free,内存永远占着。Java 的泄漏:对象仍然可达(被静态集合、长生命周期对象、线程持有),GC 永远不会回收它。所以"放手"= 断开不再需要的引用(remove 出集合、置 null、关闭资源)。

Q 3. 一个 WeakHashMap 缓存,键是 String,值为配置数据。为什么代码里 `cache.put("key", v)` 用字面量键,条目永远不消失?

✓ 参考答案

字符串字面量被常量池/元空间强引用,它的强引用链不断,弱引用键永远不会变 null,条目也就不会被清除。想演示弱引用行为,必须用 `new String("key")` 并断开局部变量——这也解释了为什么"缓存键尽量用业务对象"。

Q 4. 下面的代码有什么问题?`void read() throws Exception { FileInputStream in = new FileInputStream("a.txt"); int b = in.read(); System.out.println(b); }`

✓ 参考答案

文件句柄泄漏:read 抛异常或方法返回时 in 没关闭,底层 native 句柄泄漏。修复:`try (FileInputStream in = new FileInputStream("a.txt")) { ... }`,自动关闭,异常路径也不会漏。

Q 5. 生产环境出现 OOM:java.lang.OutOfMemoryError: Java heap space。给出排查步骤。

✓ 参考答案

① 确认是堆 OOM(而非 metaspace/线程);② jstat -gcutil 观察 GC 频率(是否 GC 风暴);③ 用 -XX:+HeapDumpOnOutOfMemoryError 生成的堆转储,在 MAT 里看支配树找"最大的对象+引用链";④ 修复代码(静态集合限长/释放资源/分批处理),必要时才调 -Xmx。不要一上来就加内存。

Q 6. 软引用与弱引用的区别?分别适合什么场景?

✓ 参考答案

软引用:内存不足时才回收,适合"丢了可惜但可重建"的缓存(大图片、计算结果);弱引用:下一次 GC 就回收,适合"键跟着业务对象走"的缓存与观察者注册。生产缓存框架(如 Caffeine)内部会综合两者做淘汰策略。

章末实验题:内存观察与资源管理

实验 5:内存观察与资源管理综合实验

实验目标:编写一个程序,综合运用本章全部知识点:内存区域观察、对象生命周期、GC 观察、弱引用缓存、内存泄漏修复、try-with-resources 资源管理。

实验要求:

- 任务 1:用 Runtime 打印 JVM 总内存与初始已用内存(覆盖 §5.1-5.2 堆与栈)
- 任务 2:创建大量短命大对象并打印每轮后的已用内存,观察 GC 回收(覆盖 §5.3 分代收集)
- 任务 3:实现一个 WeakReference 缓存,演示"业务键消失后缓存条目自动消失"(覆盖 §5.4 弱引用)
- 任务 4:实现一个静态 List 缓存并故意只增不减,随后加上限长修复,对比两者内存占用(覆盖 §5.5 泄漏与修复)
- 任务 5:自定义 AutoCloseable 资源类,用 try-with-resources 管理两个资源,观察关闭顺序(覆盖 §5.6)
- 任务 6:用递归制造 StackOverflowError,捕获后打印最大深度(覆盖 §5.7 栈区)
- 任务 7:程序运行结束后打印最终内存占用,输出完整观察报告(覆盖 §5.7 诊断)

实验环境:JDK 17+;建议 `java -Xmx256m MemoryLab` 运行,体会小堆下的 GC 压力。

第一步 写 main 骨架与任务 1、2(内存打印),跑通后再逐个实现 3-6

第二步 务 4 先跑泄漏版,用内存数字"眼见为实",再改限长版对比

第三步 务 5 观察关闭顺序:声明 a、b 两个资源,应输出"先关 b 后关 a"

参考答案要点

```

import java.lang.ref.WeakReference;
import java.util.ArrayList;
import java.util.List;

/** 内存观察与资源管理 — 第5章综合实验参考答案
 * 覆盖:堆/栈观察、GC 回收、弱引用缓存、泄漏修复、try-with-resources、栈溢出
 */
public class MemoryLab {

    // 任务 4:静态缓存(泄漏版)
    static final List<byte[]> CACHE = new ArrayList<>();
    static final int CACHE_LIMIT = 10;

    // 任务 5:自定义资源
    static class LogFile implements AutoCloseable {
        private final String name;

        LogFile(String name) {
            this.name = name;
            System.out.println(" 打开资源: " + name);
        }

        void write(String line) {
            System.out.println("    写入: " + name + " / " + line);
        }

        @Override
        public void close() {
            System.out.println(" 关闭资源: " + name);
        }
    }

    static long usedMb() {
        Runtime rt = Runtime.getRuntime();
        return (rt.totalMemory() - rt.freeMemory()) / 1024 / 1024;
    }

    // 任务 4 泄漏版:只进不出
    static void leakAdd(byte[] data) {
        CACHE.add(data);
    }

    // 任务 4 修复版:限长,超出丢最旧
    static void safeAdd(byte[] data) {
        if (CACHE.size() >= CACHE_LIMIT) {
            CACHE.remove(0);
        }
        CACHE.add(data);
    }

    // 任务 6:栈溢出
    static int depth = 0;

    static void dive() {
        depth++;
        dive();
    }

    public static void main(String[] args) throws Exception {
        System.out.println("== 任务 1:内存地图 ==");
        Runtime rt = Runtime.getRuntime();
        System.out.println(" 堆总内存(约): " + rt.totalMemory() / 1024 / 1024 + " MB");
        System.out.println(" 初始已用(约): " + usedMb() + " MB");

        System.out.println("== 任务 2:GC 回收观察 ==");
        for (int round = 1; round <= 3; round++) {
            for (int i = 0; i < 500; i++) {
                new byte[1024 * 256]; // 每轮制造约 128MB 垃圾
            }
        }
    }
}

```

```

    }
    System.gc(); // 教学用途;生产禁止
    Thread.sleep(100);
    System.out.println(" 第 " + round + " 轮后已用(约): " + usedMb() + " MB");
}

System.out.println("== 任务 3:弱引用缓存 ==");
byte[] payload = new byte[1024 * 512];
WeakReference<byte[]> ref = new WeakReference<>(payload);
System.out.println(" 断开强引用前: " + (ref.get() != null));
payload = null;
System.gc();
Thread.sleep(100);
System.out.println(" 断开强引用后: " + (ref.get() != null) + "(false=已被回收)");

System.out.println("== 任务 4:泄漏 vs 限长 ==");
long before = usedMb();
for (int i = 0; i < 30; i++) {
    safeAdd(new byte[1024 * 1024]); // 限长版:始终只有 10 条
}
System.gc();
Thread.sleep(100);
System.out.println(" 限长版缓存条目: " + CACHE.size() + ", 已用(约): " + usedMb() + " MB");
// 若改为 leakAdd:条目 30,已用内存显著增长—泄漏肉眼可见
CACHE.clear();
System.gc();
Thread.sleep(100);
System.out.println(" 清空后已用(约): " + usedMb() + " MB, 之前: " + before + " MB");

System.out.println("== 任务 5:try-with-resources ==");
try (LogFile a = new LogFile("a.log");
     LogFile b = new LogFile("b.log")) {
    a.write("hello");
    b.write("world");
} // 观察输出顺序:先关 b,后关 a(逆序关闭)

System.out.println("== 任务 6:栈溢出 ==");
try {
    dive();
} catch (StackOverflowError e) {
    System.out.println(" 递归最大深度约: " + depth);
}

System.out.println("== 任务 7:最终报告 ==");
System.out.println(" 结束已用(约): " + usedMb() + " MB");
System.out.println(" 实验完成:全部任务输出正常。");
}
}

```

设计说明:任务 2 用“每轮制造垃圾后内存不累积”直观证明 GC 在工作;任务 4 的 safeAdd 与 leakAdd 只需换一行即可对比泄漏;任务 5 输出“先关 b 后关 a”验证逆序关闭。扩展方向:任务 4 改成 WeakHashMap 缓存(与任务 3 呼应);用 -XX:+HeapDumpOnOutOfMemoryError 跑一个真正 OOM 的变体(把 safeAdd 改成无上限)并用 MAT 分析。

下一章预告:第6章 面向对象。类与对象、封装继承多态、equals/hashCode 契约、内部类与匿名类——内存管理教你怎么活,这一章教你用什么形状活。

第6章 面向对象

建议学时:5-7 小时 | 难度:★★★★☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 掌握类与对象的声明、构造器、this 与字段初始化
- 掌握封装与四种访问修饰符,理解"信息隐藏"的工程意义
- 掌握继承 extends、super、方法重写与 @Override 注解
- 理解多态与动态绑定,与 C 的虚函数表对照
- 掌握抽象类与接口的区别与选择,会写接口默认方法
- 掌握 equals/hashCode 契约:为什么必须同时重写、怎么写才对
- 掌握内部类、匿名类与 static 嵌套类的使用场景

💡 从 C 到 Java:面向对象的动机

C 程序员其实一直在写"半吊子面向对象":结构体 = 类字段; `shape_area(s)` 函数 = 方法; `void*` + 函数指针表 = 多态。Java 把这套手工活变成语言级设施:数据与操作它的函数被强制绑在一起(封装),类型体系显式表达"是什么/能干什么"(继承与接口),运行时按实际类型选方法(多态)。

继承在 C 里是"结构体嵌套 + 指针强转"——Linux 内核的 `list_head` 就是这么玩的,但类型不安全、容易错。Java 的 `extends` 让"子类是父类的一种"成为编译器检查的规则,漏了转型错误直接编译失败。

接口是 C 程序员最陌生的概念。它像"纯函数指针表 + 契约"的组合:类通过 `implements` 承诺"我能做这些事",调用方只依赖接口不依赖具体类——这是插件、框架、测试替身(mock)的地基。C 里你手工写 `struct ops { int (*area)(void*); }`,Java 里就是 `interface Shape { double area(); }`。

equals/hashCode 是重灾区。C 里比较结构体用 `memcmp`(有坑:padding 字节);Java 里 `==` 比较引用、`equals` 比较内容——但"内容相同"由你定义。写错 `hashCode` 的直接后果:HashMap/HashSet 里的元素找不到。这是面试与生产事故的高发区,\$6.6 给出唯一正确的写法模板。

学习顺序:\$6.1-6.4 建立"类/继承/多态"三板斧,\$6.5 接口,\$6.6 equals/hashCode 契约(必考必用),\$6.7-6.8 补充形态。

📦 前置知识自查

- C 结构体、函数指针、`void*` 类型擦除式多态
- 第1章:类型与数组;第4章:方法;第5章:对象在堆上的生命周期
- 第2章:`==` 与 `equals` 的区别将在本章展开

🚀 本章学习路线

1. **第一阶段(约 2 小时):**\$6.1-6.4,把 C 的 struct+函数指针 逐个翻译成 Java 类,体会"语言设施 vs 手工约定"
2. **第二阶段(约 1.5 小时):**\$6.5 接口与 \$6.7 匿名类,理解"面向接口编程"
3. **第三阶段(约 1.5 小时):**\$6.6 equals/hashCode 契约反复抄写模板,再完成章末实验
4. **验收标准:**能讲清"封装=让状态只被自己的方法修改";能写出放 HashMap 的键类;能说出接口与抽象类的选择依据

本章知识导图

- §6.1 类与对象:声明 / 构造器 / this
- §6.2 封装与访问控制:private / public / protected
- §6.3 继承:extends / super / 重写 / final
- §6.4 多态与动态绑定:与 C 虚函数对照
- §6.5 抽象类与接口:默认方法 / 选择依据
- §6.6 equals / hashCode 契约
- §6.7 内部类 / 匿名类 / static 嵌套类
- §6.8 static 成员与初始化顺序

§6.1 类与对象

6.1.1 从结构体到类

C 结构体	Java 类
struct Point { int x, y; };	class Point { int x, y; }
Point p = {1, 2};(栈上)	Point p = new Point();(堆上,字段默认 0/null)
函数:double dist(Point* p)	方法:double dist()(直接访问自己的字段)
初始化靠手写或 memset	构造器保证"对象一出生就合法"

示例 6-1 类的基本形态

```

public class PointDemo {
    static class Point {
        int x;           // 字段:默认 0
        int y;           // 字段:默认 0

        // 构造器:与类同名,没有返回类型,new 时自动执行
        Point(int x, int y) {
            this.x = x;       // this 区分"参数 x"与"字段 x"
            this.y = y;
        }

        // 无参构造器(重载):如果没写任何构造器,编译器会送一个默认的
        Point() {
            this(0, 0);       // 调用另一个构造器(必须是第一句)
        }

        double distance() {   // 方法:能直接摸自己的字段
            return Math.sqrt((double) x * x + y * y);
        }

        @Override
        public String toString() { // 第11章讲注解;这里标记"重写"
            return "(" + x + ", " + y + ")";
        }
    }

    public static void main(String[] args) {
        Point p = new Point(3, 4); // new = 分配堆内存 + 调构造器
        Point q = new Point();     // 用无参构造器
        System.out.println(p);     // (3, 4):println 自动调 toString
        System.out.println(p.distance()); // 5.0
        System.out.println(q);     // (0, 0)
    }
}

```

⚠ 易错点 1:忘了 new 与字段默认值

C 里 `Point p;` 是"申请栈空间,内容未定义";Java 里局部变量不初始化直接读会 编译报错 (比 C 安全);但字段(成员变量)不写也有默认值:int 0、double 0.0、boolean false、引用 null。"字段有默认值、局部变量没有"是新手最容易错记的规则。另外 `new Point()` 是"堆上造对象",`Point p;` 只是声明一个引用变量,不调用构造器。

§6.2 封装与访问控制

6.2.1 四种访问级别

修饰符	同类	同包	子类	任意处	C 的对应概念
private	✓				static 函数里的内部状态(头文件里藏起来)
(无,包私有)	✓	✓			同 .c 文件内的静态符号
protected	✓	✓	✓		"继承者可见"的半公开 API
public	✓	✓	✓	✓	头文件里导出的接口

示例 6-2 封装:状态只由方法修改

```

public class BankAccountDemo {
    static class BankAccount {
        private double balance;          // 私有:外面碰不到

        BankAccount(double init) {
            if (init < 0) throw new IllegalArgumentException("初始金额不能为负");
            this.balance = init;
        }

        public void deposit(double amount) {    // 公开接口
            if (amount <= 0) throw new IllegalArgumentException("金额必须为正");
            balance += amount;
        }

        // 取款:内部校验余额,这是"业务规则"放在方法里的意义
        public boolean withdraw(double amount) {
            if (amount <= 0) throw new IllegalArgumentException("金额必须为正");
            if (balance < amount) return false;    // 余额不足,拒绝
            balance -= amount;
            return true;
        }

        public double getBalance() {            // 只读访问器(getter)
            return balance;
        }
    }

    public static void main(String[] args) {
        BankAccount acc = new BankAccount(100);
        acc.deposit(50);
        acc.withdraw(30);
        System.out.println("余额: " + acc.getBalance());    // 120.0
        // acc.balance = 999999;    // 编译错误:balance 是 private
    }
}

```

★ 重点:封装的工程意义

封装不是"不让别人用",是 让状态的变化只能经过"你的规则"。C 里 `acc.balance = -5;` 随手就写;Java 里 `balance` 私有后,唯一的入口是带校验的 `deposit/withdraw`,非法状态在编译期就被挡在门外。生产要点:字段全 `private`,方法按需 `public`;需要只读暴露用 `getter`,不要直接开字段。校验失败用异常(第9章)而非静默容忍。

§6.3 继承:extends 与 super

6.3.1 继承 = is-a + 代码复用

子类 `extends` 父类:自动拥有父类的非私有字段与方法,可重写(override)方法改变行为, `super` 调用父类版本。Java 单继承(一个类只有一个父类)。构造器链:new 子类时,先从父类构造器开始(隐式调 `super()`),再到子类。

示例 6-3 继承与重写


```

public class InheritDemo {
    static class Animal {
        protected String name;           // protected:子类可见

        Animal(String name) { this.name = name; }

        public void speak() {
            System.out.println(name + " 发出声音");
        }
    }

    static class Dog extends Animal {      // 继承
        Dog(String name) { super(name); } // 必须第一句调用父类构造器

        @Override                         // 注解:让编译器检查"真的重写了"
        public void speak() {
            System.out.println(name + " 汪汪叫");
        }

        public void fetch() {              // 子类新增能力
            System.out.println(name + " 捡球");
        }
    }

    public static void main(String[] args) {
        Dog d = new Dog("旺财");
        d.speak();                         // 旺财 汪汪叫(重写生效)
        d.fetch();                         // 旺财 捡球(子类独有)

        Animal a = new Dog("大黄");        // 子类对象赋给父类引用:向上转型
        a.speak();                         // 大黄 汪汪叫(动态绑定,见 §6.4)
        // a.fetch();                      // 编译错误:Animal 没有 fetch(静态类型说了算)
    }
}

```

⚠ 易错点 2:重写与重载是两回事

重写(override):子类方法签名与父类完全一致,行为替换;加 @Override 让编译器把关(签名写错会编译报错)。**重载(overload)**:同一类里同名不同参(第4章)。C 没有这两个概念——重写在 C 里靠函数指针表替换,重载靠名字不同 (printf_float 之类的命名)。**重写看对象类型,重载看引用类型**,这个区分在第4章与本章的交叉点高频出错。

§6.4 多态与动态绑定

6.4.1 C 的虚函数表 vs Java 的动态绑定

C 的面向对象多态: `struct ShapeOps { double (*area)(void*); }`, 每个形状结构体里放自己的函数指针表,调用 `ops->area(shape)` 时按"表里存的函数"分发。Java 做的是一模一样的事,由 JVM 自动完成: **方法调用时,按对象的实际类型(不是引用类型)查找方法并执行**——这就是动态绑定(多态)。

示例 6-4 多态:一个接口,多种实现

```

public class PolyDemo {
    static abstract class Shape {           // 抽象类($6.5):不能 new
        abstract double area();           // 抽象方法:子类必须实现
    }

    static class Circle extends Shape {
        final double r;
        Circle(double r) { this.r = r; }
        double area() { return Math.PI * r * r; }
    }

    static class Rect extends Shape {
        final double w, h;
        Rect(double w, double h) { this.w = w; this.h = h; }
        double area() { return w * h; }
    }

    public static void main(String[] args) {
        Shape[] shapes = {new Circle(1), new Rect(2, 3), new Circle(2)};
        double total = 0;
        for (Shape s : shapes) {
            total += s.area();              // 运行时按实际类型分发!
        }
        System.out.printf("总面积 = %.2f\n", total);    // 3.14+6+12.57 = 21.71
        // C 的写法:shapes[i].ops->area(shapes[i]);Java 自动完成
    }
}

```

★ 重点:多态解决什么问题

多态让"新增一种形状"不需要修改汇总代码:再写一个 `Triangle extends Shape`,上面循环零改动就能算面积。开闭原则:对扩展开放,对修改关闭——这是框架、插件、测试替身的技术根基。生产中的日常形态:接口定义契约,多个实现类,调用方只依赖接口(面向接口编程)。

§6.5 抽象类与接口

6.5.1 两者的区别与选择

对比项	抽象类	接口
关键词	abstract class	interface
继承方式	extends(单继承)	implements(可多实现)
字段	可有实例字段	只能有常量(public static final)
构造器	有(子类走 super)	没有
方法	抽象方法 + 普通方法	抽象方法 + 默认方法(default) + 静态方法
语义	"is-a" 且共享状态/逻辑	"能做什么"的契约
C 的对应	结构体 + 部分实现的函数指针	纯函数指针表(ops 结构)

示例 6-5 接口与默认方法

```

public class InterfaceDemo {
    // 接口:定义"能做什么"
    interface Sortable {
        int size();

        boolean less(int i, int j);

        void swap(int i, int j);

        // 默认方法:JDK8+,给接口补"附带实现"的能力
        default void sort() { // 冒泡排序,直接可复用
            for (int i = 0; i < size() - 1; i++) {
                for (int j = 0; j < size() - 1 - i; j++) {
                    if (less(j + 1, j)) {
                        swap(j, j + 1);
                    }
                }
            }
        }
    }
}

// 实现接口:只需三个抽象方法,sort 白送
static class IntArray implements Sortable {
    private final int[] data;

    IntArray(int[] data) { this.data = data; }

    public int size() { return data.length; }

    public boolean less(int i, int j) { return data[i] < data[j]; }

    public void swap(int i, int j) {
        int t = data[i];
        data[i] = data[j];
        data[j] = t;
    }
}

public static void main(String[] args) {
    IntArray arr = new IntArray(new int[]{5, 3, 8, 1, 9, 2});
    arr.sort(); // 默认方法:一行排序
    System.out.println(java.util.Arrays.toString(arr.data)); // [1, 2, 3, 5, 8, 9]
}
}

```

⚠ 易错点 3:接口 vs 抽象类的选择

选择口诀: "要共享状态、且语义是 is-a" → 抽象类; "定义契约、允许多角色" → 接口。典型错误:为了复用一段代码就继承一个不相干的类(比如让 Account 继承 ArrayList)——继承是"is-a"关系,不是"拿来用"。Java 类单继承、接口可多实现,接口是组合式的"能力声明",抽象类才是"家族血统"。

§6.6 equals 与 hashCode 契约

6.6.1 为什么必须同时重写

`==` 比较引用是否同一个;Object 的默认 `equals` 就是 `==`。想让"内容相同"成立,重写 `equals`。而 `HashMap/HashSet` 靠 `hashCode` 分桶:两个对象 `equals` 为 `true`,则 `hashCode` 必须相等;`hashCode` 相等不要求 `equals` (碰撞)。只重写 `equals` 不重写 `hashCode`: `map.put(k1, v); map.get(k2) (k1.equals(k2))` 返回 `null`——因为 `k2` 哈希到了另一个桶。这就是契约:违反的代价是集合失灵。

示例 6-6 equals/hashCode 标准模板(JDK7+ 写法)

```

import java.util.HashMap;
import java.util.Map;
import java.util.Objects;

public class EqualsDemo {
    static class Student {
        private final String id;        // 学号:判等依据
        private final String name;

        Student(String id, String name) {
            this.id = id;
            this.name = name;
        }

        // 模板步骤:① 自反 ② 类型判断 ③ 字段逐个比
        @Override
        public boolean equals(Object o) {
            if (this == o) return true;           // 同引用,直接相等
            if (!(o instanceof Student)) return false; // 类型不符(含 null)
            Student s = (Student) o;
            return id.equals(s.id) && name.equals(s.name);
        }

        // 与 equals 用同一组字段
        @Override
        public int hashCode() {
            return Objects.hash(id, name); // JDK7+:一行生成
        }

        @Override
        public String toString() {
            return id + ":" + name;
        }
    }

    public static void main(String[] args) {
        Map<Student, String> grades = new HashMap<>();
        Student s1 = new Student("2024001", "张三");
        Student s2 = new Student("2024001", "张三"); // 内容相同,引用不同
        grades.put(s1, "A");

        System.out.println(s1 == s2);           // false:引用不同
        System.out.println(s1.equals(s2));       // true:内容相同
        System.out.println(grades.get(s2));     // A:靠 hashCode 找到同桶 + equals 命中

        // 反例:只改 equals 不改 hashCode 会导致 get 返回 null
    }
}

```

⚠ 易错点 4:equals 的对称性与可变字段

两条生产铁律: ① equals/hashCode 依据的字段必须是"不可变"的 (final 或从不修改)——字段变了,对象的哈希就变了,放进 HashSet 后再改字段,元素会"失踪"; ② 继承场景重写 equals 要小心对称性:父类 p.equals(子类 s) 为 true 但 s.equals(p) 为 false 会破坏契约——生产建议:需要子类判等时,用"字段比较"而非 instanceof,或干脆组合优先于继承。

§6.7 内部类、匿名类与静态嵌套类

6.7.1 三种形态与用途

- **静态嵌套类**: static class Inner, 不持有外部类引用,当"把相关类型聚在一起"用(第4章实验里的 Point 就是)
- **内部类**: 普通内部类,隐式持有外部类引用,可访问外部私有成员;常用于"服务外部类的小助手"
- **匿名类**: new 接口() { 实现 }, 一次性实现;在 lambda 出现后,单方法接口优先用 lambda

示例 6-7 匿名类与 lambda 的等价替换

```
import java.util.Arrays;
import java.util.Comparator;

public class AnonymousDemo {
    public static void main(String[] args) {
        String[] names = {"Tom", "Alice", "Bob"};

        // 匿名类:JDK8 之前唯一的"内联实现"方式
        Arrays.sort(names, new Comparator<String>() {
            @Override
            public int compare(String a, String b) {
                return a.length() - b.length();
            }
        });

        // lambda:完全等价,现代写法
        Arrays.sort(names, (a, b) -> a.length() - b.length());

        // 方法引用:最短
        Arrays.sort(names, Comparator.comparingInt(String::length));

        System.out.println(Arrays.toString(names));    // [Tom, Bob, Alice]
    }
}
```



工程建议:匿名类的去留

新代码里,函数式接口一律用 lambda;匿名类只留给"多个方法的接口需要一次性实现"的场景。匿名类持有外部引用,可能造成意外捕获(比如在回调里误用了可变外部变量)——lambda 捕获 effectively final 变量(\$4.5),匿名类同样受限,但写法上更容易犯错。看到旧代码里的匿名类,能读、能改写成 lambda 即可。

§6.8 static 成员与初始化顺序

6.8.1 static = 属于类,不属于对象

static 字段:所有实例共享一份(相当于 C 的全局变量,但归属类); **static 方法**:不依赖实例(工具方法,第4章); **static 初始化块**:类第一次加载时执行一次。初始化顺序:父类 static → 子类 static → 父类字段/构造 → 子类字段/构造。用 static 字段做计数器是经典用法。

示例 6-8 静态计数器与初始化顺序

```

public class StaticDemo {
    static class Employee {
        private static int nextId = 1;    // 类级:所有实例共享
        private final int id;            // final 字段:构造时定死

        static {                          // 静态块:类加载时执行一次
            System.out.println("[静态块] Employee 类加载");
        }

        Employee() {
            this.id = nextId++;           // 每 new 一个,编号递增
        }

        int getId() { return id; }
    }

    public static void main(String[] args) {
        Employee e1 = new Employee();
        Employee e2 = new Employee();
        Employee e3 = new Employee();
        System.out.println(e1.getId() + "," + e2.getId() + "," + e3.getId());
        // 1,2,3:编号是类级计数器分配的,实例间共享
        System.out.println(Employee.nextId);    // 4:static 字段可直接类名访问
    }
}
// 输出:[静态块] Employee 类加载 只出现一次—类只加载一次

```

⚠ 易错点 5:static 方法里不能碰实例成员

`static` 方法没有 `this`——在 `static` 方法里访问实例字段/实例方法,编译报错。C 的对等直觉:static 方法 = "不传 `this` 的全局函数"。main 是 `static` 的,所以所有代码都在 `static` 上下文里开场,想用实例成员必须 `new` 出来。另一个坑:static 字段是全局的,多线程并发读写要加锁(第9章)。

本章速查表

主题	要点
类 vs 结构体	类=字段+方法+构造器;new 在堆上造对象;字段有默认值,局部变量没有
构造器	与类同名无返回;this(...) 链式调用必须第一句;不写构造器有默认无参版
访问控制	private 最严 → 包私有 → protected(子类可见)→ public;字段一律 private
封装	状态只经方法修改;getter 只读暴露;非法状态在方法内拦截
继承	extends 单继承;构造器链先父后子;super 调父类;final 类不可继承
重写	签名相同、行为替换,必须加 @Override;与重载(同名不同参)区分
多态	按实际类型动态绑定;C 的虚函数表自动版;开闭原则
抽象类	abstract class;可含字段与实现;is-a 共享状态时用
接口	implements 可多实现;纯契约;default 方法 JDK8+
equals/hashCode	equals 相同 → hashCode 必须相同;模板:this==o / instanceof / 字段比;Objects.hash 生成
判等字段	必须不可变;变了会让 HashSet 元素"失踪"
三种嵌套类	static 嵌套(无外部引用)/ 内部类(有外部引用)/ 匿名类(一次性;单方法用 lambda)
static 成员	属于类;static 方法无 this 不能用实例成员;初始化顺序:父 static → 子 static → 构造

最小必会清单

- 能写出含构造器、this、getter 的类,并说出字段默认值与局部变量的区别
- 能说出四种访问修饰符的可见范围与 C 的对应
- 能写出继承 + super + @Override 重写的代码,并解释构造器链顺序
- 能解释多态与动态绑定,说出它与 C 虚函数表的关系
- 能说出抽象类与接口的选择依据,会写接口 default 方法
- 能默写 equals/hashCode 标准模板并解释契约
- 能说出三种嵌套类的区别,会把匿名类改写为 lambda

自测题

Q 1. 下列哪些是合法重写(有 @Override)?父类 `void f(int x)`:① `void f(int x)`;② `int f(int x)`;③ `void f(long x)`;④ `void f(int x) throws Exception`

✓ 参考答案

只有 ① 合法。② 返回类型不同且不是协变,编译错误;③ 参数不同是重载不是重写(父类方法仍存在);④ 重写方法不能抛比父类更宽泛的 checked 异常——Exception 比父类的(无)宽,编译错误。

Q 2. 为什么 HashMap 的键类必须同时重写 equals 和 hashCode?只重写 equals 会怎样?

✓ 参考答案

HashMap 先用 hashCode 定位桶,再用 equals 在桶内比较。两个“内容相同”的键,若 hashCode 不同就落入不同桶,get 用第二个键查找时找不到第一个键存入的值。契约:equals 相同 → hashCode 必须相同。所以必须用同一组字段同时实现两者。

Q 3. 一个 Student 类以学号 id 为判等字段。把两个内容相同的 Student 放入 HashSet 后,再调用 set 里那个对象的 setId("2024999"),会发生什么?

✓ 参考答案

元素“失踪”:hashCode 依赖 id,id 变了哈希变,但对象在 HashSet 里还是按旧哈希所在桶存放。contains/remove 用新哈希查找,找不到。这就是“判等字段必须不可变”的原因——这也是为什么生产代码的 id 通常 final。

Q 4. 接口与抽象类各自适合什么场景?举一个“该用接口却用了抽象类”的反例。

✓ 参考答案

接口:定义“能做什么”的契约,允许多角色(如 Runnable、Comparable);抽象类:is-a 关系且要共享状态与实现。反例:让 Cat extends Dog 只为复用 eat()——没有血缘关系却继承,应改成 interface Eatable + 组合。判断标准:问“子类是父类的一种吗?”,不是就绝不继承。

Q 5. 写出结果:父类构造器打印“A”,子类构造器打印“B”,new Child() 输出什么?如果父类只有带参构造器呢?

✓ 参考答案

输出“A”然后“B”:子类构造器隐式调用 super(),先执行父类构造器。若父类没有无参构造器,子类必须显式 super(参数)且必须是构造器第一句,否则编译错误。C 的对应:结构体嵌套时先初始化内层成员。

Q 6. 为什么生产代码里字段要 private?直接 public 不更简单吗?

✅ 参考答案

public 字段把"状态"与"规则"分离:任何代码都能改成非法值(负余额、空名称)。private + 方法入口,让所有修改经过校验逻辑;同时改内部实现(比如 balance 从 double 改 BigDecimal)不影响调用方。这正是 C 里"头文件只暴露函数,结构体定义藏在 .c 里"的同一门手艺。

章末实验题:教务管理系统

📄 实验 6:教务管理系统

实验目标:编写一个小型教务管理系统,综合运用本章全部知识点:类与构造器、封装、继承、多态、接口、equals/hashCode、匿名类/lambda。

实验要求:

- 任务 1:定义 `Person` 类:私有 name/age 字段 + 构造器 + getter + toString(覆盖 §6.1-6.2 类与封装)
- 任务 2:定义 `Student extends Person`:新增学号 id,重写 toString;定义 `Teacher extends Person`:新增课程 course(覆盖 §6.3 继承)
- 任务 3:定义接口 `Assessable` (能考核): `double score()`;Student 按成绩、Teacher 按课时返回(覆盖 §6.4-6.5 多态与接口)
- 任务 4:给 Student 按学号实现 equals/hashCode,用 HashSet 去重:同号学生只留一个(覆盖 §6.6 契约)
- 任务 5:用 lambda 按成绩降序排序并打印前 3 名(覆盖 §6.7 函数式与比较器)
- 任务 6:定义静态计数器统计创建的学生总数(覆盖 §6.8 static)
- 任务 7:main 输出完整报告:人员名单、去重结果、排名(覆盖全部)

实验环境:JDK 17+; javac SchoolSystem.java 编译运行。

第一步 写 Person/Student/Teacher 骨架,编译通过后再加接口与判等

第二步 任务 4 验证去重:put 两个同号不同名对象,set.size() 应为 1

第三步 任务 5 与第4章的 Stream 结合:用 stream().sorted(...) 完成排名

✅ 参考答案要点


```

import java.util.ArrayList;
import java.util.HashSet;
import java.util.List;
import java.util.Objects;
import java.util.Set;

/** 教务管理系统 — 第6章综合实验参考答案
 * 覆盖:类/封装、继承、多态、接口、equals/hashCode、lambda、static
 */
public class SchoolSystem {

    // 任务 1:基类,封装 name/age
    static class Person {
        private final String name;
        private final int age;

        Person(String name, int age) {
            this.name = name;
            this.age = age;
        }

        public String getName() { return name; }

        public int getAge() { return age; }

        @Override
        public String toString() { return name + "(" + age + ")"; }
    }

    // 任务 2:学生,继承 Person,新增学号
    static class Student extends Person {
        private final String id;
        private final double examScore;    // 期末成绩

        Student(String id, String name, int age, double examScore) {
            super(name, age);    // 先父后子
            this.id = id;
            this.examScore = examScore;
        }

        public String getId() { return id; }

        @Override
        public String toString() { return id + ":" + super.toString(); }

        // 任务 4:按学号判等(判等字段 final,永不改变)
        @Override
        public boolean equals(Object o) {
            if (this == o) return true;
            if (!(o instanceof Student)) return false;
            Student s = (Student) o;
            return id.equals(s.id);
        }

        @Override
        public int hashCode() { return Objects.hash(id); }

        // 任务 3:实现接口的方法
        @Override
        public double score() { return examScore; }
    }

    // 任务 2:教师
    static class Teacher extends Person {
        private final String course;

        Teacher(String name, int age, String course) {
            super(name, age);
        }
    }

```

```

        this.course = course;
    }

    @Override
    public String toString() { return super.toString() + " 教 " + course; }

    @Override
    public double score() { return 40.0; }    // 课时考核分(演示)
}

// 任务 3:考核接口(能力契约)
interface Assessable {
    double score();
}

// 任务 6:静态计数器
private static int totalCreated = 0;

static <T extends Person> T register(T person) {
    totalCreated++;
    return person;
}

public static void main(String[] args) {
    List<Person> people = new ArrayList<>();
    people.add(register(new Student("2024001", "张三", 19, 88.5)));
    people.add(register(new Student("2024002", "李四", 20, 76.0)));
    people.add(register(new Student("2024003", "王五", 18, 93.0)));
    people.add(register(new Teacher("陈老师", 35, "数据结构")));
    people.add(register(new Teacher("刘老师", 41, "操作系统")));
    System.out.println("任务6 共登记 " + totalCreated + " 人");

    System.out.println("== 任务1-3 人员名单(多态:toString 各打各的) ==");
    for (Person p : people) {
        System.out.println(" " + p);
        if (p instanceof Assessable) {
            System.out.println("    考核分 = " + p.score());    // 动态绑定
        }
    }

    System.out.println("== 任务4 按学号去重 ==");
    Set<Student> students = new HashSet<>();
    students.add(new Student("2024001", "张三", 19, 88.5));
    students.add(new Student("2024001", "张山", 20, 90.0));    // 同号:应去重
    students.add(new Student("2024002", "李四", 20, 76.0));
    System.out.println("    去重后学生数: " + students.size() + " (应为 2)");

    System.out.println("== 任务5 按成绩降序前3名 ==");
    students.stream()
        .sorted((a, b) -> Double.compare(b.score(), a.score()))    // lambda 比较器
        .limit(3)
        .forEach(s -> System.out.println(" " + s + " 成绩 " + s.score()));

    System.out.println("== 任务7 报告完成 ==");
}
}

```

设计说明:任务 3 把 score() 做成接口方法,main 里统一按 Assessable 调用——新增“行政人员”类只需实现接口,统计代码零改动(开闭原则);任务 4 用“同号不同名”验证 equals 按学号判等,同时演示了为什么判等字段要 final;任务 5 的 Double.compare 比直接减号更安全(防 NaN 与溢出)。扩展方向:成绩从控制台输入(第3章)、用 Map<String, Student> 按学号索引(第6章)、异常处理(第9章)、record 简化 Person(第14章)。

下一章预告:第7章 文件与 I/O。字节流与字符流、Files/Path API、序列化与反序列化——把数据落盘,是程序离开内存的第一步。

第7章 文件与 I/O

建议学时:4-6 小时 | 难度:★★★☆☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 掌握字节流与字符流两大家族,理解"流 = 数据的单向管道"
- 掌握 FileInputStream/FileOutputStream 与缓冲流的正确组合
- 理解字符编码问题:为什么中文乱码、怎么读 UTF-8 文件
- 掌握现代 NIO.2 API:Path/Files 读写、拷贝、遍历
- 掌握对象序列化与反序列化,理解 serialVersionUID 的作用
- 掌握 Properties 配置文件的读写
- 掌握生产级文件处理:try-with-resources、追加写、路径规范

💡 从 C 到 Java:文件世界的翻译

C 里的一切都是 `FILE*`:`fopen/fread/fwrite/fclose`。Java 里没有 `FILE*`,取而代之的是"流(stream)"体系:字节流(`InputStream/OutputStream`)按字节搬数据,字符流(`Reader/Writer`)按字符搬数据。而字符流底层还是字节流——中间垫着一层"编码转换器"。C 的 `printf/fprintf` 到 Java 是 `PrintStream/PrintWriter`。

最大的坑是编码。C 程序员在 Linux 上写文件,`char` 数组默认 UTF-8,基本没乱码问题;Java 里"读文件用什么字符集"是显式决策——`new FileReader(file)` 用平台默认字符集(Windows 上可能是 GBK!),读 UTF-8 文件直接乱码。**铁律:**读写文本一律显式指定 UTF-8。乱码问题 90% 由此而来。

NIO.2 是"现代姿势"。老 API(`File`、`FileInputStream`)啰嗦且容易漏关;JDK7 的 `Path / Files` 把"读整个文件、拷贝、移动、遍历"变成一行调用,配合 `try-with-resources` 几乎没有资源泄漏机会。生产代码应当以 NIO.2 为主,老 API 能读懂即可。

序列化是 C 没有的直接对应物。C 存结构体用 `fwrite` 裸写内存(字节序、版本问题一堆);Java 的 `Serializable` 把对象图整体变成字节流,再原样恢复。它不跨语言、有安全风险,但"Java 到 Java 的持久化/传输"非常方便——生产里常用于缓存与 RPC 传输(现代框架更推荐 JSON/Protobuf,本章 §7.7 讲选择)。

学习顺序:先建"流"的心智模型(§7.1-7.3),再学现代 API(§7.4),然后序列化与配置(§7.5-7.6),最后工程实践。

📦 前置知识自查

- C 文件操作:`fopen` 模式(`r/w/a`)、`fread/fwrite`、`fgets/fprintf`、`feof` 判断
- 第5章:`try-with-resources` 资源管理(本章全程使用)
- 第1章:`byte` 类型与 `char`/字符串概念;字节与字符的区别

🚀 本章学习路线

1. **第一阶段(约 1.5 小时):**§7.1-7.3 字节流/字符流与编码,动手制造一次乱码再修好
2. **第二阶段(约 1.5 小时):**§7.4 NIO.2,把老 API 的例子逐行改成 `Files` 写法
3. **第三阶段(约 1.5 小时):**§7.5 序列化 + §7.6 Properties,完成章末实验
4. **验收标准:**能写出"显式 UTF-8 读文本文件"的代码;能用 `Files` 拷贝文件;能序列化并恢复对象

本章知识导图

- §7.1 流的世界观:字节流 / 字符流 / 流包装
- §7.2 字节流实战:文件复制与缓冲
- §7.3 字符流与编码:Reader / Writer / UTF-8 铁律
- §7.4 NIO.2:Path / Files 现代 API
- §7.5 序列化与反序列化:serialVersionUID
- §7.6 Properties 配置文件
- §7.7 工程实践:追加日志 / 路径 / 格式选型

§7.1 流的世界观

7.1.1 四大基类与包装模式

方向	字节流(处理 byte)	字符流(处理 char/字符串)	C 对应
读	InputStream(FileInputStream 等)	Reader(FileReader 等)	fread / fgets
写	OutputStream(FileOutputStream 等)	Writer(Writer 等)	fwrite / fprintf

包装模式是 Java 流的灵魂:基础流只管"连接数据源/目的地",装饰流负责"加缓冲、转字符、按行读"。一个完整流 = 基础流 + 若干装饰流,最外层装饰流决定"用起来多方便"。读文本文件的经典组合:

```
FileInputStream -> InputStreamReader -> BufferedReader
```

对应到 C:FILE* 内部其实也是"系统 fd + 用户态缓冲"的两层结构,只是 C 帮你包好了;Java 把每一层都拆开让你选。

★ 重点:流的五个常识

- 流是 单向 的:读流不能写,写流不能读;要读写一个文件就开两个流
- 流必须 关闭 :try-with-resources 包住最外层流,内层自动关
- 读到最后返回 -1 (字节流)或 null (readLine),不是异常
- 写满缓冲要 flush() :缓冲流把数据攒着,不 flush 可能丢尾巴
- 流是 顺序 访问:没有 fseek 的自由定位不是日常需求,需要时用 RandomAccessFile(少见)

§7.2 字节流实战:文件复制

示例 7-1 字节流文件复制(经典三段式)

```

import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.BufferedInputStream;
import java.io.BufferedOutputStream;
import java.io.IOException;

public class CopyBytes {
    public static void main(String[] args) throws IOException {
        // C 的 fopen("rb"/"wb") 对应:FileInputStream/FileOutputStream
        // 缓冲流:一次读 8KB 到内存,而不是逐字节系统调用
        try (var in = new BufferedInputStream(new FileInputStream("a.bin"));
            var out = new BufferedOutputStream(new FileOutputStream("b.bin"))) {
            byte[] buf = new byte[8192];
            int n;
            while ((n = in.read(buf)) != -1) { // 读一坨;返回实际字节数
                out.write(buf, 0, n); // 只写实际读到的部分
            }
        } // 两个流自动关闭(先关 out 后关 in)
        System.out.println("复制完成");
    }
}

```

⚠ 易错点 1:write 一定要用 "buf, 0, n"

最后一次 read 可能只填满一部分缓冲区,用 `out.write(buf, 0, n)` 只写有效部分;写 `out.write(buf)` 会把上一次残留的脏字节也写进去,文件末尾出现垃圾数据。这是文件复制代码最高频的 bug。同理,read 的返回值 n 每次都要检查。

§7.3 字符流与编码

7.3.1 为什么乱码:字节与字符之间隔着一层"翻译官"

磁盘上只有字节。"中" 在 UTF-8 里是 3 个字节 `E4 B8 AD`,在 GBK 里是 2 个字节 `D6 D0`。字符流就是"翻译官":读时按指定字符集把字节翻译成字符,写时反向翻译。翻译官选错(字符集不匹配) = 乱码。两个铁律:

1. 读文本文件: `FileReader` 必须显式传 UTF-8,否则用平台默认(Windows 常见 GBK/UTF-8 混乱)
2. 写文本文件:同一条铁律;写完 flush 或关闭

示例 7-2 字符流:显式 UTF-8 读 + 写

```

import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;

public class TextIo {
    public static void main(String[] args) throws IOException {
        Path path = Path.of("notes.txt");

        // 写法 1(老风格):手动组装装饰流
        try (var in = new BufferedReader(
            new java.io.InputStreamReader(
                new java.io.FileInputStream(path.toFile()),
                StandardCharsets.UTF_8))) {
            String line;
            while ((line = in.readLine()) != null) { // 读一行;结尾返回 null
                System.out.println("读到: " + line);
            }
        }

        // 写法 2(现代):Files.newBufferedReader 一行拿到 UTF-8 BufferedReader
        try (var in = Files.newBufferedReader(path, StandardCharsets.UTF_8)) {
            String line;
            while ((line = in.readLine()) != null) {
                System.out.println("再读: " + line);
            }
        }

        // 写: newBufferedWriter 同样带字符集参数
        try (var out = Files.newBufferedWriter(path, StandardCharsets.UTF_8)) {
            out.write("第一行\n");
            out.write("第二行\n"); // 关流时自动 flush
        }
    }
}

```

⚠ 易错点 2:FileReader/FileWriter 的默认字符集陷阱

`new FileReader(file)` 与 `new FileWriter(file)` 用 **平台默认字符集**——在 Windows 上通常是 GBK,在 Linux 服务器上通常是 UTF-8。同一份代码两个平台行为不同,文件跨机器拷贝就乱码。生产规则:这两类直接禁用裸构造,要么用 `new InputStreamReader(in, StandardCharsets.UTF_8)`,要么用 `Files.newBufferedReader(path, UTF_8)`。字符集问题必须在编译时就写死,不能依赖环境。

§7.4 NIO.2:Path 与 Files

7.4.1 现代文件 API 全家桶

`Path` 描述位置(对应 C 的路径字符串,但类型安全),`Files` 是静态工具类(对应 C 的 `stat/rename/unlink` 等系统调用)。日常 90% 的文件操作 `Files` 都有现成方法:

操作	Files 方法	C 对应
读整个文件	Files.readString(path, UTF_8) / readAllLines	read + 循环
写整个文件	Files.writeString(path, text, UTF_8)	fwrite
拷贝	Files.copy(src, dst, REPLACE_EXISTING)	手工循环复制
移动/改名	Files.move(src, dst)	rename()
删除	Files.deleteIfExists(path)	unlink()
属性	Files.size / exists / isDirectory	stat()
遍历目录	Files.list / Files.walk(递归)	readdir 循环
创建目录	Files.createDirectories(可多级)	mkdir -p 的手工版

示例 7-3 Files 全家桶实战

```
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.StandardCopyOption;

public class FilesDemo {
    public static void main(String[] args) throws IOException {
        // 多级目录一次建好(等价 mkdir -p)
        Path dir = Path.of("data", "2026", "08");
        Files.createDirectories(dir);

        // 写整个文件:UTF-8 显式指定;文本默认不换行,自己加 \n
        Path f = dir.resolve("report.txt"); // resolve = 拼路径
        Files.writeString(f, "2026-08-26 实验报告\n第7章\n", StandardCharsets.UTF_8);

        // 读整个文件(小文件专用)
        String content = Files.readString(f, StandardCharsets.UTF_8);
        System.out.println("内容:\n" + content);

        // 追加写:Append 参数
        Files.writeString(f, "追加一行\n", StandardCharsets.UTF_8,
            java.nio.file.StandardOpenOption.APPEND);

        // 拷贝 + 覆盖
        Path bak = dir.resolve("report.bak.txt");
        Files.copy(f, bak, StandardCopyOption.REPLACE_EXISTING);

        // 遍历目录(非递归)
        try (var stream = Files.list(dir)) {
            stream.forEach(p -> System.out.println("条目: " + p.getFileName()));
        }

        // 递归遍历
        try (var walk = Files.walk(Path.of("data"))) {
            walk.filter(p -> p.toString().endsWith(".txt"))
                .forEach(p -> System.out.println("txt 文件: " + p));
        }

        // 清理:删除备份
        Files.deleteIfExists(bak);
        System.out.println("done");
    }
}
```

工程建议:Files 的边界

`readString/readAllLines` 只适合小文件 (几 MB 以内);大文件必须流式处理(\$7.2 的缓冲流或 `Files.lines`)。 `Files.list` 返回的是流,必须 `try-with-resources` 关闭,否则句柄泄漏。另外 `Path.of` 与 `Paths.get` 等价,新代码用 `Path.of`。

§7.5 序列化与反序列化

7.5.1 对象 → 字节流 → 对象

类实现 `java.io.Serializable` (空接口,标记用),对象就能交给 `ObjectOutputStream` 写成字节,再用 `ObjectInputStream` 原样读回。 `serialVersionUID` 是版本号:类结构变了(增删字段),UID 变了,老数据反序列化会抛 `InvalidClassException`。生产铁律:每个 `Serializable` 类都显式声明 `serialVersionUID`,不依赖编译器生成。

示例 7-4 序列化与版本控制

```
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;
import java.io.IOException;
import java.io.Serializable;
import java.nio.file.Files;
import java.nio.file.Path;

public class SerializeDemo {
    // 实现 Serializable 标记接口:不写任何方法
    static class Student implements Serializable {
        private static final long serialVersionUID = 1L;    // 显式版本号:铁律!

        final String name;
        final int score;

        Student(String name, int score) {
            this.name = name;
            this.score = score;
        }

        @Override
        public String toString() { return name + ":" + score; }
    }

    public static void main(String[] args) throws Exception {
        Path f = Path.of("student.dat");

        // 写:对象 → 字节流
        try (var out = new ObjectOutputStream(
            Files.newOutputStream(f))) {
            out.writeObject(new Student("张三", 88));
            out.writeObject(new Student("李四", 76));
        }

        // 读:字节流 → 对象(顺序必须与写一致)
        try (var in = new ObjectInputStream(
            Files.newInputStream(f))) {
            Student s1 = (Student) in.readObject();    // 需要强转
            Student s2 = (Student) in.readObject();
            System.out.println(s1 + " / " + s2);        // 张三:88 / 李四:76
        }
    }
}
```


⚠ 易错点 3:反序列化的两大风险

① 安全风险:反序列化等于执行"数据里的指令",从不信任的来源(网络、用户上传)反序列化会被攻击者利用(经典 RCE 链)。生产原则:反序列化只用于"自己写、自己读"的数据;跨系统传输用 JSON(Jackson/Gson)。② 兼容性:字段增删会让旧数据读不出来——serialVersionUID 不变时,新增字段读回默认值(null/0),删除字段的数据会丢;对格式敏感的场景考虑版本字段或 JSON。

§7.6 Properties 配置文件

7.6.1 键值对配置的标准读法

Properties 读写 `key=value` 格式的 .properties 文件,是 Java 最传统的配置方式(现代项目也用 YAML,但 Properties 依然到处可见)。加载后用 `getProperty` 取配置,带默认值防 NPE。

示例 7-5 Properties 读写

```
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.Properties;

public class ConfigDemo {
    public static void main(String[] args) throws IOException {
        Path cfg = Path.of("app.properties");

        // 写配置(演示;生产配置一般由运维/安装器生成)
        Properties p = new Properties();
        p.setProperty("server.port", "8080");
        p.setProperty("app.name", "教务系统");
        try (var out = Files.newBufferedWriter(cfg, StandardCharsets.UTF_8)) {
            p.store(out, "生成于演示"); // 注释头 + 键值对
        }

        // 读配置:标准生产姿势
        Properties loaded = new Properties();
        try (var in = Files.newBufferedReader(cfg, StandardCharsets.UTF_8)) {
            loaded.load(in);
        }
        int port = Integer.parseInt(
            loaded.getProperty("server.port", "8080")); // 带默认值
        String name = loaded.getProperty("app.name", "未知");
        System.out.println("端口=" + port + ", 应用=" + name);
        // 取不到的键返回 null;getProperty(key, def) 给默认值
    }
}
```

🚀 工程建议:配置的读取时机

配置在启动时读一次,放进常量/不可变对象,不要在每次请求时重新读文件(慢且浪费)。生产配置常配合环境变量覆盖:代码读系统属性/环境变量优先,Properties 兜底: `System.getenv("PORT")` 存在就用它。配置中心(Nacos/Spring Cloud Config)本质还是"远程拉一份配置到本地再走这套逻辑"。

§7.7 工程实践

7.7.1 追加日志与路径规范

- **追加写日志:** `Files.writeString(path, line, UTF_8, StandardOpenOption.APPEND, CREATE)` ——多线程追加用日志框架(logback),别自己并发写文件

- **路径规范:**用 Path 拼接而不是字符串 + (Windows 分隔符 \ 与 Linux / 的差异由 Path 处理); .. 处理用 normalize
- **相对路径依赖 cwd:**生产用绝对路径或配置注入,不要依赖"当前目录"

7.7.2 格式选型速查

场景	选型	理由
Java 内部持久化/缓存	序列化 / 新版用 record	对象图整体保存,最省事
跨语言/对外 API	JSON(Jackson)	人类可读、生态最大、安全
高性能二进制	Protobuf / 消息队列自带	体积小、解析快、带 schema
简单键值配置	Properties / YAML	人写人读
大文件流式处理	缓冲流 + 循环	不占内存

本章速查表

主题	要点
四大基类	InputStream/OutputStream(字节)、Reader/Writer(字符);流单向、要关闭
装饰流	BufferedInputStream/BufferedReader 等;外层包内层;try-with-resources 包最外层
读文件	read 返回 -1 结束;readLine 返回 null 结束;write 必须用 write(buf, 0, n)
编码铁律	文本读写一律显式 StandardCharsets.UTF_8;禁用裸 FileReader/FileWriter
NIO.2	Path.of / Files.readString / writeString / copy / move / walk;小文件专用
大文件	缓冲流 + 8KB 数组循环;Files.lines 流式
序列化	implements Serializable + 显式 serialVersionUID;读写顺序一致;不信任来源禁反序列化
Properties	load/store;getProperty(key, 默认值) 防 NPE;启动时读一次
追加写	Files.writeString(path, s, UTF_8, APPEND, CREATE)
路径	用 Path 拼接,不用字符串 +;生产用绝对路径或配置注入
格式选型	Java 内部→序列化;跨语言→JSON;高性能→Protobuf
乱码排查	确认文件实际编码(hexdump 看字节)→ 确认读写字符集一致 → 修改代码显式 UTF-8

最小必会清单

- 能画出"FileInputStream → InputStreamReader → BufferedReader"的包装链并解释每层职责
- 能写出字节流文件复制代码(含 write(buf, 0, n) 的正确姿势)
- 能解释乱码成因,并写出显式 UTF-8 读写文本的两行代码
- 能用 Files 完成:读整个文件、写整个文件、拷贝、遍历目录
- 能写出序列化类(含 serialVersionUID)并说出安全红线
- 能读写 Properties 配置并说出默认值参数的用途

自测题

Q 1. 读一个 UTF-8 文件出现乱码,可能的原因有哪些?排查顺序?

 **参考答案**

原因:① 读取字符集不是 UTF-8(如 FileReader 用了平台默认 GBK);② 文件本身不是 UTF-8(GBK/ANSI 编码);③ 多次转换叠加损坏。排查:先用工具确认文件字节(如 hexdump 看"中"是 E4B8AD 还是 D6D0),再确认代码里字符集参数,最后统一为显式 UTF-8 读写。乱码一旦写回文件就是不可逆损坏,务必先备份。

Q 2. 下面代码有什么问题? `byte[] buf = new byte[4096]; in.read(buf); out.write(buf);`

✓ 参考答案

read 的返回值没检查,且 write 写整个 buf。最后一次 read 可能不足 4096 字节,write(buf) 会把上次残留内容一起写进去,文件尾部出现垃圾。正确写法: `int n; while ((n = in.read(buf)) != -1) out.write(buf, 0, n);`

Q 3. 序列化版本兼容:类加了 `serialVersionUID = 1L` 后新增一个字段,读旧数据会怎样?为什么显式声明 UID?

✓ 参考答案

UID 不变时,新增字段读回默认值(引用 null、数字 0),不抛异常——这是"宽松兼容";删除字段的数据会丢失。若没显式声明 UID,编译器根据类结构自动生成,增删字段后 UID 变化,读旧数据抛 `InvalidClassException`。所以显式声明让版本行为可预期,并且改版时你能主动决定是否 bump 版本。

Q 4. 从不可信来源(用户上传文件)反序列化,为什么危险?

✓ 参考答案

反序列化会实例化数据中指定的类并调用其方法,攻击者可以构造恶意字节流触发危险类(如 Runtime、JNDI 相关)执行任意代码——著名的反序列化 RCE。生产铁律:只对可信数据反序列化;跨系统用 JSON 并校验字段白名单。

Q 5. 需要把 2GB 的日志文件按行统计行数,用 `readString` 还是流式?写出代码。

✓ 参考答案

```
long count = 0;
try (var lines = Files.lines(Path.of("big.log"), StandardCharsets.UTF_8)) {
    count = lines.count();
}
System.out.println("行数: " + count);
```

必须流式:readString 会把 2GB 塞进内存直接 OOM。Files.lines 返回惰性流,一行一行读,内存占用与文件大小无关。同理大文件处理用 stream() 的过滤/映射,别先读全量。

Q 6. 追加日志时用 `writeString` 加 APPEND,多个线程同时写会怎样?正确做法?

✓ 参考答案

并发写同一文件会互相覆盖/交错,出现损坏或丢行。生产用日志框架(logback/log4j2)管理文件追加、滚动与并发;实在要手写,用单线程写队列或加锁,但那是重复造轮子。多线程与锁在本书第9章展开。

章末实验题:成绩单文件处理器

实验 7:成绩单文件处理器

实验目标:编写一个成绩单处理程序,综合运用本章全部知识点:字节流、字符流、编码、NIO.2、序列化、Properties。

实验要求:

- 任务 1:程序启动时用 Properties 读取 config.properties(端口无关,读报告标题与输出目录)(覆盖 §7.6 配置)
- 任务 2:用 Files 读取 CSV 成绩文件 scores.csv(UTF-8,每行"姓名,分数"),按行解析(覆盖 §7.4 NIO.2 读)
- 任务 3:计算平均分、最高分、及格人数,用 Files.writeString 生成 report.txt 报告(覆盖 §7.4 NIO.2 写)
- 任务 4:用缓冲字符流把原始成绩逐行"复制"成 backup.csv 备份(覆盖 §7.2-7.3 流式复制)
- 任务 5:定义 Student 类(含 serialVersionUID),把解析结果整体序列化到 students.dat,再读回验证(覆盖 §7.5 序列化)
- 任务 6:用 Files.copy 把报告复制一份到备份目录,并列出输出目录全部文件(覆盖 §7.4 拷贝/遍历)
- 任务 7:输出完整处理报告(覆盖全部)

实验环境:JDK 17+;先手工创建 scores.csv(至少 5 行数据,注意存成 UTF-8)。

第一步 写数据类 Student 与 CSV 解析,再用 System.out 验证解析正确,再落盘

第二步 务 5 注意:Student 要实现 Serializable 且两个字段建议 final

第三步 有流都包 try-with-resources;写完立即验证文件内容(打开看看)

参考答案要点

```

import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;
import java.io.Serializable;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.StandardCopyOption;
import java.nio.file.StandardOpenOption;
import java.util.ArrayList;
import java.util.List;
import java.util.Properties;

/** 成绩单文件处理器 — 第7章综合实验参考答案
 * 覆盖:Properties、NIO.2 读写、流式复制、序列化、拷贝与遍历
 */
public class ScoreFileProcessor {

    // 任务 5:序列化数据类型,显式版本号
    static class Student implements Serializable {
        private static final long serialVersionUID = 1L;

        final String name;
        final int score;

        Student(String name, int score) {
            this.name = name;
            this.score = score;
        }

        @Override
        public String toString() { return name + ":" + score; }
    }

    public static void main(String[] args) throws Exception {
        // 任务 1:读取配置(带默认值)
        Properties cfg = new Properties();
        if (Files.exists(Path.of("config.properties"))) {
            try (var in = Files.newBufferedReader(Path.of("config.properties"),
                StandardCharsets.UTF_8)) {
                cfg.load(in);
            }
        }
        String title = cfg.getProperty("report.title", "期末成绩报告");
        String outDir = cfg.getProperty("output.dir", "out");
        Path out = Path.of(outDir);
        Files.createDirectories(out); // 目录不存在就建
        System.out.println("== 任务 1 配置: " + title + " / " + outDir);

        // 任务 2:Files 读 CSV 并解析
        List<Student> students = new ArrayList<>();
        try (var reader = Files.newBufferedReader(Path.of("scores.csv"),
            StandardCharsets.UTF_8)) {
            String line;
            while ((line = reader.readLine()) != null) {
                line = line.trim();
                if (line.isEmpty() || line.startsWith("#")) {
                    continue; // 跳过空行与注释
                }
                String[] parts = line.split(",");
                if (parts.length != 2) {
                    System.out.println("跳过格式错误行: " + line);
                    continue;
                }
                students.add(new Student(parts[0].trim(),
                    Integer.parseInt(parts[1].trim())));
            }
        }
    }
}

```

```

    }
    System.out.println("== 任务 2 解析 " + students.size() + " 条成绩");

    // 任务 3:统计并写报告
    int sum = 0, pass = 0, max = 0;
    for (Student s : students) {
        sum += s.score;
        if (s.score >= 60) pass++;
        if (s.score > max) max = s.score;
    }
    double avg = students.isEmpty() ? 0 : (double) sum / students.size();
    StringBuilder report = new StringBuilder();
    report.append("=== ").append(title).append(" ===\n");
    report.append("人数: ").append(students.size()).append("\n");
    report.append(String.format("平均分: %.1f\n", avg));
    report.append("最高分: ").append(max).append("\n");
    report.append("及格人数: ").append(pass).append("\n");
    Path reportFile = out.resolve("report.txt");
    Files.writeString(reportFile, report.toString(), StandardCharsets.UTF_8);
    System.out.println("== 任务 3 报告已生成: " + reportFile);

    // 任务 4:缓冲字符流逐行备份(流式,不整文件进内存)
    Path backup = out.resolve("backup.csv");
    try (var reader = Files.newBufferedReader(Path.of("scores.csv"),
        StandardCharsets.UTF_8);
        var writer = Files.newBufferedWriter(backup, StandardCharsets.UTF_8)) {
        String line;
        while ((line = reader.readLine()) != null) {
            writer.write(line);
            writer.newLine();
        }
    }
    System.out.println("== 任务 4 备份完成: " + backup);

    // 任务 5:序列化整个列表,再读回验证
    Path dat = out.resolve("students.dat");
    try (var oos = new ObjectOutputStream(Files.newOutputStream(dat))) {
        oos.writeObject(students);
    }
    try (var ois = new ObjectInputStream(Files.newInputStream(dat))) {
        @SuppressWarnings("unchecked")
        List<Student> restored = (List<Student>) ois.readObject();
        System.out.println("== 任务 5 反序列化验证: " + restored.size() + " 条,"
            + "首条 " + restored.get(0));
    }

    // 任务 6:拷贝报告 + 遍历目录
    Path copy = out.resolve("report.backup.txt");
    Files.copy(reportFile, copy, StandardCopyOption.REPLACE_EXISTING);
    System.out.println("== 任务 6 输出目录内容:");
    try (var list = Files.list(out)) {
        list.sorted().forEach(p ->
            System.out.println("    " + p.getFileName() + " (" + sizeOf(p) + " B)"));
    }

    System.out.println("== 任务 7 处理完成 ==");
}

static long sizeOf(Path p) {
    try {
        return Files.size(p);
    } catch (Exception e) {
        return -1;
    }
}
}

```

设计说明:任务 4 特意用流式逐行复制(与任务 2 的整文件读形成对照):CSV 如果很大,readString 会 OOM,而这里内存占用恒定。任务 5 的 @SuppressWarnings("unchecked") 是强转泛型列表的标准姿势。扩展方向:用 Files.walk 递归处理子目录、把统计结果同时写 JSON 版本(第13章引入第三方库时会讲)、对超大文件用 Stream 并行统计(第10章)。

下一章预告:第8章 网络通信。Socket 编程与 HttpClient——TCP 的 Java 封装、HTTP 客户端、网络编程的坑与安全红线。

第8章 网络通信

建议学时:4-6 小时 | 难度:★★★★☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 掌握 Java 的 TCP 编程:ServerSocket 与 Socket,与 C 的 socket 系列函数对照
- 掌握多客户端处理模型:每连接一线程的经典架构
- 掌握 UDP 编程:DatagramSocket 的发送与接收
- 掌握 HttpClient:GET/POST、超时、错误处理(JDK11+)
- 理解网络编程的坑:阻塞、半关闭、超时、粘包与拆包
- 树立网络安全红线:不信任远端输入、防注入

💡 从 C 到 Java:网络编程的手工活变少了

C 的网络编程是"仪式感"的代名词:socket() → bind() → listen() → accept(),每步都要检查返回值、处理 errno,再手写 read/write 循环。Java 把这套流程压缩成两个类:服务端 ServerSocket(accept 即完成 listen+accept),客户端 Socket(connect 完成握手)。连接的读写直接就是第7章的 InputStream/OutputStream——网络流和文件流是同一套 API,这是 Java 的设计哲学:一切都是流。

最大的思维差异是阻塞。C 的 recv 阻塞,Java 的 in.read 同样阻塞——没有数据就一直挂着。C 程序员用 select/poll/epoll 解决多连接;Java 经典方案是"每连接一个线程":accept 到连接就 new 一个线程去读,主循环继续 accept。这背后是第9章要展开的线程模型。现代高并发(Netty、虚拟线程)是另一套,但"一连接一线程"依然是理解一切的起点。

HTTP 是 C 程序员的"手工活重灾区":手写请求行、解析响应头、处理 chunked——Java 的 java.net.http.HttpClient (JDK11+)把 GET/POST、超时、重定向、异步全部内置,十几行完成 C 里上百行的活。生产调第三方 REST API,这是标准姿势。

安全是网络章节的必修课:C 时代的心智是"自己的程序互相通信";今天的网络程序面对的是不可信输入——JSON 注入、路径穿越、无认证端口裸奔。本章每个示例都带安全笔记,红线在第8.7节。

学习顺序:TCP 先建立连接模型(\$8.2-8.3),UDP 快速对照(\$8.4),然后 HTTP(\$8.5),最后是坑与安全(\$8.6-8.7)。

🧰 前置知识自查

- C 网络基础:socket/bind/listen/accept/connect、recv/send、TCP 三次握手概念
- 第7章:流式读写与 try-with-resources(网络读写复用同一套)
- 第4章:方法;基本的多线程直觉(第9章将完整展开)

🚀 本章学习路线

1. **第一阶段(约 1.5 小时):**\$8.2-8.3 跑通"回显服务器 + 客户端",用两个终端观察收发
2. **第二阶段(约 1 小时):**\$8.4 UDP 对照;\$8.5 HttpClient 请求真实接口
3. **第三阶段(约 1.5 小时):**\$8.6-8.7 坑与安全,完成章末实验
4. **验收标准:**能写出 TCP 回显服务器;能用 HttpClient 带超时地 GET 并解析响应;能说出三条安全红线

本章知识导图

- §8.1 网络编程总览:与 C socket 家族对照
- §8.2 TCP 编程:ServerSocket / Socket
- §8.3 多客户端:每连接一线程
- §8.4 UDP 编程:DatagramSocket
- §8.5 HttpClient:GET / POST / 超时
- §8.6 网络编程的坑:阻塞 / 半关闭 / 粘包
- §8.7 安全红线:不可信输入

§8.1 网络编程总览

8.1.1 与 C 的 socket 家族对照

C 函数	Java 对应	说明
socket(AF_INET, SOCK_STREAM, 0)	new ServerSocket(port) / new Socket(host, port)	构造即创建,类型由类决定
bind + listen	ServerSocket 构造器内部完成	一次搞定
accept	server.accept() → Socket	阻塞等待连接
connect	new Socket(host, port)	构造即连接(可带超时)
recv / send	socket.getInputStream() / getOutputStream()	就是第7章的流!
close	socket.close()	关闭连接
errno	IOException(异常体系)	异常替代错误码

★ 重点:网络连接 = 两端各有一对流

TCP 连接建立后,两端各拿到"对方的输入流/输出流":你写出去的数据出现在对端的读流里,对端写的数据从你的读流进来。所以网络编程 = 第7章的流编程 + 连接的建立与关闭管理。别被"网络"两个字吓住,它只是流的另一种数据源。

§8.2 TCP 编程:回显服务器

示例 8-1 TCP 回显服务器(单连接版)

```

import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.IOException;
import java.io.InputStreamReader;
import java.io.OutputStreamWriter;
import java.net.ServerSocket;
import java.net.Socket;
import java.nio.charset.StandardCharsets;

public class EchoServer {
    public static void main(String[] args) throws IOException {
        int port = 9000;
        // 构造即 bind + listen; C 的三步压缩成一步
        try (ServerSocket server = new ServerSocket(port)) {
            System.out.println("监听端口 " + port + " ...");

            // accept 阻塞: 等一个客户端连上来
            try (Socket client = server.accept()) {
                System.out.println("客户端接入: " + client.getRemoteSocketAddress());

                // 网络流: 显式 UTF-8, 与文件读写同一套姿势
                var in = new BufferedReader(new InputStreamReader(
                    client.getInputStream(), StandardCharsets.UTF_8));
                var out = new BufferedWriter(new OutputStreamWriter(
                    client.getOutputStream(), StandardCharsets.UTF_8));

                String line;
                while ((line = in.readLine()) != null) {
                    System.out.println("收到: " + line);
                    out.write("回显: " + line + "\n"); // 原样弹回
                    out.flush(); // 缓冲流必须刷
                }
                System.out.println("客户端断开");
            }
        }
    }
}

```

■ 示例 8-2 TCP 客户端

```

import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.IOException;
import java.io.InputStreamReader;
import java.io.OutputStreamWriter;
import java.net.Socket;
import java.nio.charset.StandardCharsets;

public class EchoClient {
    public static void main(String[] args) throws IOException {
        // 构造即 connect; SocketTimeoutException 表示连不上
        try (Socket sock = new Socket("127.0.0.1", 9000)) {
            var in = new BufferedReader(new InputStreamReader(
                sock.getInputStream(), StandardCharsets.UTF_8));
            var out = new BufferedWriter(new OutputStreamWriter(
                sock.getOutputStream(), StandardCharsets.UTF_8));

            out.write("你好, 服务器\n");
            out.flush();
            System.out.println("服务器回应: " + in.readLine()); // 阻塞等回应
        }
    }
}
// 运行顺序: 先启动 EchoServer, 再启动 EchoClient

```

⚠ 易错点 1:阻塞与超时

readLine 没有数据时 **永远阻塞**——它不会超时。生产代码必须设置超时: `sock.setSoTimeout(3000)`, 超时后抛 `SocketTimeoutException`(`IOException` 子类), 循环里 catch 后决定重试或断开。另一个隐蔽点: 客户端不主动 close, 服务器端 readLine 就一直等, 看不到"客户端断开"——用 finally 或 try-with-resources 保证 close。

§8.3 多客户端:每连接一线程

8.3.1 架构:accept 循环 + 工作线程

单连接版只能服务一个客户端。经典多客户端模型: **主循环只做 accept; 每接一个连接, 交给一个新线程处理, 主循环立刻回去等下一个**。这正是 C 里 pthread 版服务器的 Java 对应, 线程创建用 Executors(第9章详解, 这里先用最简单的)。

示例 8-3 多客户端聊天服务器(核心结构)

```
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.net.ServerSocket;
import java.net.Socket;
import java.nio.charset.StandardCharsets;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class ChatServer {
    // 线程池:固定 10 个工作线程,避免无限制创建线程(第9章详解)
    private static final ExecutorService pool = Executors.newFixedThreadPool(10);

    public static void main(String[] args) throws IOException {
        try (ServerSocket server = new ServerSocket(9100)) {
            System.out.println("聊天服务器启动, 端口 9100");
            while (true) {
                // 主循环只做一件事:等连接
                Socket client = server.accept();
                // 把连接交给工作线程,立刻回去 accept 下一个
                pool.submit(() -> handle(client));
            }
        }
    }

    private static void handle(Socket client) {
        // 每个客户端一个线程:阻塞读自己的流,互不干扰
        try (client; // try-with-resources 关连接
            var in = new BufferedReader(new InputStreamReader(
                client.getInputStream(), StandardCharsets.UTF_8))) {
            System.out.println "[" + Thread.currentThread().getName()
                + "] 新用户: " + client.getRemoteSocketAddress());
            String line;
            while ((line = in.readLine()) != null) {
                System.out.println "[" + Thread.currentThread().getName()
                    + "] 广播: " + line);
                // 完整版:把消息转发给所有在线的客户端(需要维护连接列表)
            }
            System.out.println("用户离开: " + client.getRemoteSocketAddress());
        } catch (IOException e) {
            System.out.println("连接异常: " + e.getMessage());
        }
    }
}
```

工程建议:线程模型的演进路线

"一连接一线程"的正确打开方式是线程池,不是裸 new Thread :裸创建在高并发下会把内存与 CPU 打爆。演进路线:一连接一线程(好理解)→ 线程池(控制资源)→ NIO 事件驱动(Netty,大量连接少量活跃)→ 虚拟线程(JDK21+,极轻量线程,回到"一连接一线程"的写法但无成本)。理解第一代,是理解后面的前提。

§8.4 UDP 编程

8.4.1 无连接的数据报

UDP 不需要 accept/connect 握手: DatagramPacket 装着"数据 + 目标地址",DatagramSocket 负责发送与接收。数据报有大小上限(约 64KB)、可能丢失或乱序。适合:DNS、音视频、游戏位置同步、心跳探测。

示例 8-4 UDP 收发

```
import java.net.DatagramPacket;
import java.net.DatagramSocket;
import java.net.InetAddress;
import java.nio.charset.StandardCharsets;

public class UdpDemo {
    public static void main(String[] args) throws Exception {
        // 发送端:一次发一包
        try (DatagramSocket sender = new DatagramSocket()) {
            byte[] data = "ping".getBytes(StandardCharsets.UTF_8);
            DatagramPacket pkt = new DatagramPacket(data, data.length,
                InetAddress.getByAddress("127.0.0.1"), 9200);
            sender.send(pkt);
            System.out.println("已发送 UDP 包");
        }

        // 接收端:绑定端口,等待数据报(先启动接收端)
        try (DatagramSocket receiver = new DatagramSocket(9200)) {
            byte[] buf = new byte[1024];
            DatagramPacket pkt = new DatagramPacket(buf, buf.length);
            receiver.receive(pkt); // 阻塞等包
            String msg = new String(pkt.getData(), 0, pkt.getLength(),
                StandardCharsets.UTF_8);
            System.out.println("收到: " + msg + " 来自 "
                + pkt.getAddress() + ":" + pkt.getPort());
        }
    }
}
```

易错点 2:UDP 的四个"不保证"

UDP 不保证:送达、顺序、不重复、不丢包。业务需要可靠性就必须在应用层补(序号、重传、确认),或者干脆用 TCP。经典错误:把 UDP 当 TCP 用,丢包后一脸茫然。选型判断:丢一包无所谓的心跳→ UDP;一字节都不能丢的(转账)→ TCP。另外 UDP 包默认有大小上限,应用层要控制单包数据量。

§8.5 HttpClient:现代 HTTP 客户端

8.5.1 GET 与 POST

java.net.http.HttpClient (JDK11+)是内置的 HTTP 客户端: HttpRequest 描述请求,HttpClient 发送,HttpResponse 承载响应。GET 简单、POST 带 body;超时与重定向策略在客户端构建时配置。C 里手写 HTTP 的苦活在这里是标准库能力。

示例 8-5 HttpClient 实战

```

import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.time.Duration;

public class HttpDemo {
    public static void main(String[] args) throws Exception {
        // 客户端:全局一个,配置超时与跟随重定向
        HttpClient client = HttpClient.newBuilder()
            .connectTimeout(Duration.ofSeconds(5)) // 连接超时
            .followRedirects(HttpClient.Redirect.NORMAL)
            .build();

        // ---- GET ----
        HttpRequest getReq = HttpRequest.newBuilder()
            .uri(URI.create("https://httpbin.org/get"))
            .timeout(Duration.ofSeconds(10)) // 请求超时
            .GET()
            .build();
        HttpResponse<String> resp =
            client.send(getReq, HttpResponse.BodyHandlers.ofString());
        System.out.println("GET 状态码: " + resp.statusCode()); // 200
        System.out.println("GET 响应体前 120 字符: "
            + resp.body().substring(0, 120));

        // ---- POST(JSON body)----
        String json = "{\"name\":\"张三\",\"score\":88}";
        HttpRequest postReq = HttpRequest.newBuilder()
            .uri(URI.create("https://httpbin.org/post"))
            .header("Content-Type", "application/json")
            .POST(HttpRequest.BodyPublishers.ofString(json))
            .build();
        HttpResponse<String> postResp =
            client.send(postReq, HttpResponse.BodyHandlers.ofString());
        System.out.println("POST 状态码: " + postResp.statusCode());
        System.out.println("POST 响应体前 120 字符: "
            + postResp.body().substring(0, 120));
    }
}

```

★ 重点:状态码与错误处理

send 只在“网络层失败”(连不上、超时)抛 IOException;业务失败(404、500)是正常的 HTTP 响应。必须检查 statusCode():2xx 成功、3xx 重定向(可自动跟随)、4xx 客户端错、5xx 服务端错。生产姿势:封装“响应校验 + 重试策略 + 熔断”,但初学先记住:不检查状态码 = 把错误当成功。

§8.6 网络编程的坑

坑	现象	对策
阻塞	read 永远等不到数据,程序挂死	setSoTimeout 超时 + catch SocketTimeoutException
半关闭	一端不再写,另一端 readLine 返回 null 才知道;close 直接断双方	用 shutdownOutput() 只关写方向;约定 报文结束标记
粘包/拆包	一次 read 读到多条消息(粘包)或一条 被拆成多次(拆包)	应用层协议必须定边界:按行(文本)/ 长度 前缀 / JSON 对象
半包写	write 一次没写完,对方 read 到半截数 据	write 的返回值检查;BufferedWriter 全行 写 + flush
端口占用	BindException: Address already in use	换端口;服务端优雅关闭 (ServerSocket.close 释放)

示例 8-6 粘包演示与按行协议修复

```
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.net.Socket;
import java.nio.charset.StandardCharsets;

public class StickyDemo {
    // 场景:客户端一口气 write 三条消息,服务端一次 read 可能全读到
    // 修复:应用层约定"一行一条消息",用 readLine 按行切分
    public static void main(String[] args) throws IOException {
        // 假设已拿到连接 socket
        Socket sock = new Socket("127.0.0.1", 9100);
        sock.setSoTimeout(3000); // 3 秒没数据就超时
        try (var in = new BufferedReader(new InputStreamReader(
            sock.getInputStream(), StandardCharsets.UTF_8))) {
            String line;
            while ((line = in.readLine()) != null) {
                System.out.println("一条消息: " + line);
            }
        } catch (java.net.SocketTimeoutException e) {
            System.out.println("读超时:本次无更多数据(约定:超时即结束)");
        }
    }
}
```

⚠ 易错点 3:协议设计是网络程序的第一设计决策

TCP 是字节流,没有"消息"概念——消息边界必须由应用层协议定义。三条主路:① 文本行协议(每行一条,readLine);② 长度前缀(4 字节长度 + payload);③ 自描述格式(JSON/Protobuf,生产首选)。C 程序员常用"固定结构体 + sizeof"——Java 里同样可行(长度前缀),但二进制协议的字节序要显式处理(DataOutputStream 的 writeInt 是网络序,别手搓)。

§8.7 安全红线

⚠ 易错点 4:网络安全的四条红线

1. 不信任任何远端输入:长度检查、类型检查、白名单校验;网络数据当"可能被攻击者精心构造"对待
2. 防止注入:把网络数据拼进 SQL/命令/路径前必须转义或参数化;路径穿越(../)用 normalize 后检查前缀
3. 认证与授权:暴露端口的服务必须有认证(令牌/API Key),别裸奔;生产环境监听 0.0.0.0 前想清楚
4. 加密传输:涉及敏感数据的传输用 HTTPS/TLS(Java 的 `HttpsURLConnection`/`HttpClient` 原生支持),不要自己实现加密

示例 8-7 红线落地:输入校验模板

```
import java.util.regex.Pattern;

public class SanitizeDemo {
    // 学号白名单:只允许 7 位数字;其余一律拒绝
    private static final Pattern STUDENT_ID = Pattern.compile("\\d{7}");

    // 校验模板:长度 + 内容双重把关,失败抛异常而不是返回"猜的值"
    static String sanitizeNick(String raw) {
        if (raw == null || raw.length() > 20 || raw.length() < 1) {
            throw new IllegalArgumentException("昵称长度必须在 1-20");
        }
        String trimmed = raw.trim();
        if (trimmed.contains("
") || trimmed.contains("
")) {
            throw new IllegalArgumentException("昵称不能包含换行");    // 防协议注入
        }
        return trimmed;
    }

    static boolean isStudentId(String s) {
        return s != null && STUDENT_ID.matcher(s).matches();
    }
}
```

本章速查表

主题	要点
TCP 服务端	<code>new ServerSocket(port) → accept() → 读写流 → close</code> ;三步合一
TCP 客户端	<code>new Socket(host, port)</code> 即连接; <code>getInputStream/getOutputStream</code> 就是流
阻塞	<code>read</code> 无数据永远等; <code>setSoTimeout(ms) + catch SocketTimeoutException</code>
多客户端	<code>accept</code> 循环 + 线程池提交连接处理;别裸 <code>new Thread</code>
UDP	<code>DatagramPacket(数据+地址) + DatagramSocket</code> ;无连接;不保证送达/顺序/不丢
UDP 选型	心跳/音视频/丢包无所谓 → UDP;一字节不能丢 → TCP
HttpClient	<code>HttpRequest + HttpClient.send + HttpResponse</code> ;body 用 <code>BodyHandlers.ofString()</code>
状态码	404/500 是正常响应不是异常;必须检查 <code>statusCode</code> ;超时才抛 <code>IOException</code>
粘包/拆包	TCP 无消息边界;协议三选一:行协议 / 长度前缀 / JSON
半关闭	<code>close</code> 断双向;只停写用 <code>shutdownOutput</code> ;对方 EOF = <code>readLine</code> null
超时配置	HttpClient: <code>connectTimeout</code> + 请求 <code>timeout</code> 都要设
安全红线	不信任输入 / 防注入 / 要认证 / 加密传输(HTTPS)

最小必会清单

- 能默写 TCP 回显服务器骨架(ServerSocket/accept/流/关闭)
- 能写出带超时的客户端读取循环
- 能说出"每连接一线程 + 线程池"架构并画出主循环流程
- 能说出 UDP 与 TCP 的选型判断
- 能用 HttpClient 完成带超时的 GET 并检查状态码
- 能解释粘包成因并给出三种协议对策
- 能说出四条网络安全红线

自测题

Q 1. 服务器 accept 到一个连接后,客户端发来 100 字节,服务端一次 read(buf) 可能读到多少?为什么?

✓ 参考答案

可能是 1~100 之间的任意值(还可能拆成多次读):TCP 是字节流,数据到达的时间与分割不受发送方 write 次数约束。所以必须循环 read 直到读够协议要求的量(长度前缀协议)或读到行结束符(行协议),不能假设一次 read 就是一条消息。

Q 2. 客户端突然断电,服务端的 readLine 会怎样?怎么及时发现?

✓ 参考答案

断电时 TCP 无正常 FIN,服务端 readLine 一直阻塞,可能永远等不到。对策:setSoTimeout 让读超时;应用层心跳(定期发 ping,超过 N 次没回应判死)。OS 层面 TCP keepalive 是另一个兜底,但间隔太长,生产一般靠应用层心跳。

Q 3. HttpClient.send 抛异常与状态码 500 有什么区别?

✓ 参考答案

send 抛 IOException:网络层失败(连不上、DNS 解析失败、超时)——程序不知道服务器是否处理了请求。状态码 500:请求到达服务器,服务器内部出错——服务器知道、也返回了明确答复。处理策略不同:前者可重试(幂等请求),后者看业务(500 往往重试也无用)。

Q 4. 服务器需要同时服务 5000 个在线连接,但多数连接长期空闲。为什么"裸线程每连接"不合适?现代解法?

✓ 参考答案

每线程约 1MB 栈(第5章),5000 线程占几 GB 虚拟内存,且线程切换开销大。传统解法:Netty 的 NIO 事件驱动(一个线程管大量连接);JDK21+ 虚拟线程:轻量到百万级,恢复"一连接一线程"的简单写法。生产判断:连接量级千级以下 → 线程池够用;万级以上 → Netty 或虚拟线程。

Q 5. 客户端发来字符串,服务器要把它拼进 SQL 查询。有什么风险?怎么办?

✓ 参考答案

SQL 注入:字符串里带 ' OR 1=1 -- 之类的内容会改写查询语义。对策:使用参数化查询(PreparedStatement 的 ? 占位),绝不字符串拼接 SQL;同时做输入白名单校验(如学号必须是数字)。这是第8.7节红线的具体落地。

Q 6. UDP 协议为什么不适合传输"订单支付确认"?

✓ 参考答案

UDP 不保证送达与顺序:支付确认丢了,用户以为没支付成功,业务就错了。UDP 适合"丢失可容忍、实时性优先"的场景(游戏位置、语音、心跳)。业务关键数据用 TCP(或基于 UDP 的应用层可靠性协议 QUIC,但那仍是"可靠传输"的语义)。

章末实验题:命令聊天室 + 天气查询

实验 8:命令聊天室与 HTTP 查询

实验目标:编写两部分程序:① 多客户端命令行聊天室(服务端 + 客户端);② 用 HttpClient 查询公共 API 并解析结果。综合运用本章全部知识点。

实验要求:

- 任务 1:服务端维护在线客户端集合,收到消息后广播给所有在线者(覆盖 §8.2-8.3 TCP 与多线程)
- 任务 2:每连接一线程用线程池;客户端断开时从集合移除(覆盖 §8.3)
- 任务 3:客户端提供昵称,发送格式 "昵称: 消息",服务端广播时带发送者(覆盖 §8.2 客户端)
- 任务 4:客户端设置 5 秒读超时,超时后提示并继续(覆盖 §8.6 超时处理)
- 任务 5:服务端与客户端之间采用"一行一条消息"的行协议(覆盖 §8.6 粘包对策)
- 任务 6:用 HttpClient GET 查询 <https://httpbin.org/ip>,解析出 origin 字段打印(覆盖 §8.5 HTTP)
- 任务 7:POST 一条 JSON 到 <https://httpbin.org/post>,打印回显的 json 字段(覆盖 §8.5 POST)
- 任务 8:所有网络数据按"不可信输入"处理:行长度限制、异常捕获、优雅关闭(覆盖 §8.7 安全)

实验环境:JDK 17+;需要联网(任务 6-7);两个终端分别启动服务端与客户端。

第一步 跑通单客户端回显(示例 8-1/8-2),再加在线集合与广播

第二步 播时对"发送者"本人要不要回显,自己决定并在注释里说明理由

第三步 务 6-7 先 curl 试通接口,再写 Java 代码对照

参考答案要点

```

import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.IOException;
import java.io.InputStreamReader;
import java.io.OutputStreamWriter;
import java.net.ServerSocket;
import java.net.Socket;
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.nio.charset.StandardCharsets;
import java.time.Duration;
import java.util.Map;
import java.util.Set;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

/** 命令聊天室 + HTTP 查询 — 第8章综合实验参考答案
 * 覆盖:TCP 多客户端、线程池、广播、超时、行协议、HttpClient、安全红线
 */
public class ChatRoom {

    // 在线集合:并发安全,value 是写给该客户端的流
    private static final Map<String, BufferedWriter> clients =
        new ConcurrentHashMap<>();
    private static final ExecutorService pool = Executors.newFixedThreadPool(10);

    public static void main(String[] args) throws Exception {
        if (args.length > 0 && args[0].equals("server")) {
            startServer(9200);
        } else if (args.length > 0 && args[0].equals("client")) {
            startClient("127.0.0.1", 9200, "游客" + (int) (Math.random() * 1000));
        } else if (args.length > 0 && args[0].equals("http")) {
            httpTasks();
        } else {
            System.out.println("用法: java ChatRoom server | client | http");
        }
    }

    // ---- 服务端 ----
    private static void startServer(int port) throws IOException {
        try (ServerSocket server = new ServerSocket(port)) {
            System.out.println("聊天室监听 " + port);
            while (true) {
                Socket client = server.accept();    // 主循环只 accept
                pool.submit(() -> handleClient(client));
            }
        }
    }

    private static void handleClient(Socket client) {
        String nick = null;
        try (client;
            var in = new BufferedReader(new InputStreamReader(
                client.getInputStream(), StandardCharsets.UTF_8));
            var out = new BufferedWriter(new OutputStreamWriter(
                client.getOutputStream(), StandardCharsets.UTF_8))) {
            // 第一行是昵称(行协议:一条消息 = 一行)
            nick = in.readLine();
            if (nick == null || nick.length() > 20) {
                return;    // 红线:限制输入长度
            }
            clients.put(nick, out);
            broadcast("系统", nick + " 加入聊天室");
            String line;
            while ((line = in.readLine()) != null) {

```

```

        if (line.length() > 200) {
            line = line.substring(0, 200);    // 红线:消息长度上限
        }
        broadcast(nick, line);
    }
} catch (IOException e) {
    System.out.println("连接异常: " + e.getMessage());
} finally {
    if (nick != null) {
        clients.remove(nick);
        broadcast("系统", nick + " 离开");
    }
}
}

// 广播:遍历在线集合,逐个写(写失败就移除该用户)
private static void broadcast(String from, String msg) {
    String packet = from + ": " + msg + "\n";    // 行协议
    for (var entry : clients.entrySet()) {
        try {
            entry.getValue().write(packet);
            entry.getValue().flush();
        } catch (IOException e) {
            clients.remove(entry.getKey());    // 写不动的用户踢出
        }
    }
}

// ---- 客户端 ----
private static void startClient(String host, int port, String nick)
    throws IOException {
    try (Socket sock = new Socket(host, port)) {
        sock.setSoTimeout(5000);    // 读超时 5 秒
        var in = new BufferedReader(new InputStreamReader(
            sock.getInputStream(), StandardCharsets.UTF_8));
        var out = new BufferedWriter(new OutputStreamWriter(
            sock.getOutputStream(), StandardCharsets.UTF_8));
        out.write(nick + "\n");    // 先报到昵称
        out.flush();

        // 读线程:收广播(超时只是提示,不断线)
        Thread reader = new Thread(() -> {
            try {
                String line;
                while ((line = in.readLine()) != null) {
                    System.out.println(line);
                }
            } catch (java.net.SocketTimeoutException e) {
                System.out.println("(5 秒无消息,继续等待)");
            } catch (IOException e) {
                System.out.println("连接已断开");
            }
        });
        reader.setDaemon(true);
        reader.start();

        // 主线程:读键盘发送
        try (var kb = new BufferedReader(
            new InputStreamReader(System.in, StandardCharsets.UTF_8))) {
            String line;
            while ((line = kb.readLine()) != null) {
                out.write(line + "\n");
                out.flush();
            }
        }
    }
}

// ---- HTTP 任务 ----

```

```

private static void httpTasks() throws Exception {
    HttpClient client = HttpClient.newBuilder()
        .connectTimeout(Duration.ofSeconds(5))
        .build();

    // 任务 6:GET 拿本机出口 IP
    HttpResponse<String> ipResp = client.send(
        HttpRequest.newBuilder()
            .uri(URI.create("https://httpbin.org/ip"))
            .timeout(Duration.ofSeconds(10))
            .GET().build(),
        HttpResponse.BodyHandlers.ofString());
    System.out.println("GET /ip 状态: " + ipResp.statusCode());
    // 简单解析:{ "origin": "x.x.x.x" } 取冒号后到引号前的部分
    String body = ipResp.body();
    String origin = body.substring(body.indexOf("origin") + 10,
        body.lastIndexOf(''));
    System.out.println("本机出口 IP: " + origin);

    // 任务 7:POST JSON
    HttpResponse<String> postResp = client.send(
        HttpRequest.newBuilder()
            .uri(URI.create("https://httpbin.org/post"))
            .header("Content-Type", "application/json")
            .POST(HttpRequest.BodyPublishers.ofString(
                "{\"chat\":\"你好\"}"))
            .build(),
        HttpResponse.BodyHandlers.ofString());
    System.out.println("POST 状态: " + postResp.statusCode());
    System.out.println("POST 回显: " + postResp.body()
        .substring(0, Math.min(200, postResp.body().length())));
}
}

```

设计说明:在线集合用 ConcurrentHashMap(第9章的并发容器)——多个线程同时读写在线列表,普通 HashMap 会崩;广播对“写失败”的连接做移除,避免僵尸用户阻塞线程池;客户端读线程用 daemon 线程,主线程退出时不阻塞。任务 6 的 JSON 解析是“手搓”的(用 indexOf),生产必须用 Jackson 等库(第13章),这里故意演示“能读但别写”。扩展方向:私聊指令、历史消息、心跳保活、HTTPS + 令牌认证。

下一章预告:第9章 并发与异常。线程的生命周期、同步与锁、异常体系与最佳实践——网络程序的并发问题,和 Java 的异常世界观,都在这一章。

第9章 并发与异常

建议学时:6-8 小时 | 难度:★★★★★ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 掌握异常体系:Error / Exception / RuntimeException 的分层与意义
- 理解 checked 与 unchecked 异常的区别,掌握声明与处理的规则
- 掌握 try-catch-finally、多 catch、自定义异常的最佳实践
- 掌握线程的创建方式与生命周期
- 掌握 synchronized 与原子类,理解"竞态"与"可见性"
- 掌握 java.util.concurrent:线程池、并发容器
- 理解死锁成因,建立并发安全的工程习惯

💡 从 C 到 Java:错误处理与并发的范式革命

异常是对 errno 的全面升级。 C 的错误处理靠返回值 + errno:每个函数都要检查返回值,忘了检查就静默出错;错误信息靠 perror 打印。Java 用异常——错误发生时"抛出"一个对象,沿调用栈向上找处理者。强制处理的 checked 异常,是编译器逼你把错误处理写进代码;不强制处理的 unchecked 异常,是编程错误(空指针、下标越界)。C 程序员最需要适应的:不是"怎么抛",而是"哪些必须处理、哪些不该处理"。

并发:从 pthread 到语言内建。 C 的线程是 pthread 库的第三方体验:pthread_create/mutex_lock 全是函数;Java 里 Thread / synchronized 是语言与标准库的一等公民。但 并发难题——竞态、可见性、死锁——一模一样,C 里怎么踩坑,Java 里换个马甲继续踩。核心思维不变:共享可变状态是万恶之源,要么锁,要么别共享,要么用原子操作。

线程池是生产标配。 C 程序员裸建线程(像第8章的裸 Thread);生产 Java 用 Executors/ThreadPoolExecutor 管理线程生命周期——池化复用、上限可控、队列缓冲。这是"资源管理"思维在并发领域的体现,和第5章"内存由 GC 管"同源:别亲手持有稀缺资源,交给专业管家。

学习顺序:先异常(\$9.1-9.3,独立主题),再线程基础(\$9.4),同步(\$9.5),并发工具(\$9.6),最后死锁与纪律(\$9.7)。

📦 前置知识自查

- C 的 errno/返回码错误处理;pthread 线程与互斥锁的基本概念
- 第8章:多客户端线程池已"预装"过 Executors 的用法
- 第4章:lambda(线程任务用 Runnable);第6章:对象与封装

🚀 本章学习路线

1. **第一阶段(约 2 小时):**\$9.1-9.3 异常体系,把"返回值检查"思维翻译成"异常处理"思维
2. **第二阶段(约 2 小时):**\$9.4-9.5 线程与同步,亲手制造一次竞态再修好
3. **第三阶段(约 2 小时):**\$9.6-9.7 并发容器/线程池/死锁,完成章末实验
4. **验收标准:**能画出异常体系图并说出 checked/unchecked 规则;能解释竞态与可见性;能写出线程池并发统计

本章知识导图

- §9.1 异常体系:Throwable / Error / Exception
- §9.2 checked 与 unchecked:声明、处理、取舍
- §9.3 try-catch-finally 与自定义异常
- §9.4 线程:创建与生命周期
- §9.5 同步:synchronized / 原子类 / volatile
- §9.6 java.util.concurrent:线程池与并发容器
- §9.7 死锁与并发纪律

§9.1 异常体系总览

9.1.1 Throwable 家族树

类型	代表	含义	C 对应	必须处理?
Error	OutOfMemoryError、StackOverflowError	JVM 级别的严重问题,程序无法恢复	段错误、OOM kill	不处理(处理也没用)
Checked Exception	IOException、SQLException	可预料的业务/IO 失败,调用方应当处理	文件打开失败等"可恢复错误"	必须(声明或捕获)
RuntimeException(Unchecked)	NullPointerException、IndexOutOfBoundsException、IllegalArgumentException	编程错误,修复代码而不是捕获	空指针解引用、越界	不必(不强制)

★ 重点:处理还是不处理,这是个规则问题

Checked 异常:编译器强制你面对。要么方法 `throws IOException` 声明"我会甩锅给调用方",要么 try-catch 就地处理——二选一,漏了编译失败。Unchecked 异常:编译器不管,但你必须知道它不该被"处理"——NullPointerException 的正确修法是"判空",不是 catch 它。生产代码里 catch RuntimeException 几乎都是坏味道(第9.3节)。

§9.2 checked 与 unchecked:规则与取舍

示例 9-1 声明与处理的三条路

```

import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;

public class ExceptionDemo {
    // 路 1:声明 throws,把异常甩给调用方(适合"我不负责恢复"的层)
    static String readFile(String path) throws IOException {
        return Files.readString(Path.of(path), StandardCharsets.UTF_8);
    }

    // 路 2:捕获处理,就地恢复(适合"我能处理"的层)
    static String readOrDefault(String path) {
        try {
            return Files.readString(Path.of(path), StandardCharsets.UTF_8);
        } catch (IOException e) {
            System.err.println("读文件失败,用默认配置: " + e.getMessage());
            return "{}"; // 降级方案
        }
    }

    // 路 3:捕获后包装再抛(换语义,加上下文)
    static String readConfig(String path) throws IllegalStateException {
        try {
            return Files.readString(Path.of(path), StandardCharsets.UTF_8);
        } catch (IOException e) {
            throw new IllegalStateException("配置文件读取失败: " + path, e);
            // 原异常作为 cause 保留:调用栈不丢
        }
    }

    public static void main(String[] args) {
        System.out.println(readOrDefault("missing.txt")); // 降级输出 {}
        // readFile 必须处理:要么 try-catch,要么 main 也 throws
        try {
            System.out.println(readFile("app.properties"));
        } catch (IOException e) {
            System.out.println("main 层兜底: " + e.getMessage());
        }
    }
}

```

⚠ 易错点 1:吞异常与过度声明

两个反面典型: ① `catch (Exception e) {}` 空手吞掉——错误无声消失,生产事故的温床;至少打日志,要么抛出去。 ② `main` 一杆子 `throws Exception`——程序入口的异常只被 JVM 打印堆栈,等于没处理。工程惯例:底层"抛"(throws),中间层"包"(wrap 加上下文),边界层"处理"(恢复或告警退出),入口绝不吞。

§9.3 try-catch-finally 与自定义异常

9.3.1 多 catch 与 finally

多 catch 顺序:子类在前,父类在后(先 `catch FileNotFoundException` 再 `catch IOException`,否则子类永远轮不到)。
`finally` 无论是否异常都执行,常用来清理——但第5章已说,资源清理优先 `try-with-resources`; `finally` 留给人"关闭手动资源、统计、解锁"。JDK7+ 支持多异常一个 catch: `catch (A | B e)`。

示例 9-2 多 catch 与 finally 执行顺序

```

public class FinallyDemo {
    public static void main(String[] args) {
        try {
            System.out.println("try 里");
            if (args.length > 0 && args[0].equals("boom")) {
                throw new IllegalArgumentException("故意出错");
            }
            return; // 提前 return,finally 照样执行!
        } catch (IllegalArgumentException e) {
            System.out.println("catch: " + e.getMessage());
        } catch (RuntimeException e) { // 父类在后
            System.out.println("catch 父类: " + e);
        } finally {
            System.out.println("finally:无论怎样都执行");
        }
        System.out.println("finally 之后:仅无异常/异常已处理才到");
    }
}
// 输出:try 里 → catch → finally;或:try 里 → finally → finally 之后

```

示例 9-3 自定义异常

```

// 业务异常:继承 RuntimeException(调用方不必强制处理,由框架统一捕获)
public class BusinessException extends RuntimeException {
    private final String code; // 业务错误码,给前端/日志用

    public BusinessException(String code, String message) {
        super(message);
        this.code = code;
    }

    public String getCode() { return code; }
}

// 用法示例
class OrderService {
    void pay(long orderId, double amount) {
        if (amount <= 0) {
            throw new BusinessException("INVALID_AMOUNT", "金额必须为正");
        }
        // 业务逻辑...
        throw new BusinessException("BALANCE_NOT_ENOUGH", "余额不足");
    }
}

```



工程建议:异常是 API 的一部分

生产团队会维护"异常码字典":自定义异常携带错误码与可读消息,日志与前端共用。三个纪律:① 自定义异常命名以 `Exception` 结尾;② 携带 `cause` 不丢根因;③ 校验失败用 `IllegalArgumentException/IllegalState`,不要发明新异常。业务错误(余额不足)与系统错误(数据库挂了)分开建模,这是大型系统的基础设施。

§9.4 线程:创建与生命周期

9.4.1 三种创建方式

方式	写法	适用
Thread 子类	<code>class T extends Thread { public void run() {...} }</code>	不推荐(继承占掉名额)
Runnable 接口	<code>new Thread(() -> {...}).start()</code>	推荐:任务与线程解耦
Callable + Future	<code>executor.submit(callable) → future.get()</code>	需要返回值/异常

示例 9-4 线程创建与生命周期

```
public class ThreadDemo {
    public static void main(String[] args) throws Exception {
        // 方式 1:Runnable + lambda,最常用
        Thread t1 = new Thread(() -> {
            for (int i = 0; i < 5; i++) {
                System.out.println("线程1: " + i);
                try { Thread.sleep(50); } // 让出 CPU,睡 50ms
                catch (InterruptedException e) { return; }
            }
        }, "worker-1"); // 线程名:日志排查关键
        t1.start();      // 启动(不是 run!)

        // 方式 2:Callable 拿返回值
        java.util.concurrent.ExecutorService pool =
            java.util.concurrent.Executors.newSingleThreadExecutor();
        java.util.concurrent.Future<Integer> f = pool.submit(() -> {
            int sum = 0;
            for (int i = 1; i <= 100; i++) sum += i;
            return sum; // 有返回值,异常也会带回
        });
        System.out.println("Callable 结果: " + f.get()); // 5050
        pool.shutdown(); // 线程池必须关

        t1.join(); // 等 t1 结束再退出 main
        System.out.println("main 结束");
    }
}
// 生命周期:NEW(start 前)→ RUNNABLE → BLOCKED/WAITING/TIMED_WAITING → TERMINATED
```

⚠ 易错点 2:t.start() 与 t.run()

start() 开新线程,run() 只是当前线程里调个方法。写成 t.run() 的后果:任务在 main 线程串行执行,完全没有并发,且不报错——最隐蔽的并发 bug 之一。另外线程的异常:run() 里抛出的 RuntimeException 不会被调用方捕获(线程边界隔离),生产用 UncaughtExceptionHandler 记录日志。

§9.5 同步:竞态、锁与原子性

9.5.1 竞态演示与修复

C 的 counter++ 在多线程下会丢更新;Java 的 counter++ 一样(读-改-写不是原子的)。修法三选一:synchronized 锁、原子类 AtomicInteger、或者干脆不共享。C 里是 pthread_mutex,Java 里是 synchronized 关键字。

示例 9-5 竞态:坏例子与三个修复

```
import java.util.concurrent.atomic.AtomicInteger;

public class RaceDemo {
    // 坏例子:普通 int,并发 ++ 会丢计数
    static int bad = 0;

    // 修复 1:原子类(无锁,最适合"计数器"这种单一操作)
    static final AtomicInteger good = new AtomicInteger();

    // 修复 2:synchronized 方法(锁住整个方法)
    static synchronized void incBad() { bad++; }

    public static void main(String[] args) throws Exception {
        int N = 10; // 10 个线程
        var pool = java.util.concurrent.Executors.newFixedThreadPool(N);
        for (int t = 0; t < N; t++) {
            pool.submit(() -> {
                for (int i = 0; i < 10000; i++) {
                    bad++; // 竞态:结果 ≤ 100000
                    good.incrementAndGet();// 原子:结果 = 100000
                }
            });
        }
        pool.shutdown();
        while (!pool.awaitTermination(10, java.util.concurrent.TimeUnit.SECONDS)) {
        }
        System.out.println("普通 int(可能有竞态): " + bad);
        System.out.println("AtomicInteger(必定正确): " + good.get());
    }
}
```

★ 重点:可见性 —— 比锁更隐蔽的问题

多线程下,一个线程改了变量,另一个线程 **不一定立刻看得到** ——CPU 缓存与编译器优化导致"副本"存在。synchronized 与 volatile 都保证可见性;普通字段不保证。volatile 适合"一写多读的开关标志"(如停止标志),但不解决"读-改-写"复合操作的原子性——计数还得原子类或锁。生产判断:不确定就用锁或原子类,volatile 是精打细算时的优化。

§9.6 java.util.concurrent:并发工具箱

9.6.1 线程池三件套

工具	用途	要点
Executors.newFixedThreadPool(n)	固定线程池	并发上限可控;任务排队
Executors.newCachedThreadPool()	弹性线程池	空闲 60s 回收;别接无界任务(会建海量线程)
Executors.newSingleThreadExecutor()	单线程串行	任务排队执行,天然串行化
Future.get()	取任务结果	阻塞等待;任务异常在 get 时抛出(ExecutionException)
shutdown()	优雅关闭	不再接受新任务,跑完存量;忘了关 = 进程不退出

9.6.2 并发容器

场景	C 的直觉	Java 正解
多线程读写 Map	HashMap + 手工锁	ConcurrentHashMap(分段锁/无锁,读无锁写有限竞争)
多线程读写 List	数组 + 锁	CopyOnWriteArrayList(读多写少)或 Collections.synchronizedList
生产者消费者队列	队列 + 条件变量	ArrayBlockingQueue / LinkedBlockingQueue(自带阻塞)
计数器	int + 锁	AtomicLong / LongAdder(高并发计数)

示例 9-6 线程池 + 并发容器:并发统计

```
import java.util.List;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.atomic.LongAdder;

public class ConcurrentDemo {
    public static void main(String[] args) throws Exception {
        // 场景:多个线程同时统计"各年级人数"
        List<String> grades = List.of("高一", "高二", "高三", "高一", "高一",
                                     "高三", "高二", "高一");

        ConcurrentHashMap<String, LongAdder> counts = new ConcurrentHashMap<>();
        ExecutorService pool = Executors.newFixedThreadPool(4);

        for (String g : grades) {
            pool.submit(() -> {
                // computeIfAbsent:没有就建 LongAdder,原子地做
                counts.computeIfAbsent(g, k -> new LongAdder()).increment();
            });
        }
        pool.shutdown();
        pool.awaitTermination(5, java.util.concurrent.TimeUnit.SECONDS);

        counts.forEach((k, v) -> System.out.println(k + ": " + v.sum()));
        // 高一: 4, 高二: 2, 高三: 2
    }
}
```

⚠ 易错点 3:HashMap 并发写会"挂"而不是"错"

多线程写普通 HashMap 可能死循环(JDK7)/丢数据(JDK8),而不是抛异常——这比"结果不对"危险得多,因为无声。凡是并发环境出现的共享 Map,默认 ConcurrentHashMap;JDK8 的 ConcurrentHashMap 读完全无锁,性能损失极小,没有理由不用。

§9.7 死锁与并发纪律

9.7.1 死锁四条件与工程对策

死锁四条件:C 里学的(互斥、占有且等待、不可剥夺、循环等待)在 Java 完全适用。工程对策:① 锁的顺序全局一致(都先锁 A 再锁 B);② 尽量用无锁方案(原子类、ConcurrentHashMap);③ 用 tryLock 加超时,拿不到就放弃而不是死等。

示例 9-7 死锁演示与 tryLock 修复

```

import java.util.concurrent.locks.ReentrantLock;

public class DeadlockDemo {
    static final Object A = new Object();
    static final Object B = new Object();

    // 死锁版:两个线程按相反顺序抢锁,互相等对方释放
    static void deadlock() {
        Thread t1 = new Thread(() -> {
            synchronized (A) {
                System.out.println("t1 拿到 A");
                try { Thread.sleep(50); } catch (InterruptedException e) { }
                synchronized (B) { System.out.println("t1 拿到 B"); } // 等 t2 放 B
            }
        });
        Thread t2 = new Thread(() -> {
            synchronized (B) {
                System.out.println("t2 拿到 B");
                try { Thread.sleep(50); } catch (InterruptedException e) { }
                synchronized (A) { System.out.println("t2 拿到 A"); } // 等 t1 放 A
            }
        });
        t1.start();
        t2.start();
        // 两个线程永远卡在"互相等":程序僵死,无输出"拿到第二个锁"
    }

    // 修复版:tryLock 带超时,拿不到就放弃重试,不死等
    static void fixed() {
        var lockA = new ReentrantLock();
        var lockB = new ReentrantLock();
        boolean ok = lockA.tryLock() && lockB.tryLock(); // 非阻塞尝试
        if (ok) {
            try {
                System.out.println("两个锁都拿到,干活");
            } finally {
                lockB.unlock();
                lockA.unlock(); // 释放顺序与加锁相反
            }
        } else {
            System.out.println("拿不到锁,放弃(或稍后重试)"); // 锁未全拿时别部分释放
        }
    }
}

```



工程建议:并发纪律十条(节选)

- 能不改共享状态就不改:局部变量优先,不可变对象优先 (第14章的 record 是利器)
- 共享可变状态:要么锁,要么原子类,要么并发容器;三选一,不许裸奔
- 锁的粒度:锁短代码块而不是整个方法;但别把原子操作拆碎(拆碎 = 新竞态)
- 线程名、日志、异常 cause 三件套齐全,否则线上问题无法定位
- 线程池一律 shutdown;写线程的代码先想"它什么时候退出"

本章速查表

主题	要点
异常体系	Throwable → Error(不处理)/ Exception(处理)/ RuntimeException(修代码)
checked 规则	要么 throws 声明,要么 try-catch;漏了编译失败;IOException 是典型
unchecked	NPE/越界/IllegalArgument;正确修法:判空/校验,不是 catch
多 catch	子类在前父类在后;JDK7+ 可 catch(A B e)
finally	永远执行(return 也执行);资源清理优先 try-with-resources
自定义异常	继承 RuntimeException + 错误码;命名 Exception 结尾;保留 cause
线程创建	Runnable + lambda 最常用;Callable 拿返回值;start() 不是 run()
生命周期	NEW → RUNNABLE → BLOCKED/WAITING/TIMED_WAITING → TERMINATED
竞态	++ 不是原子;修复:原子类 / synchronized / 不共享
可见性	volatile 保证可见不保证原子;不确定就用锁或原子类
线程池	Executors.newFixedThreadPool(n);任务 Runnable/Callable;必须 shutdown
并发容器	ConcurrentHashMap / CopyOnWriteArrayList / BlockingQueue / LongAdder
死锁	锁顺序一致 / tryLock 超时 / 无锁优先;四条件复现诊断

最小必会清单

- 能画出异常体系树并解释三种类型的处理策略
- 能说出 checked 异常的两条合法出路,能举出三种典型
- 能写出多 catch + finally,并说出 finally 在 return 时仍执行
- 能用 lambda 创建线程并说出 start/run 的区别
- 能解释竞态与可见性,写出 AtomicInteger 计数
- 能说出三种线程池的适用场景与 shutdown 的必要性
- 能解释死锁四条件并写出 tryLock 修复

自测题

Q 1. 下面的异常各属哪类,该怎么处理?① 文件不存在;② 数组越界;③ 内存耗尽;④ 余额不足(业务)

✅ 参考答案

① IOException,checked:声明或处理(降级/提示)。② IndexOutOfBoundsException,unchecked:修代码(检查下标),不是 catch。③ OutOfMemoryError,Error:不捕获,优化内存使用(第5章)。④ 自定义业务异常(继承 RuntimeException):由框架统一捕获返回错误码,业务层不散落 try-catch。

Q 2. 为什么说"catch (Exception e) { }"是坏味道?至少写出两种正确做法。

✅ 参考答案

空 catch 吞掉错误:程序继续跑,但状态可能已坏,问题被掩盖到无法排查。正确做法:① 至少打日志(log.error("...", e));② 抛回或包装(throw new BusinessException(...));③ 能恢复就显式降级(返回默认值并记录)。原则:异常要么处理,要么传递,不许消失。

Q 3. 两个线程对同一个 int 做 10 万次 ++,结果可能小于 20 万。为什么?给出两种修复。

✓ 参考答案

++ 是"读-加-写"三步,两个线程可能同时读到同一个值,各自 +1 再写回,一次更新被覆盖(丢失更新)。修复:AtomicInteger.incrementAndGet()(无锁 CAS);或 synchronized 方法/块(锁)。注意 volatile 不能修复:它管可见性,不管复合操作原子性。

Q 4. Future.get() 会抛哪些异常?任务里抛的 RuntimeException 怎么拿到?

✓ 参考答案

InterruptedException(被中断)与 ExecutionException(任务抛出的异常被包装)。任务内的 RuntimeException 会作为 ExecutionException 的 cause 抛出,用 e.getCause() 拿到原始异常。所以"线程里的异常无人知"只发生在裸 Thread 的 run() 里;用线程池 + Callable 异常能被捕获。

Q 5. 生产环境 JVM 进程不退出,排查第一步看什么?为什么?

✓ 参考答案

jstack 打印线程栈,找"卡住的线程":最常见的是非 daemon 线程池没 shutdown(线程池线程不会自己退出),或死锁(栈里能看到两个线程互相持有对方等待的锁)。C 的 gdb attach 是同类操作。看完线程再看资源:jcmd Thread.print 同样有效。

Q 6. ConcurrentHashMap 的 computeIfAbsent 是干什么的?为什么比"先 get 再 put"安全?

✓ 参考答案

computeIfAbsent(key, fn):键不存在时原子地执行 fn 生成值并放入,存在则返回已有值。"先 get,不存在再 put"两步在并发下会产生竞态:两个线程同时发现不存在,各自 put,后写的覆盖先写的(丢失更新)。computeIfAbsent 把"检查-创建-放入"合成一个原子操作。

章末实验题:并发成绩统计系统

实验 9:并发成绩统计系统

实验目标:编写一个多线程成绩统计系统,综合运用本章全部知识点:异常体系、checked 处理、自定义异常、线程、竞态修复、原子类、线程池、并发容器。

实验要求:

- 任务 1:定义 GradeFileException extends RuntimeException 携带错误码(覆盖 §9.1-9.3 异常)
- 任务 2:读取 scores.csv(可缺失),文件不存在时抛 GradeFileException("FILE_NOT_FOUND"),main 捕获后打印友好提示(覆盖 §9.2-9.3 checked 处理)
- 任务 3:每条记录解析时校验格式,非法行跳过并计数(覆盖 §9.3 多 catch 与防御)
- 任务 4:用 4 线程线程池并行统计"每个分数段的人数"(0-59/60-79/80-100),每个线程处理一批数据(覆盖 §9.4-9.5 线程与竞态)
- 任务 5:统计计数器用 LongAdder 与 ConcurrentHashMap,保证并发正确(覆盖 §9.5-9.6 原子与并发容器)
- 任务 6:定义 volatile 停止标志,主线程统计完成后让工作线程优雅退出(覆盖 §9.5 可见性)
- 任务 7:main 汇总输出:总人数、各分数段人数、非法行数、耗时(覆盖全部)

实验环境:JDK 17+;scores.csv 至少 200 行数据(可手工生成,或运行任务 8:内置生成器)。

第一步 写串行版(读文件、解析、统计),验证正确,再加线程

第二步 务 4 的分批:把总行数均分成 4 段,每线程处理一段;注意"下标边界"

第三步 务 6 的停止标志:工作线程循环里检查 flag,主线程置位后 join

✅ 参考答案要点

```

import java.io.BufferedReader;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.atomic.LongAdder;
import java.util.concurrent.atomic.AtomicInteger;

/** 并发成绩统计系统 — 第9章综合实验参考答案
 * 覆盖:异常体系、自定义异常、多 catch、线程池、原子类、并发容器、volatile
 */
public class ConcurrentScoreStats {

    // 任务 1:自定义业务异常,携带错误码
    static class GradeFileException extends RuntimeException {
        private final String code;

        GradeFileException(String code, String message) {
            super(message);
            this.code = code;
        }

        String getCode() { return code; }
    }

    public static void main(String[] args) {
        try {
            List<Integer> scores = loadScores(Path.of("scores.csv"));
            report(scores);
        } catch (GradeFileException e) {
            System.out.println("业务错误[" + e.getCode() + "]: " + e.getMessage());
        } catch (Exception e) { // 兜底:防止线程边界把异常吞掉
            System.out.println("系统错误: " + e);
        }
    }

    // 任务 2 + 3:读取与校验;文件缺失抛业务异常
    private static List<Integer> loadScores(Path path) {
        List<Integer> scores = new ArrayList<>();
        if (!Files.exists(path)) {
            throw new GradeFileException("FILE_NOT_FOUND", "找不到 " + path);
        }
        try (var reader = Files.newBufferedReader(path, StandardCharsets.UTF_8)) {
            String line;
            int lineNo = 0;
            while ((line = reader.readLine()) != null) {
                lineNo++;
                try {
                    int s = Integer.parseInt(line.trim());
                    if (s < 0 || s > 100) {
                        System.out.println("第 " + lineNo + " 行分数越界,跳过: " + line);
                        continue;
                    }
                    scores.add(s);
                } catch (NumberFormatException e) {
                    System.out.println("第 " + lineNo + " 行格式非法,跳过: " + line);
                }
            }
        } catch (java.io.IOException e) { // checked:包装成业务异常向上抛
            throw new GradeFileException("IO_ERROR", "读取失败: " + e.getMessage());
        }
        return scores;
    }
}

```



```

// 任务 4-7:并发统计
private static void report(List<Integer> scores) throws Exception {
    // 分数段表:并发安全的"段名 → 计数器"
    ConcurrentHashMap<String, LongAdder> buckets = new ConcurrentHashMap<>();
    // 任务 6:volatile 停止标志(一写多读)
    // 用 AtomicBoolean 替代,效果等同且类型安全
    AtomicInteger working = new AtomicInteger(scores.size());

    int threads = 4;
    ExecutorService pool = Executors.newFixedThreadPool(threads);
    int chunk = (scores.size() + threads - 1) / threads;

    long start = System.currentTimeMillis();
    for (int t = 0; t < threads; t++) {
        final int from = t * chunk;
        final int to = Math.min(from + chunk, scores.size());
        pool.submit(() -> {
            for (int i = from; i < to; i++) {
                int s = scores.get(i);
                String bucket;
                if (s < 60) bucket = "0-59";
                else if (s < 80) bucket = "60-79";
                else bucket = "80-100";
                // computeIfAbsent:并发安全地取/建 LongAdder
                buckets.computeIfAbsent(bucket, k -> new LongAdder()).increment();
                working.decrementAndGet(); // 进度计数(原子)
            }
        });
    }
    pool.shutdown(); // 优雅关闭:不再接新任务
    boolean done = pool.awaitTermination(30, TimeUnit.SECONDS);
    if (!done) {
        System.out.println("警告:统计超时未完成");
    }
    long cost = System.currentTimeMillis() - start;

    // 任务 7:汇总报告
    System.out.println("== 并发成绩统计报告 ==");
    System.out.println("总人数: " + scores.size());
    buckets.forEach((k, v) -> System.out.println("  " + k + " 分: "
        + v.sum() + " 人"));
    System.out.println("处理完成标记: " + (working.get() == 0));
    System.out.println("耗时: " + cost + " ms(4 线程)");
}
}

```

设计说明:任务 2 故意让 loadScores 抛"业务异常"而非裸 IOException——数据文件缺失是业务事件,应由调用方决定文案;任务 5 用 computeIfAbsent 保证"建桶 + 计数"原子,这正是自测题 6 的实战版;任务 6 用 AtomicInteger 当"剩余工作量"计数器,既验证了所有任务都执行完,也演示了原子类的进度追踪(volatile 只能做布尔开关,计数必须原子)。扩展方向:改成 Callable 让每个线程返回自己的统计,再合并(Future.get 的错误会暴露在主线);加大数据量(100 万行)对比串行与并行的耗时。

下一章预告:第10章 泛型与集合框架。ArrayList 与 LinkedList、HashMap 原理、泛型与类型擦除、PECS 原则——集合是并发之后的下一块基石。

第10章 泛型与集合框架

建议学时:6-8 小时 | 难度:★★★★☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 掌握集合框架的体系:Collection(List/Set/Queue)与 Map
- 理解泛型与类型擦除,与 C++ 模板的异同
- 掌握通配符与 PECS 原则,能读懂框架源码里的复杂泛型
- 掌握 ArrayList 与 LinkedList 的原理与选型
- 理解 HashMap 的原理:哈希、冲突、扩容,以及它为什么依赖 equals/hashCode
- 掌握 Comparable 与 Comparator 两种排序方式
- 掌握迭代与遍历删除的正确姿势(fail-fast)

💡 从 C 到 Java:集合 = 造好的轮子

C 程序员每次需要"动态数组"都要自己造:malloc + realloc + 拷贝,手写链表还要维护头指针。Java 的集合框架把这些轮子全部造好:ArrayList(动态数组)、LinkedList(链表)、HashMap(哈希表)、HashSet、TreeMap(红黑树)——对应 C 里"轮子厂"级别的工程。java.util.Arrays.asList、List.of 甚至直接给你"声明即集合"。

泛型是 Java 对 C++ 模板的"打折版"。C++ 模板在编译期为每种类型生成代码(模板实例化);Java 泛型在编译期检查类型、运行期擦除类型(**类型擦除**): ArrayList<String> 和 ArrayList<Integer> 在运行期是同一个类 ArrayList。好处:不用复制代码;代价:不能 new T()、不能 instanceof T、List<String> 与 List<Object> 不是父子关系(协变问题,引出 PECS)。

HashMap 是第6章的"回扣"。哈希表的桶定位靠 hashCode,桶内比较靠 equals——第6章让你把这两个方法写对,就是为了这一刻。HashMap 的原理(数组+链表/红黑树、负载因子、扩容)是面试常客,也是理解"为什么键类要不可变"的根源。

排序两条路。C 的 qsort 需要一个比较函数指针;Java 是 Comparable(类自己说"我怎么排序")与 Comparator(外部说"这次怎么排序")——后者正是 lambda 的最佳舞台(第4章),comparingInt 等方法引用把排序写成一行。

学习顺序:先认识集合家族(§10.1),泛型与擦除(§10.2-10.3),常用容器原理(§10.4-10.5),排序与遍历(§10.6-10.7)。

📦 前置知识自查

- C 的动态数组/链表手写经验;qsort 的比较函数指针
- 第6章:equals/hashCode 契约(本章的 HashMap 直接依赖)
- 第4章:lambda 与方法引用(Comparator 的现代写法)
- 第1章:数组;第9章:并发容器与普通容器的区别

🚀 本章学习路线

1. **第一阶段(约 2 小时):**§10.1-10.3 集合家族与泛型擦除,理解"擦除"的三个不能
2. **第二阶段(约 2 小时):**§10.4-10.5 容器原理,重点 HashMap 的"数组+链表"心智模型
3. **第三阶段(约 2 小时):**§10.6-10.7 排序与遍历,完成章末实验
4. **验收标准:**能说出擦除的三个限制;能画出 HashMap 结构;能写 PECS 解释;能完成单词统计

本章知识导图

- §10.1 集合框架总览:Collection / List / Set / Map
- §10.2 泛型基础:泛型类 / 泛型方法 / 类型擦除
- §10.3 通配符与 PECS:extends / super
- §10.4 ArrayList vs LinkedList
- §10.5 HashMap 原理:哈希 / 冲突 / 扩容
- §10.6 排序:Comparable / Comparator
- §10.7 迭代与 fail-fast

§10.1 集合框架总览

10.1.1 家族树与 C 对照

接口	特性	实现(常用)	C 的对应物
List	有序、可重复、按下标访问	ArrayList、LinkedList	动态数组、链表
Set	无序(按实现)、不可重复	HashSet、TreeSet	手写哈希集合
Queue/Deque	队列语义(先进先出)	ArrayDeque、LinkedList	环形队列、链表队列
Map	键值对、键不可重复	HashMap、TreeMap	哈希表、二叉搜索树

示例 10-1 集合基本操作

```

import java.util.ArrayList;
import java.util.HashMap;
import java.util.HashSet;
import java.util.List;
import java.util.Map;
import java.util.Set;

public class CollectionBasics {
    public static void main(String[] args) {
        // List:有序可重复
        List<String> names = new ArrayList<>();
        names.add("张三");
        names.add("李四");
        names.add("张三"); // 允许重复
        System.out.println(names.get(1)); // 李四(按下标)
        System.out.println(names); // [张三, 李四, 张三]

        // Set:去重
        Set<String> uniq = new HashSet<>(names);
        System.out.println(uniq); // [李四, 张三](顺序不保证)

        // Map:键值对
        Map<String, Integer> scores = new HashMap<>();
        scores.put("张三", 88);
        scores.put("李四", 76);
        System.out.println(scores.get("张三")); // 88
        System.out.println(scores.getOrDefault("王五", -1)); // -1:取不到给默认

        // 不可变集合:JDK9+,声明即定,不可增删
        List<Integer> fixed = List.of(1, 2, 3);
        // fixed.add(4); // 抛 UnsupportedOperationException
        System.out.println(fixed);
    }
}

```

⚠ 易错点 1:集合存的是引用,不是副本

把对象放进集合,放的是引用——之后修改对象字段,集合里的"内容"跟着变。C 里往数组塞结构体是复制,Java 是"登记地址"。用 final/不可变对象当集合元素,能避免一大批"集合内容莫名变化"的 bug(第14章 record 的用武之地)。

§10.2 泛型基础与类型擦除

10.2.1 泛型类与泛型方法

示例 10-2 泛型类与泛型方法

```

import java.util.List;

public class GenericDemo {
    // 泛型类:T 是类型参数,实例化时指定
    static class Box<T> {
        private T value;

        Box(T value) { this.value = value; }

        T get() { return value; }

        void set(T value) { this.value = value; }
    }

    // 泛型方法:类型参数在方法上,独立于类
    static <T> T firstOr(List<T> list, T def) {
        return list.isEmpty() ? def : list.get(0);
    }

    // 类型参数的数量不限
    static class Pair<K, V> {
        final K key;
        final V value;

        Pair(K key, V value) { this.key = key; this.value = value; }
    }

    public static void main(String[] args) {
        Box<String> sb = new Box<>("hi"); // 菱形 <>:类型推断
        Box<Integer> ib = new Box<>(42);
        System.out.println(sb.get() + " " + ib.get());

        System.out.println(firstOr(List.of(1, 2), -1)); // 1
        System.out.println(firstOr(List.of(), "默认")); // 默认

        Pair<String, Integer> p = new Pair<>("张三", 88);
        System.out.println(p.key + ":" + p.value);
    }
}

```

10.2.2 与 C++ 模板对照:类型擦除的代价

对比项	C++ 模板	Java 泛型
实现时机	编译期:每种类型实例化一份代码	编译期检查、运行期擦除:只有一份代码
<code>new T()</code>	允许	不允许(擦除后没有 T 的类型信息)
<code>instanceof T</code>	允许	不允许(同上)
<code>List<String></code> 与 <code>List<Integer></code>	不同类型	同一个类(擦除后都是 List)
编译期检查	实例化时检查	使用处检查(编译期)
基本类型	int 直接用	不支持,用 Integer(自动装箱)

★ 重点:擦除带来的三个"不能"与一个"要"

① 不能 `new T()`;② 不能 `instanceof T`;③ 不能建泛型数组 `new T[]` (要建就用 `List<T>`)。一个"要":运行时拿类型信息,泛型类要传 `Class<T>` 参数(工厂模式: `create(Class<T> c) { c.getDeclaredConstructor().newInstance(); }`)。理解擦除,你就明白为什么 `List<String>` 不能直接转 `List<Object>` ——下一节的 PECS 就是为了对付这种"类型关系不直观"。

§10.3 通配符与 PECS

10.3.1 为什么需要 ? extends 与 ? super

因为擦除与不变性: `List<String>` 不是 `List<Object>` 的子类型,把 `List<String>` 传给"接收 `List<Object>`"的方法会编译失败。通配符解决"兼容多种具体类型"的需求: `? extends T` 表示"`T` 或 `T` 的子类",`? super T` 表示"`T` 或 `T` 的父类"。

示例 10-3 通配符与 PECS

```
import java.util.ArrayList;
import java.util.List;

public class WildcardDemo {
    // ? extends Number:可接收 List<Integer>/List<Double> 等
    static double sumOf(List<? extends Number> nums) {
        double s = 0;
        for (Number n : nums) {
            s += n.doubleValue();    // 读:安全,都是 Number
        }
        return s;
    }

    // ? super Integer:可接收 List<Integer>/List<Number>/List<Object>
    static void fillOnes(List<? super Integer> list, int count) {
        for (int i = 0; i < count; i++) {
            list.add(Integer.valueOf(i));    // 写:安全,Integer 是 Integer 的父类们的子类
        }
    }

    public static void main(String[] args) {
        System.out.println(sumOf(List.of(1, 2, 3)));    // 6.0
        System.out.println(sumOf(List.of(1.5, 2.5)));    // 4.0

        List<Number> nums = new ArrayList<>();
        fillOnes(nums, 3);
        System.out.println(nums);    // [0, 1, 2]
    }
}
```

★ 重点:PECS 口诀:Producer Extends, Consumer Super

方法只"产出"数据给调用方(读)→ 参数用 `? extends`;只"消费"数据(写/添加)→ 参数用 `? super`。读出写进:`extends` 的列表只能读(`add` 编译错误),`super` 的列表只能写(`get` 出来是 `Object`)。口诀来源: `Collections.copy(dest, src)` 的签名 `copy(List<? super T> dest, List<? extends T> src)` ——`dest` 被写所以 `super`,`src` 被读所以 `extends`。看到框架源码里这两个通配符,用口诀秒懂。

⚠ 易错点 2:泛型数组与 unchecked 警告

`List<String>[] arr = new List<String>[10]` 编译错误(不能建泛型数组); `new List[10]` 会给出 `unchecked` 警告。正确姿势:数组场景换 `List<List<String>>`。同理,老代码里裸类型(`List` 不加尖括号)会警告——生产代码把警告当错误对待,一律补齐类型参数。

§10.4 ArrayList vs LinkedList

操作	ArrayList(动态数组)	LinkedList(双向链表)
按下标访问 get(i)	O(1):直接算地址	O(n):从头/尾走一半
尾部添加 add(x)	O(1) 摊还(满了扩容,拷贝 1.5 倍)	O(1)
中间插入/删除	O(n):后续元素搬移	O(n) 找位置 + O(1) 改指针(定位仍是 O(n))
内存	连续数组,省(每个元素无额外指针)	每个节点 2 个指针 + 对象头,约贵 3-5 倍
C 对应	malloc + realloc 的动态数组	struct Node { data; next; prev; }

⚠ 易错点 3:LinkedList 的"头部操作"神话

面试常问"大量头插用 LinkedList 更快"——对,但只有 Queue/Deque 语义(头尾操作)时成立;大量"按下标访问"的场景 LinkedList 比 ArrayList 慢两个数量级。生产默认 List 就是 ArrayList;需要队列用 ArrayDeque;LinkedList 的适用面其实很窄。别被"链表万能"的刻板印象带偏。

§10.5 HashMap 原理

10.5.1 数组 + 链表(红黑树)的心智模型

HashMap 的存储:一个数组(桶),每个桶是一条链表(冲突时挂链),JDK8+ 链表超 8 个转红黑树。put 流程:算 hashCode → 与桶数取模定位 → 桶内 equals 逐个比,有则覆盖,无则插入。get 流程:同样定位后 equals 找目标。为什么键类要正确 equals/hashCode:定位靠 hash,比较靠 equals,哪个错了都找不到。

- **负载因子 0.75 与扩容**:元素数 > 桶数 × 0.75 时扩容为 2 倍并重哈希(全部重排,O(n) 的一次性代价)
- **默认容量 16**:预估数据量时用 `new HashMap<>(expectedSize)` 提前定容,减少扩容拷贝
- **允许 null**:HashMap 的键和值都允许 null;HashTable 不允许(老类,别用)
- **不保证顺序**:要有序用 LinkedHashMap(插入序)或 TreeMap(键序)

示例 10-4 HashMap 的键必须正确:反例演示

```

import java.util.HashMap;
import java.util.Map;

public class HashMapKeyDemo {
    // 错误键类:没重写 equals/hashCode(用 Object 的"引用相等")
    static class BadKey {
        final String id;
        BadKey(String id) { this.id = id; }
        // 忘了 equals/hashCode!
    }

    // 正确键类:第6章模板
    static class GoodKey {
        final String id;
        GoodKey(String id) { this.id = id; }

        @Override
        public boolean equals(Object o) {
            if (this == o) return true;
            if (!(o instanceof GoodKey)) return false;
            return id.equals(((GoodKey) o).id);
        }

        @Override
        public int hashCode() { return id.hashCode(); }
    }

    public static void main(String[] args) {
        Map<BadKey, String> badMap = new HashMap<>();
        badMap.put(new BadKey("s1"), "数据A");
        System.out.println("错误键类取回: "
            + badMap.get(new BadKey("s1"))); // null!找不着

        Map<GoodKey, String> goodMap = new HashMap<>();
        goodMap.put(new GoodKey("s1"), "数据B");
        System.out.println("正确键类取回: "
            + goodMap.get(new GoodKey("s1"))); // 数据B
    }
}

```

★ 重点:TreeMap 与排序语义

需要"按键有序遍历"时用 TreeMap(红黑树,按键的自然序或 Comparator)。它完全不需要 equals/hashCode,只靠 Comparable/Comparator 的 compareTo。选型:查得快 → HashMap;遍历要有序 → TreeMap;插入序保持 → LinkedHashMap。三者的 get 复杂度:HashMap O(1)、TreeMap O(log n)、LinkedHashMap O(1)。

§10.6 排序:Comparable 与 Comparator

10.6.1 两条路的区别

方式	语义	写法	适用
Comparable	类自己定义"自然顺序"	class X implements Comparable<X> { compareTo }	该类只有一个天然顺序(如学号)
Comparator	外部定义"本次顺序"	Comparator.comparingInt(...).reversed()	多种排序需求; 不侵入类定义

示例 10-5 两种排序实战


```

import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;

public class SortDemo {
    // Comparable:学生自带自然顺序(按学号)
    static class Student implements Comparable<Student> {
        final String id;
        final String name;
        final int score;

        Student(String id, String name, int score) {
            this.id = id;
            this.name = name;
            this.score = score;
        }

        @Override
        public int compareTo(Student o) {
            return id.compareTo(o.id);          // 负数=我小,0=相等,正数=我大
        }

        @Override
        public String toString() { return id + ":" + name + "(" + score + ")"; }
    }

    public static void main(String[] args) {
        List<Student> list = new ArrayList<>(List.of(
            new Student("2024002", "李四", 76),
            new Student("2024001", "张三", 88),
            new Student("2024003", "王五", 93)));

        list.sort(null);                          // 用自然顺序(compareTo):按学号
        System.out.println("自然序: " + list);

        list.sort(Comparator.comparingInt(s -> s.score).reversed()); // 成绩降序
        System.out.println("成绩降序: " + list);

        list.sort(Comparator.comparing(s -> s.name));          // 姓名
        System.out.println("姓名序: " + list);

        // 链式:先成绩降序,同分按学号
        list.sort(Comparator.comparingInt((Student s) -> s.score)
            .reversed()
            .thenComparing(s -> s.id));
        System.out.println("多级排序: " + list);
    }
}

```



工程建议:排序写法规范

用 `Comparator.comparingInt/comparing`,不要手写 `(a, b) -> a.x - b.x`:减法在溢出时给出错误排序,double 的 NaN 更是灾难。需要自然顺序但类没实现 `Comparable` 时,写一个静态 `Comparator` 常量复用:`public static final Comparator<Student> BY_NAME = Comparator.comparing(s -> s.name);`。

§10.7 迭代与 fail-fast

10.7.1 遍历删除的正确姿势

用 `foreach`(或 `Iterator`)遍历集合时 直接调用 `list.remove(x)` 会抛 `ConcurrentModificationException(fail-fast)`——迭代器记账与集合实际修改次数不一致就报警。三种正确姿势:

示例 10-6 遍历删除三姿势

```

import java.util.ArrayList;
import java.util.List;
import java.util.ListIterator;

public class RemoveDemo {
    public static void main(String[] args) {
        // 需求:删除所有小于 60 的成绩
        List<Integer> scores = new ArrayList<>(List.of(45, 88, 62, 91, 55, 73));

        // 姿势 1:Iterator 的 remove(经典)
        var it = scores.iterator();
        while (it.hasNext()) {
            if (it.next() < 60) {
                it.remove();          // 迭代器的 remove 是安全的
            }
        }
        System.out.println("姿势1: " + scores);    // [88, 62, 91, 73]

        // 姿势 2:ListIterator 遍历中改(带下标)
        scores.addAll(List.of(45, 55));
        ListIterator<Integer> li = scores.listIterator();
        while (li.hasNext()) {
            if (li.next() < 60) {
                li.remove();
            }
        }
        System.out.println("姿势2: " + scores);    // [88, 62, 91, 73]

        // 姿势 3:Stream 过滤重建(函数式,推荐)
        scores.addAll(List.of(45, 55));
        scores = scores.stream().filter(s -> s >= 60).toList();
        System.out.println("姿势3: " + scores);    // [88, 62, 91, 73]
    }
}

```

⚠ 易错点 4:foreach 里 remove 的经典崩溃

`for (Integer s : scores) { if (s < 60) scores.remove(s); }` 运行时抛 `ConcurrentModificationException`; 更隐蔽的是“删最后一个元素恰好不报错”——行为依赖数据,灾难。记忆:foreach 底层就是 `Iterator`,修改必须通过 `Iterator.remove`;多线程修改同一集合也一样抛异常(这正是 fail-fast 的设计目的:尽早暴露,别静默错下去)。

本章速查表

主题	要点
家族树	List(有序可重)/ Set(去重)/ Queue(FIFO)/ Map(键值);实现:ArrayList/HashSet/ArrayDeque/HashMap
泛型	编译期检查、运行期擦除;Box<T> 与 static <T> 方法;菱形 <> 推断
擦除三不能	不能 new T() / instanceof T / 泛型数组;类型信息用 Class<T> 传参
通配符	? extends T(读)/ ? super T(写);List<String> 不是 List<Object>
PECS	Producer Extends, Consumer Super;copy(dest super, src extends)
ArrayList	动态数组:get O(1)、尾插摊还 O(1)、中间插删 O(n);默认选择
LinkedList	双向链表:头尾 O(1);按下标访问 O(n);适用面窄
HashMap	数组+链表(树);put/get 先 hash 定位后 equals 比较;负载因子 0.75 扩容 2 倍
键类铁律	键必须正确 equals/hashCode 且不可变;预估容量 new HashMap<>(n)
TreeMap	红黑树,按键序;只依赖 Comparable/Comparator,不依赖 hashCode
排序	Comparable(自然序)/ Comparator(外部序);用 comparingInt 不手写减法
遍历删除	Iterator.remove / ListIterator / Stream 过滤重建;foreach 里 remove 抛 ConcurrentModificationException
不可变集合	List.of / Set.of / Map.of;增删抛 UnsupportedOperationException

最小必会清单

- 能画出集合框架树,说出四种接口的语义与常用实现
- 能说出类型擦除与 C++ 模板的三个区别、三个"不能"
- 能写出 PECS 的例子并解释 copy 的签名
- 能说出 ArrayList 与 LinkedList 各操作复杂度,并说出默认选择
- 能画出 HashMap 的"数组+链表"结构,解释 put/get 流程
- 能解释为什么键类必须正确 equals/hashCode 且不可变
- 能写出三种遍历删除姿势,解释 fail-fast

自测题

Q 1. 为什么 `List<String>` 不能传给参数为 `List<Object>` 的方法?那传什么可以?

✅ 参考答案

若允许,方法里可以 add 一个 Integer 到该列表,破坏类型安全——所以泛型是不变的(不是协变)。接收方想兼容多种具体类型,参数应声明为 `List<? extends Object>` (或 `List<?>`)——只读兼容;如果要写,声明 `List<? super String>`。

Q 2. 泛型方法 `static <T> T max(List<T> list)` 为什么编译不过?怎么改?

✅ 参考答案

T 是任意类型,编译器无法保证有比较能力。改为 `static <T extends Comparable<? super T>> T max(List<T> list)`:约束 T 必须实现了 Comparable(其父类实现也可,所以是 ? super T)。这个签名正是 Collections.max 的原型——PECS 与泛型约束的综合应用。

Q 3. 往 HashMap 放 100 万个元素,性能明显下降。可能原因?怎么处理?

✓ 参考答案

① 键的 hashCode 分布差(如都取模到同一桶),链表变长——查键类的 hashCode 实现;② 扩容次数多——预估容量 `new HashMap<>(1_000_000 / 0.75)` 或更大,减少 rehash;③ 键类 equals 昂贵——检查实现。诊断用 jvisualvm 看对象分布,先查键类再调容量。

Q 4. 遍历 List 时想删除"倒数第 2 个元素",用哪种方式最稳?

✓ 参考答案

ListIterator 最稳:定位到最后,previous 一步,再 remove——迭代器的 remove 不会触发 fail-fast,而且能精确控制位置。注意 ListIterator 的 remove 删除的是"最近一次 next/previous 返回的元素",调用前必须先走一步。

Q 5. 自定义类放进 TreeSet,却不实现 Comparable,会发生什么?

✓ 参考答案

TreeSet 构造时若未传 Comparator,元素必须实现 Comparable,否则运行期抛 ClassCastException(不能转 Comparable)。对比:HashSet 只要求 equals/hashCode。所以"放 TreeSet 的类实现 Comparable,放 HashSet 的类实现 equals/hashCode"——两个体系各自为政。

Q 6. 写出代码:从单词列表统计词频,按出现次数降序输出前 3 个。

✓ 参考答案

```
List<String> words = List.of("a", "b", "a", "c", "a", "b", "d", "e", "f");
Map<String, Long> freq = new HashMap<>();
for (String w : words) {
    freq.merge(w, 1L, Long::sum);    // 没有则 1,有则加 1(方法引用)
}
freq.entrySet().stream()
    .sorted(Map.Entry.<String, Long>comparingByValue().reversed())
    .limit(3)
    .forEach(e -> System.out.println(e.getKey() + ": " + e.getValue()));
// a: 3,b: 2,c: 1(按次数降序前 3)
```

要点:merge 是"统计词频"的标准 API(比 get/put 三步安全);排序用 comparingByValue;Map.Entry.<String, Long> 是显式类型见证,让链式比较器编译通过。

章末实验题:单词统计系统

实验 10:单词统计系统

实验目标:编写一个文本单词统计系统,综合运用本章全部知识点:集合家族、泛型方法与泛型类、PECS、ArrayList/LinkedList 选型、HashMap 词频、Comparable/Comparator 排序、迭代与 fail-fast。

实验要求:

- 任务 1:读入文本文件(UTF-8),按空白与非字母切分单词(覆盖第7章 IO 与本章集合)
- 任务 2:用 `HashMap<String, Integer>` 统计词频,用 `merge` 或 `get/put`(覆盖 §10.5 HashMap)
- 任务 3:定义泛型方法 `<K, V extends Comparable<? super V>> List<Map.Entry<K, V>> sortByValue(Map<K, V> map, boolean desc)`,供词频排序复用(覆盖 §10.2-10.3 泛型与约束)
- 任务 4:定义一个词条类 `WordCount` implements `Comparable`(按次数降序、同次按字母),演示自然顺序(覆盖 §10.6 Comparable)
- 任务 5:用 `Comparator` 链式排序实现"次数降序 + 字母升序",与任务 4 对比(覆盖 §10.6 Comparator)
- 任务 6:遍历删除"出现次数 ≤ 1 "的词条,分别用 `Iterator.remove` 与 `Stream` 过滤两种方式,并各写一份(覆盖 §10.7 遍历删除)
- 任务 7:输出统计报告:总词数、去重词数、Top 10、删除低频词后的列表大小(覆盖全部)

实验环境:JDK 17+;准备一个英文文本文件 `article.txt`(几百词即可)。

第一步 完成读文件 + 分词 + 词频,打印 HashMap 内容验证

第二步 务 3 的泛型方法签名是本章难点:先写死类型跑通,再泛化

第三步 务 6 的两种删除方式输出应一致,可作为对照检查

参考答案要点

```

import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.HashMap;
import java.util.Iterator;
import java.util.List;
import java.util.Map;

/** 单词统计系统 — 第10章综合实验参考答案
 * 覆盖:集合家族、泛型方法、Comparable/Comparator、HashMap、遍历删除、IO
 */
public class WordStat {

    // 词条类:实现 Comparable(次数降序,同次按字母升序)
    static class WordCount implements Comparable<WordCount> {
        final String word;
        final int count;

        WordCount(String word, int count) {
            this.word = word;
            this.count = count;
        }

        @Override
        public int compareTo(WordCount o) {
            int byCount = Integer.compare(o.count, count); // 次数降序
            return byCount != 0 ? byCount : word.compareTo(o.word);
        }

        @Override
        public String toString() { return word + "(" + count + ")"; }
    }

    // 任务 3:泛型方法—按键值排序,支持升降序
    static <K, V extends Comparable<? super V>> List<Map.Entry<K, V>>
    sortByValue(Map<K, V> map, boolean desc) {
        List<Map.Entry<K, V>> entries = new ArrayList<>(map.entrySet());
        Comparator<Map.Entry<K, V>> cmp = Map.Entry.comparingByValue();
        entries.sort(desc ? cmp.reversed() : cmp);
        return entries;
    }

    public static void main(String[] args) throws IOException {
        // 任务 1:读文件 + 分词(只留字母,转小写)
        String text = Files.readString(Path.of("article.txt"),
            StandardCharsets.UTF_8).toLowerCase();
        String[] tokens = text.split("[^a-z]+");

        // 任务 2:词频统计(merge 标准姿势)
        Map<String, Integer> freq = new HashMap<>();
        for (String t : tokens) {
            if (!t.isEmpty()) {
                freq.merge(t, 1, Integer::sum);
            }
        }
        System.out.println("总词数: " + tokens.length
            + ", 去重词数: " + freq.size());

        // 任务 4:Comparable 自然顺序(直接转 WordCount 排序)
        List<WordCount> byNatural = new ArrayList<>();
        freq.forEach((w, c) -> byNatural.add(new WordCount(w, c)));
        byNatural.sort(null); // 用 compareTo
        System.out.println("Comparable 前 5: " + byNatural.subList(0, 5));

        // 任务 5:Comparator 链式排序(不侵入类的另一种顺序)
    }
}

```

```

List<WordCount> byComp = new ArrayList<>(byNatural);
byComp.sort(Comparator.comparingInt((WordCount wc) -> wc.count)
    .reversed()
    .thenComparing(wc -> wc.word)); // 次数降序 + 字母升序
System.out.println("Comparator 前 5: " + byComp.subList(0, 5));

// 任务 6a:Iterator.remove 删除低频词(次数 ≤ 1)
Iterator<Map.Entry<String, Integer>> it = freq.entrySet().iterator();
int removed1 = 0;
while (it.hasNext()) {
    if (it.next().getValue() <= 1) {
        it.remove();
        removed1++;
    }
}
System.out.println("Iterator 删除低频词: " + removed1
    + ", 剩余: " + freq.size());

// 任务 6b:Stream 过滤重建(不修改原集合,函数式)
Map<String, Integer> freq2 = new HashMap<>();
for (String t : tokens) {
    if (!t.isEmpty()) {
        freq2.merge(t, 1, Integer::sum);
    }
}
List<Map.Entry<String, Integer>> kept = freq2.entrySet().stream()
    .filter(e -> e.getValue() > 1)
    .toList();
System.out.println("Stream 过滤低频词: 保留 " + kept.size() + " 个");

// 任务 3:泛型方法复用(词频 Map 排序,降序前 10)
List<Map.Entry<String, Integer>> top = sortByValue(freq, true);
System.out.println("Top 10:");
top.stream().limit(10)
    .forEach(e -> System.out.println(" " + e.getKey()
        + ": " + e.getValue()));

// 任务 7:报告完成
System.out.println("统计完成,低频词(≤1)已清除。");
}
}

```

设计说明:任务 3 的签名是本章知识的高潮: `<K, V extends Comparable<? super V>>` 同时用到了泛型方法、泛型约束与 PECS(值类型 V 要可比较,其父类实现了 Comparable 也行);任务 4 与 5 故意让两种排序输出一致,证明"自然顺序"与"外部顺序"殊途同归;任务 6 的两种删除方式互相对照,验证 fail-fast 的规避。扩展方向:用 TreeMap 直接按字母序遍历输出全量词表;把 WordCount 改成 record(第14章)去掉样板代码;对超大文件用 Stream 流式统计(第4章)。

下一章预告:第11章 反射与动态特性。Class 对象、构造器与方法的反射调用、动态代理——框架的地基,和 Java 的"运行时自省"能力。

第11章 反射与动态特性

建议学时:4-6 小时 | 难度:★★★★☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 理解 Class 对象:Java 里"类型本身"也是对象
- 掌握反射读取:获取字段、方法、构造器,读取注解
- 掌握反射调用:invoke、构造对象、访问私有成员
- 掌握动态代理 Proxy:在调用前后插入逻辑
- 理解"反射 + 注解 = 框架基座"的底层原理
- 理解反射的代价与安全边界,知道什么时候别用反射

💡 从 C 到 Java:程序认识自己

C 程序在运行时对自己一无所知:没有"这个结构体有哪些字段"的查询 API,没有"这个函数能不能被调用"的运行时检查——一切在编译期定死。Java 有 **反射(Reflection)**:程序在运行时可以检查自己的类结构、读取注解、调用任意方法、创建任意对象。C 里最接近的体验是:用函数指针表 + 字符串映射模拟"按名字找函数"——而反射是这门手艺的语言级完整版。

为什么生产代码需要它?因为框架必须"不认识你的类也能操作你的类":JSON 序列化库拿到任意对象要列出所有字段;ORM(MyBatis/Hibernate)要把任意对象映射成 SQL 参数;依赖注入(Spring)要按注解找到字段并注入;测试框架要运行任意类的 @Test 方法。这些库在编译期根本不知道你的类存在——它们全靠反射在运行时发现与操作。反射 + 注解(第12章)是全部 Java 框架的地基。

动态代理是反射的"行动版"。它让框架"包一层壳"接住你对接口的调用:AOP(事务、日志、权限)、RPC 的远程调用、Mock 测试——都是在壳里做的。Proxy.newProxyInstance 十几行就能实现"所有方法调用都先打日志"的拦截器,C 里等价物是手写一层 dispatch 表。

代价必须正视。反射调用比直接调用慢一个数量级(JDK 有优化缓存,但性能敏感路径仍要避免);setAccessible(true) 能突破封装——这是权力的诱惑,也是安全的裂缝。本章最后给出"何时用、何时绝不用"的清单。

学习顺序:Class 对象(\$11.1)→ 读写结构(\$11.2-11.3)→ 动态代理(\$11.4)→ 框架原理(\$11.5)→ 代价与安全(\$11.6)。

🧰 前置知识自查

- 第6章:类、封装、getter/setter;第10章:泛型(反射 API 大量泛型)
- 第4章:lambda(动态代理的 InvocationHandler 可用 lambda)
- 第12章将讲注解——本章先用"注解是什么"的最小直觉(@Override 那种)

🚀 本章学习路线

1. **第一阶段(约 1.5 小时)**:§11.1-11.2,亲手列出"任意类的全部字段与方法"
2. **第二阶段(约 1.5 小时)**:§11.3-11.4 反射调用与动态代理,实现一个日志拦截器
3. **第三阶段(约 1.5 小时)**:§11.5 框架原理 + §11.6 代价安全,完成章末实验
4. **验收标准**:能用反射把任意对象转成 Map;能写动态代理日志器;能说出三个"别用反射"的场景

本章知识导图

- §11.1 Class 对象:类型的类型
- §11.2 读取类结构:字段 / 方法 / 注解
- §11.3 反射调用:invoke / 构造 / setAccessible
- §11.4 动态代理:Proxy 与 InvocationHandler
- §11.5 反射 + 注解 = 框架基座
- §11.6 代价与安全边界

§11.1 Class 对象:类型的类型

11.1.1 三种拿法

每个类在 JVM 里对应一个 `Class<?>` 对象(第5章的元空间就是存它)。三种拿法:① 实例.`getClass()`;② 类字面量 `String.class`;③ `Class.forName("全限定名")`——只有它不需要编译期知道类型,这是框架们"按字符串操作类型"的入口。

示例 11-1 Class 对象三拿法

```
public class ClassObjectDemo {
    static class Student {
        private String name;
        private int score;
    }

    public static void main(String[] args) throws Exception {
        // 拿法 1:实例的 getClass
        Student s = new Student();
        Class<?> c1 = s.getClass();

        // 拿法 2:类字面量(编译期确定)
        Class<Student> c2 = Student.class;

        // 拿法 3:字符串加载(运行期才知道类名—框架的标准姿势)
        Class<?> c3 = Class.forName("ClassObjectDemo$Student");
        // 注意:嵌套类的全限定名是 外层$内层

        System.out.println(c1 == c2 && c2 == c3);    // true:同一个 Class 对象
        System.out.println("类名: " + c1.getName()); // ClassObjectDemo$Student
        System.out.println("简单名: " + c1.getSimpleName()); // Student
        System.out.println("父类: " + c1.getSuperclass());    // class java.lang.Object

        // 一个类只有一个 Class 对象:反射拿到的就是它
        Object created = c1.getDeclaredConstructor().newInstance();
        System.out.println(created instanceof Student);    // true
    }
}
```

★ 重点:Class 对象 = 类的"体检报告"

Class 对象上全是"查询接口":`getName/getSuperclass/getInterfaces/getDeclaredFields/getDeclaredMethods/getAnnotations`。C 的 `sizeof/offsetof` 只给布局;反射给的是完整的结构清单 + 运行时操作能力。这也解释了第5章为什么元空间装着"类元数据"——反射读的就是它。

§11.2 读取类结构:字段、方法与注解

示例 11-2 用反射“解剖”任意类

```
import java.lang.reflect.Field;
import java.lang.reflect.Method;
import java.lang.reflect.Modifier;

public class InspectDemo {
    static class Student {
        public String name;           // 公有字段
        private int score;           // 私有字段

        public void setScore(int s) { this.score = s; }

        private void secret() { }    // 私方法
    }

    public static void main(String[] args) {
        Class<?> c = Student.class;

        System.out.println("== 所有字段(含私有,不含继承)==");
        for (Field f : c.getDeclaredFields()) {
            System.out.println(" " + Modifier.toString(f.getModifiers())
                               + " " + f.getType().getSimpleName() + " " + f.getName());
        }

        System.out.println("== 所有方法(含私有)==");
        for (Method m : c.getDeclaredMethods()) {
            System.out.println(" " + m.getName() + "("
                               + m.getParameterCount() + " 参数) 返回 "
                               + m.getReturnType().getSimpleName());
        }

        System.out.println("== 按名字精确找 ==");
        try {
            Field score = c.getDeclaredField("score"); // 名字错了抛 NoSuchFieldException
            System.out.println(" 找到字段: " + score.getName());
            Method m = c.getDeclaredMethod("secret"); // 名字+参数类型
            System.out.println(" 找到方法: " + m.getName());
        } catch (NoSuchFieldException | NoSuchMethodException e) {
            System.out.println(" 没找到: " + e);
        }
    }
}
```

⚠ 易错点 1: getFields 与 getDeclaredFields 的区别

getFields 只返回 public(含父类继承的);getDeclaredFields 返回本类声明的全部(含 private,不含父类)。同理 getMethods/getDeclaredMethods。框架代码里 90% 用 getDeclaredFields + setAccessible——因为要操作私有字段。区分记法: "Declared = 自己声明的一切,不管访问级别;无 Declared = 公开可见的一切,含继承"。名字含 checked 异常(NoSuchFieldException 等),必须处理。

§11.3 反射调用:invoke 与 setAccessible

示例 11-3 反射读写字段、调用方法

```

import java.lang.reflect.Field;
import java.lang.reflect.Method;

public class InvokeDemo {
    static class Student {
        private String name = "张三";
        private int score = 60;

        public void addBonus(int bonus) { score += bonus; }

        @Override
        public String toString() { return name + ":" + score; }
    }

    public static void main(String[] args) throws Exception {
        Student s = new Student();

        // 读私有字段:先 getDeclaredField,再 setAccessible(true)
        Field name = Student.class.getDeclaredField("name");
        name.setAccessible(true);           // 突破 private(见 §11.6 代价)
        System.out.println("反射读 name: " + name.get(s));    // 张三

        // 写私有字段
        name.set(s, "李四");
        System.out.println("反射写 name 后: " + s);           // 李四:60

        // 调用公有方法:getMethod(名字, 参数类型...)
        Method addBonus = Student.class.getMethod("addBonus", int.class);
        Object result = addBonus.invoke(s, 15);              // 返回值;void 返回 null
        System.out.println("invoke 后: " + s);               // 李四:75 (result=null)

        // 调用私有方法
        Method secret = Student.class.getDeclaredMethod("secret");
        secret.setAccessible(true);
        // secret.invoke(s);    // 也能调,但"生产代码调私有方法"是设计坏味道

        // 用反射创建对象(替代 new;框架不认识你的类时用它)
        Student s2 = Student.class.getDeclaredConstructor().newInstance();
        System.out.println("反射新建: " + s2);              // 张三:60
    }
}

```

★ 重点:invoke 的返回值与异常包装

invoke 返回 Object,void 方法返回 null,基本类型自动装箱。被调方法抛出的任何异常,invoke 都会包装成 InvocationTargetException 再抛——要用 e.getCause() 拿真正的异常,直接看 e 的堆栈会一头雾水(堆栈指向 invoke 而非业务代码)。这是反射调试的第一坑。

§11.4 动态代理:给接口套一层壳

11.4.1 Proxy 与 InvocationHandler

动态代理:运行期为一个接口生成实现类,所有方法调用都先经过 `InvocationHandler.invoke`。调用方拿到的对象是"代理",行为由你的处理器决定——可以在真正实现前后插入日志、鉴权、事务、延迟加载。C 的对应物:手工写一层"转发函数指针表",Java 是语言级生成。

■ 示例 11-4 日志代理:零侵入地记录所有调用

```

import java.lang.reflect.InvocationHandler;
import java.lang.reflect.Method;
import java.lang.reflect.Proxy;

public class ProxyDemo {
    // 业务接口:真正的服务实现
    interface PaymentService {
        boolean pay(long orderId, double amount);

        double query(long orderId);
    }

    // 真实实现(不想动它的代码)
    static class PaymentServiceImpl implements PaymentService {
        @Override
        public boolean pay(long orderId, double amount) {
            return amount > 0; // 假装真的扣款成功
        }

        @Override
        public double query(long orderId) {
            return 99.5;
        }
    }

    // 日志处理器:任何方法调用先打日志再转发
    static class LogHandler implements InvocationHandler {
        private final Object target; // 被代理的真实对象

        LogHandler(Object target) { this.target = target; }

        @Override
        public Object invoke(Object proxy, Method method, Object[] args)
            throws Throwable {
            System.out.println("【代理】调用 " + method.getName()
                + ", 参数: " + java.util.Arrays.toString(args));
            Object result = method.invoke(target, args); // 转发给真实对象
            System.out.println("【代理】返回: " + result);
            return result;
        }
    }

    public static void main(String[] args) {
        // 生成代理:接口列表 + 处理器;返回的对象类型就是接口
        PaymentService service = (PaymentService) Proxy.newProxyInstance(
            PaymentService.class.getClassLoader(),
            new Class<?>[]{PaymentService.class}, // 要代理的接口
            new LogHandler(new PaymentServiceImpl()));

        service.pay(1001L, 88.0);
        service.query(1001L);
        // 输出:两次调用都被日志包裹,业务代码零改动
    }
}

```



工程建议:动态代理的三大现实应用

1. **AOP(面向切面)**:Spring 的事务/日志/安全注解,本质是代理在方法前后插逻辑(Spring 对接口用 JDK 代理,对类用 CGLIB 子类代理)
2. **RPC 远程调用**:客户端拿到"代理接口",invoke 里把参数序列化发到服务端——调用方感觉像本地调用(Feign、Dubbo 的原理)
3. **Mock 测试**:Mockito 用代理拦截接口调用,返回预设值,无需真实依赖

§11.5 反射 + 注解 = 框架基座

11.5.1 一个迷你映射器

框架模式三步走: ① 用注解(第12章)给类打标记; ② 反射扫描注解与字段; ③ 按标记执行通用逻辑。下面实现"对象 ↔ Map"的迷你映射器——JSON 库和 ORM 的核心动作就是它。注解先"借用"一个自定义注解(第12章会教如何定义)。

示例 11-5 迷你对象映射器(框架雏形)

```
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
import java.lang.reflect.Field;
import java.util.HashMap;
import java.util.Map;

public class MiniMapper {

    // 自定义注解:字段的"列名"(第12章详解定义语法)
    @Retention(RetentionPolicy.RUNTIME) // 必须:反射才能读到
    @Target(ElementType.FIELD)           // 只能标字段
    @interface Column {
        String value();
    }

    // 业务类:字段打上注解
    static class Student {
        @Column("stu_id")
        private String id;
        @Column("stu_name")
        private String name;
        private int score; // 没注解的字段:跳过

        Student(String id, String name, int score) {
            this.id = id;
            this.name = name;
            this.score = score;
        }

        // 需要 getter/setter 的"无参构造 + setter"版本见扩展练习
        String getId() { return id; }
    }

    // 核心:对象 → Map(通用逻辑:不认识 Student 也能转)
    static Map<String, Object> toMap(Object obj) throws Exception {
        Map<String, Object> map = new HashMap<>();
        for (Field f : obj.getClass().getDeclaredFields()) {
            Column col = f.getAnnotation(Column.class); // 反射读注解
            if (col == null) {
                continue; // 没注解就不导出
            }
            f.setAccessible(true);
            map.put(col.value(), f.get(obj)); // 注解值当键
        }
        return map;
    }

    public static void main(String[] args) throws Exception {
        Student s = new Student("2024001", "张三", 88);
        System.out.println(toMap(s));
        // {stu_id=2024001, stu_name=张三}:score 因无注解被跳过
    }
}
```

★ 重点:框架的"不认识你"是怎么成立的

toMap 的形参是 Object——它编译期不认识 Student,运行期通过 `getClass().getDeclaredFields()` 发现字段,通过 `getAnnotation` 发现契约,通过 `setAccessible + get` 读取值。你新增任何类、打上注解,这个"通用逻辑"立即生效,一行框架代码不用改。这就是 Spring/MyBatis/Jackson 与你写的业务代码解耦的全部秘密。框架 = 通用反射逻辑 + 注解契约 + 你的业务类。

§11.6 代价与安全边界

问题	说明	对策
性能	反射调用慢一个数量级(解析、装箱、安全检查)	热路径不用反射;框架内部有缓存(JVM 的反射缓存、MethodHandle)
可读性	类型错误延迟到运行期;IDE 找不到调用关系	封装成框架/工具;业务代码不裸写反射
封装破坏	<code>setAccessible(true)</code> 能读私有、改 final	只在自己代码里用;别对第三方库的对象乱开
模块化冲突	JPMS 下模块可拒绝反射(第13章)	模块开放 open 或显式授权

⚠ 易错点 2:三个"别用反射"的场景

1. 能用接口/多态解决,别用反射:直接调用快、类型安全、可读(反思:框架才需要"不认识你")
2. 性能敏感的循环体内,别用反射:十亿级调用场景,缓存 Method 对象或换编译期方案
3. 依赖"私有成员"做业务,别用反射:那是在补封装设计的洞,正确做法是修类的设计

示例 11-6 性能对策:缓存反射对象

```
import java.util.concurrent.ConcurrentHashMap;

public class ReflectCache {
    // 反射查找很贵(解析、安全检查);缓存后只查一次
    private static final ConcurrentHashMap<String, java.lang.reflect.Method>
        CACHE = new ConcurrentHashMap<>();

    static java.lang.reflect.Method cachedMethod(String className,
        String methodName) throws Exception {
        return CACHE.computeIfAbsent(className + "#" + methodName, key -> {
            try {
                return Class.forName(className).getMethod(methodName);
            } catch (ReflectiveOperationException e) {
                throw new IllegalArgumentException("反射失败: " + key, e);
            }
        });
    }

    public static void main(String[] args) throws Exception {
        // 首次查找后,后续调用直接命中缓存
        var m1 = cachedMethod("java.lang.String", "length");
        var m2 = cachedMethod("java.lang.String", "length");
        System.out.println(m1 == m2);    // true:同一个 Method 对象
    }
}
```

本章速查表

主题	要点
Class 对象	类型也是对象;三种拿法:getClass() / 类.class / Class.forName("全限定名")
嵌套类名字	全限定名是 外层\$内层(如 ClassObjectDemo\$Student)
字段	getDeclaredFields 全部(含私有)/ getFields 仅 public(含继承)
方法	getDeclaredMethod(名字, 参数类型...) 精确找;NoSuchMethodException 是 checked
setAccessible	突破 private 访问;用前先 getDeclared;只对自己代码用
invoke	返回 Object;void 返回 null;被调异常包成 InvocationTargetException,取 getCause()
新建对象	getDeclaredConstructor().newInstance();需要无参构造器
动态代理	Proxy.newProxyInstance(加载器, 接口数组, InvocationHandler);调用全部经过 invoke
代理应用	AOP 事务日志 / RPC 远程调用 / Mock 测试
框架模式	注解标记 → 反射扫描 → 通用逻辑;Object 形参 + getDeclaredFields 发现一切
性能	反射慢一个数量级;热路径禁用;框架用缓存弥补
安全	setAccessible 是权力也是裂缝;JPMS 可封禁反射(第13章)

最小必会清单

- 能说出 Class 对象的三种拿法与各自的编译期依赖
- 能列出"任意类的所有字段(含私有)及其类型"的代码
- 能写出反射读写字段、调用方法的完整代码(含 setAccessible 与异常处理)
- 能解释 InvocationTargetException 的包装与 getCause
- 能写出动态代理日志拦截器,并说出三大应用场景
- 能解释"反射 + 注解 = 框架基座"的三步模式
- 能说出三个"别用反射"的场景与性能对策

自测题

Q 1. Class.forName 与 类.class 的区别是什么?框架为什么用 forName?

✓ 参考答案

类.class 在编译期确定,编译器就知道类型;Class.forName 接收字符串,运行期才解析——字符串可以来自配置文件、用户输入、注册表。框架在编译期不认识你的业务类,只能拿到"类名字字符串"运行时加载。代价:名字写错、类不存在,错误发生在运行期(ClassNotFoundException)。

Q 2. 反射调用一个抛异常的方法,为什么 catch 到的是 InvocationTargetException?怎么拿真正的异常?

✓ 参考答案

invoke 为了不吞掉被调方法的异常,把任何异常都包装进 InvocationTargetException 抛给调用方(这样 invoke 的签名才能是"只抛 ReflectiveOperationException 家族")。真正异常在 e.getCause() 里,排查日志时要打印 getCause 的完整堆栈,否则只看到 invoke 一层的栈,找不到业务错误点。

Q 3. getDeclaredFields 拿到的字段名,能不能直接用于 setAccessible + get?有哪些前置条件?

✅ 参考答案

可以。前置:① 字段确实在该类声明(不在父类——父类字段要 `getSuperclass` 递归找);② `setAccessible(true)` 成功(JPMS 未封禁、安全管理器允许);③ 静态字段 `get(null)`、实例字段 `get(实例)`。另外 `getDeclaredFields` 不保证顺序——依赖顺序的逻辑必须显式排序(如按注解的 `order` 值)。

Q 4. 动态代理为什么只能代理接口,不能代理类?

✅ 参考答案

JDK 代理生成的是“实现接口的新类”——Java 单继承,无法继承任意目标类。代理类没有目标类的私有状态与 `final` 方法。要代理类(无接口的类)用 CGLIB/ByteBuddy(生成子类,字节码技术),但 `final` 类与 `final` 方法依然代理不了。选型:面向接口编程(第6章)的好处在这里兑现——接口是天然的代理边界。

Q 5. 用反射实现的 JSON 序列化,为什么比手写 `getter` 拼接慢?哪些环节慢?

✅ 参考答案

慢在三处:① 每次 `get` 要装箱(基本类型转 `Object`)与类型判断;② `setAccessible` 有安全检查(现代 JVM 有缓存优化);③ 方法分发走 `invoke` 的间接层。缓解:缓存 `Field/Method` 对象(不每次重新查找)、用 `MethodHandle`、或编译器在首次反射后生成直接调用的字节码(Jackson 的 `Afterburner`、Gson 的反射缓存)。热路径上,手写 `getter` 依然最快。

Q 6. 你的工具方法接受任意对象并打印其所有字段名与值。写出核心代码(注意私有字段与父类字段)。

✅ 参考答案

```
static void dumpFields(Object obj) throws IllegalAccessException {
    Class<?> c = obj.getClass();
    while (c != null) { // 沿继承链往上走
        for (Field f : c.getDeclaredFields()) {
            if (java.lang.reflect.Modifier.isStatic(f.getModifiers())) {
                continue; // 跳过静态字段
            }
            f.setAccessible(true);
            System.out.println(c.getSimpleName() + "." + f.getName()
                               + " = " + f.get(obj));
        }
        c = c.getSuperclass(); // 处理父类字段
    }
}
```

要点:while 循环沿 `getSuperclass` 处理父类字段(到 `Object` 为止);跳过静态字段;`setAccessible` 包住完整读取。生产等价物:Apache Commons 的 `FieldUtils.getAllFields`、Gson 的内部反射——理解原理后,工具库只是这几十行的工程化。

章末实验题:迷你对象映射器

实验 11:迷你对象映射器 + 日志代理

实验目标:编写两部分程序:① 对象 ↔ Map 的迷你映射器(含注解支持与类型转换);② 动态代理日志拦截器。综合运用本章全部知识点。

实验要求:

- 任务 1:定义注解 @Column(value, nullable),分别可标字段;nullable 默认 true(覆盖 §11.2 注解读取,注解定义语法见第12章预习)
- 任务 2:toMap(Object):只导出带 @Column 的字段,键用注解 value;私有字段正常读取(覆盖 §11.2-11.3 字段反射)
- 任务 3:fromMap(Class<T>, Map):按注解映射回对象,字段类型做基本类型转换(数字字符串转 int 等),null 且 nullable=false 时抛异常(覆盖 §11.3 构造与写入)
- 任务 4:字段映射支持继承:父类的 @Column 字段也处理(覆盖 §11.2 继承链)
- 任务 5:定义接口 Repository 与实现,用动态代理包装:每次调用前打印"调用 + 参数",调用后打印"耗时"(覆盖 §11.4 动态代理)
- 任务 6:代理对方法调用做"幂等保护":query 类方法(名字以 query 开头)失败自动重试 1 次(覆盖 §11.4 代理逻辑)
- 任务 7:main 里演示:Student 对象转 Map、Map 转回对象、代理调用,输出完整报告(覆盖全部)

实验环境:JDK 17+。

第一步 写注解与 toMap(最容易见效),跑通再写 fromMap 的类型转换

第二步 任务 4 与自测题 6 的思路一致:while (c != null) 沿父类走

第三步 任务 6 的"重试"在 InvocationHandler.invoke 里用循环包住 method.invoke

参考答案要点

```

import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
import java.lang.reflect.Field;
import java.lang.reflect.InvocationHandler;
import java.lang.reflect.Method;
import java.lang.reflect.Proxy;
import java.util.HashMap;
import java.util.Map;

/** 迷你对象映射器 + 日志代理 — 第11章综合实验参考答案
 * 覆盖:注解定义与读取、字段反射、继承链扫描、类型转换、动态代理、重试
 */
public class MiniMapperLab {

    // 任务 1:字段注解
    @Retention(RetentionPolicy.RUNTIME)
    @Target(ElementType.FIELD)
    @interface Column {
        String value();

        boolean nullable() default true; // 默认允许 null
    }

    // 业务类:含父类字段(任务 4)
    static class BaseEntity {
        @Column("id", nullable = false)
        private Long id;
    }

    static class Student extends BaseEntity {
        @Column("stu_name")
        private String name;
        @Column("stu_score")
        private Integer score; // 用包装类型,便于 null 语义
        private String secret; // 无注解:不参与映射
    }

    // 任务 2:对象 → Map(沿继承链扫描)
    static Map<String, Object> toMap(Object obj) throws IllegalAccessException {
        Map<String, Object> map = new HashMap<>();
        for (Class<?> c = obj.getClass(); c != null; c = c.getSuperclass()) {
            for (Field f : c.getDeclaredFields()) {
                Column col = f.getAnnotation(Column.class);
                if (col == null) {
                    continue;
                }
                f.setAccessible(true);
                map.put(col.value(), f.get(obj));
            }
        }
        return map;
    }

    // 任务 3:Map → 对象(类型转换 + nullable 校验)
    static <T> T fromMap(Class<T> type, Map<String, Object> data) throws Exception {
        T obj = type.getDeclaredConstructor().newInstance(); // 需要无参构造
        for (Class<?> c = type; c != null; c = c.getSuperclass()) {
            for (Field f : c.getDeclaredFields()) {
                Column col = f.getAnnotation(Column.class);
                if (col == null || !data.containsKey(col.value())) {
                    continue;
                }
                f.setAccessible(true);
                Object raw = data.get(col.value());
                if (raw == null) {
                    if (!col.nullable()) {

```

```

        throw new IllegalArgumentException(
            "字段 " + f.getName() + " 不允许为 null");
    }
    f.set(obj, null);
    continue;
}
// 基本类型转换:字符串/数字 → 字段类型
Class<?> ft = f.getType();
Object converted = raw;
if (ft == Long.class || ft == long.class) {
    converted = Long.parseLong(String.valueOf(raw));
} else if (ft == Integer.class || ft == int.class) {
    converted = Integer.parseInt(String.valueOf(raw));
}
f.set(obj, converted);
}
}
return obj;
}

// 任务 5+6:代理接口与处理器
interface Repository {
    String queryById(long id);

    void save(String data);
}

static class RepositoryImpl implements Repository {
    @Override
    public String queryById(long id) {
        return "数据#" + id;
    }

    @Override
    public void save(String data) {
        System.out.println("    [真实保存] " + data);
    }
}

static class LoggingHandler implements InvocationHandler {
    private final Object target;

    LoggingHandler(Object target) { this.target = target; }

    @Override
    public Object invoke(Object proxy, Method method, Object[] args)
        throws Throwable {
        System.out.println("    [代理] 调用 " + method.getName()
            + " 参数 " + java.util.Arrays.toString(args));
        long start = System.nanoTime();
        int attempts = method.getName().startsWith("query") ? 2 : 1;
        Throwable last = null;
        for (int i = 0; i < attempts; i++) {
            try {
                Object result = method.invoke(target, args);
                long cost = (System.nanoTime() - start) / 1_000_000;
                System.out.println("    [代理] 返回 " + result + " 耗时 " + cost + "ms");
                return result;
            } catch (java.lang.reflect.InvocationTargetException e) {
                last = e.getCause();
                System.out.println("    [代理] 第 " + (i + 1)
                    + " 次失败: " + last.getMessage());
            }
        }
        throw last;
    }
}

// 重试耗尽,抛真正的异常

public static void main(String[] args) throws Exception {

```

```

// 任务 7:完整演示
Student s = new Student();
s.id = 1001L;
s.name = "张三";
s.score = 88;
s.secret = "不导出";           // 无注解字段

Map<String, Object> map = toMap(s);
System.out.println("== toMap ==");
System.out.println(" " + map);   // {stu_score=88, stu_name=张三, id=1001}

Student back = fromMap(Student.class, map);
System.out.println("== fromMap ==");
System.out.println(" id=" + back.id + " name=" + back.name
    + " score=" + back.score + " secret=" + back.secret);

System.out.println("== 代理 ==");
Repository repo = (Repository) Proxy.newProxyInstance(
    Repository.class.getClassLoader(),
    new Class<?>[]{Repository.class},
    new LoggingHandler(new RepositoryImpl()));
repo.queryById(42);
repo.save("订单数据");
System.out.println("实验完成。");
}
}

```

设计说明:任务 4 的继承链扫描(toMap/fromMap 里的 for 循环沿 getSuperclass 上行)同时覆盖了父类字段与"字段在哪个类声明"的可读性;任务 3 的类型转换只演示 Long/Integer 两条分支,实际框架是"类型转换器注册表"(String→LocalDate、byte[]→图片等),原理一致;任务 6 的重试策略放在代理里——业务代码完全无感,这正是 AOP 的核心价值。扩展方向:把 fromMap 支持构造器参数映射(第14章 record 天然适合)、加 @Transient 排除字段、用泛型 + Class<T> 参数把工具类改成类型安全版本。

下一章预告:第12章 注解与元编程。注解的定义、元注解、处理器的完整实现——把第11章"借用"的注解正式讲透。

第12章 注解与元编程

建议学时:4-6 小时 | 难度:★★★☆☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 理解注解的本质:编译器与框架可读的结构化元数据
- 掌握注解定义语法:@interface、元素类型、默认值
- 掌握四个核心元注解:Retention / Target / Documented / Inherited
- 掌握运行时注解处理:反射读取与执行校验逻辑
- 了解编译期注解处理(APT)与内置注解
- 掌握注解驱动的工程模式:校验器、配置绑定

💡 从 C 到 Java:会执行的"注释"

C 程序员对"注释"的心智:给人看的,编译器忽略。Java 的 **注解(Annotation)** 是给程序看的"注释"——编译器、框架、工具可以读取并据此行动的结构化元数据。第11章"借用"过它:字段上的 @Column 注解,让一个不认识你的框架知道"这个字段导出叫什么名字"。

注解 = 声明式编程。你声明"这个字段不能为 null",而不是写"if (x == null) throw ...";框架在运行时读注解,替你执行校验。好处:业务代码只写"是什么",通用逻辑(校验、序列化、注入)在框架里写一次,对所有带注解的类生效。这就是"配置与代码分离"的终极形态——配置就在代码里,但以元数据的形式。

C 里最接近的东西是什么?没有直接对应物。最像的是:代码生成器读取的特殊标记注释 + 函数指针表。X-Macro、__attribute__((...)) 有一点味道,但注解是类型安全、可携带参数、可被反射查询的正式语言设施。

Retention 决定注解的命运。SOURCE(编译完就扔,如 @Override)、CLASS(进字节码,默认)、RUNTIME(进字节码且反射可读)。想要"运行时读取执行"的注解,必须标 RUNTIME——这是新手第一坑:定义得挺好,反射却读不到。

学习顺序:先懂"注解是什么"与内置注解(\$12.1),定义语法(\$12.2),元注解(\$12.3),运行时处理(\$12.4),编译期处理(\$12.5),工程模式(\$12.6)。

📦 前置知识自查

- 第11章:反射读字段与注解(本章直接续写)
- 第6章:封装与字段;第10章:泛型(注解元素与泛型约束)
- 能识别 @Override/@Deprecated 等见过的注解

🚀 本章学习路线

1. **第一阶段(约 1 小时):**\$12.1-12.2 定义与元注解,写出自己的第一个 @interface
2. **第二阶段(约 1.5 小时):**\$12.3-12.4 Retention/Target 与运行时处理,实现字段校验器
3. **第三阶段(约 1.5 小时):**\$12.5-12.6 编译期处理与工程模式,完成章末实验
4. **验收标准:**能定义带默认值注解;能解释四个元注解;能写出运行时校验器

本章知识导图

- §12.1 注解的本质:元数据 / 内置注解
- §12.2 定义注解:@interface / 元素 / 默认值
- §12.3 元注解:Retention / Target / Documented / Inherited
- §12.4 运行时处理:反射读取与执行
- §12.5 编译期处理:APT 与内置注解
- §12.6 注解工程模式:校验器 / 配置绑定

§12.1 注解的本质与内置注解

12.1.1 注解 vs 注释 vs C 的替代品

对比项	C 注释	Java 注解
读者	人	编译器、框架、工具(人也读)
是否进产物	编译后消失	按 Retention:可进字节码、可反射
能否携带数据	自由文本(无法结构化读取)	结构化元素(类型检查的键值对)
能否触发行为	不能	能:编译器检查、处理器生成代码、框架执行逻辑
C 的近似物	/* 注释 */、X-Macro 宏	(无直接对应;__attribute__ 略近)

12.1.2 三个最常用的内置注解

注解	作用	谁消费它
@Override	标记"这是重写父类方法"	编译器:签名写错立即报错(第6章)
@Deprecated	标记"别用了,有替代"	编译器:使用时给警告;IDE 划线
@SuppressWarnings("...")	压制特定警告(如 unchecked)	编译器

示例 12-1 内置注解实战

```
import java.util.List;

public class BuiltinAnnotations {
    // @Deprecated:标记旧 API,调用处会警告
    @Deprecated
    static void oldMethod() {
        System.out.println("旧方法,请用 newMethod");
    }

    static void newMethod() {
        System.out.println("新方法");
    }

    // @SuppressWarnings:压掉"裸类型"警告(旧代码迁移时的过渡手段)
    @SuppressWarnings({"unchecked", "rawtypes"})
    static List rawList() {
        return new java.util.ArrayList(); // 裸类型:正常会警告
    }

    public static void main(String[] args) {
        newMethod();
        oldMethod(); // IDE 会提示"已弃用"
        System.out.println(rawList().size());
    }
}
```

★ 重点:注解自己不做任何事

注解本身是"标签",行为全部来自消费者: `@Override` 没写对,编译器检查; `@Deprecated` 没处理,警告;自定义注解没人读,就是装饰。所以学注解 = 学"定义" + 学"消费"。消费有两类:编译器(编译期处理,§12.5)与反射/框架(运行时处理,§12.4)。

§12.2 定义注解:@interface

12.2.1 语法与元素规则

定义用 `@interface` 关键字;里面声明"元素"(像方法声明,没有方法体):元素类型只允许 8 种基本类型、String、Class、枚举、注解、以及它们的数组;可以有默认值(default)。使用时不写有默认值的元素。

示例 12-2 定义自己的注解

```

import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

// 字段校验注解:标记"这个字段不能为空"
@Retention(RetentionPolicy.RUNTIME)    // 运行时保留(反射可读)
@Target(ElementType.FIELD)              // 只能用在字段上
public @interface NotNull {
    // 元素:使用时可省略(有默认值)
    String message() default "字段不能为 null";

    // 元素:必须显式提供(无默认值)
    boolean checkEmpty();                // 空字符串也算非法
}

// 用法:把校验"声明"在字段上
class User {
    @NotNull(checkEmpty = true)
    private String name;

    @NotNull(checkEmpty = false, message = "邮箱可选,但不能为 null")
    private String email;
}

```

⚠ 易错点 1:元素类型与命名限制

元素类型有限制:基本类型/String/Class/枚举/注解/它们的数组——想存"任意对象"不行,想存 List 不行。命名惯例:只有一个元素时叫 value,使用时可省略 = 写成 @Column("stu_id") (第11章);多个元素时必须写 @NotNull(checkEmpty = true)。注解本身也是类型,可以嵌套(注解里放注解),框架里常见。

§12.3 元注解:注解的注解

元注解	作用	生产要点
@Retention	注解活多久	SOURCE 编译即弃;CLASS 进字节码;RUNTIME 反射可读(自定义运行逻辑必用)
@Target	注解能标在哪	TYPE 类/接口 / FIELD 字段 / METHOD 方法 / PARAMETER 参数 / CONSTRUCTOR 构造器 / ANNOTATION_TYPE 注解 / PACKAGE 包;不写 = 到处都能标(不推荐)
@Documented	进 javadoc	公开 API 注解建议加
@Inherited	子类继承父类的注解	只对类上的注解有效;字段/方法注解不继承

示例 12-3 元注解组合拳


```

import java.lang.annotation.Documented;
import java.lang.annotation.ElementType;
import java.lang.annotation.Inherited;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

// 类级别的"权限"注解:可继承、进 javadoc
@Documented
@Inherited
@Retention(RetentionPolicy.RUNTIME)
@Target({ElementType.TYPE, ElementType.METHOD})
public @interface RequiresAuth {
    String role() default "user";    // 需要的角色

    boolean audit() default false;   // 是否记录审计日志
}

// 用法:类上标一次,子类自动继承
@RequiresAuth(role = "admin", audit = true)
class AdminApi { }

class SubAdminApi extends AdminApi { }    // 自动继承 @RequiresAuth

```

⚠ 易错点 2:Retention 没设成 RUNTIME,反射读不到

默认 Retention 是 CLASS:注解进字节码,但 JVM 不保留给反射。getAnnotation 返回 null,代码不报错,就是读不到——最隐蔽的注解 bug。自检顺序:① RetentionPolicy.RUNTIME 写了没;② Target 包含你要标的元素类型没;③ 反射用 getDeclaredAnnotation 而不是 getAnnotation(后者对继承的处理不同)。

示例 12-4 读取类级注解(含 @Inherited 效果)

```

import java.lang.reflect.Method;

public class ReadClassAnno {
    // 读取类的权限注解:getAnnotation 对类级注解会沿继承链查找
    static String roleOf(Class<?> type) {
        RequiresAuth ra = type.getAnnotation(RequiresAuth.class);
        return ra == null ? "(无权限要求)" : ra.role();
    }

    // 读取方法上的注解(不继承:重写后是新方法)
    static boolean auditOf(Class<?> type, String methodName) throws Exception {
        Method m = type.getMethod(methodName);
        RequiresAuth ra = m.getAnnotation(RequiresAuth.class);
        return ra != null && ra.audit();
    }

    public static void main(String[] args) throws Exception {
        System.out.println(roleOf(AdminApi.class));    // admin
        System.out.println(roleOf(SubAdminApi.class)); // admin:继承生效
    }
}
// 注:RequiresAuth 定义见示例 12-3,此处省略

```

§12.4 运行时处理:注解校验器

12.4.1 处理流程:扫描 → 读注解 → 执行逻辑

运行时处理的标准三步(第11章已预热):① getDeclaredFields 拿到字段;② 字段.getAnnotation(X.class) 读注解;③ 按注解参数执行通用逻辑。下面实现一个完整的校验器——生产校验框架(Hibernate Validator)就是它的工程化版本。

示例 12-4 运行时注解校验器

```

import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
import java.lang.reflect.Field;
import java.util.ArrayList;
import java.util.List;

public class Validator {
    // 注解 1:不能为 null/空串
    @Retention(RetentionPolicy.RUNTIME)
    @Target(ElementType.FIELD)
    public @interface NotEmpty {
        String message() default "字段不能为空";
    }

    // 注解 2:数值范围
    @Retention(RetentionPolicy.RUNTIME)
    @Target(ElementType.FIELD)
    public @interface Range {
        int min() default 0;

        int max() default Integer.MAX_VALUE;

        String message() default "数值越界";
    }

    // 通用校验:不认识 Order 类也能校验它
    static List<String> validate(Object obj) throws IllegalAccessException {
        List<String> errors = new ArrayList<>();
        for (Field f : obj.getClass().getDeclaredFields()) {
            f.setAccessible(true);
            Object value = f.get(obj);

            NotEmpty ne = f.getAnnotation(NotEmpty.class);
            if (ne != null && (value == null
                || (value instanceof String s && s.isEmpty())) {
                errors.add(f.getName() + ": " + ne.message());
            }

            Range r = f.getAnnotation(Range.class);
            if (r != null && value instanceof Number n
                && (n.intValue() < r.min() || n.intValue() > r.max())) {
                errors.add(f.getName() + ": " + r.message()
                    + "(范围 " + r.min() + "~" + r.max() + ")");
            }
        }
        return errors;
    }

    // 业务类:声明校验规则
    static class Order {
        @NotEmpty
        private String orderId;

        @NotEmpty(message = "收件人必须填写")
        private String receiver;

        @Range(min = 1, max = 9999, message = "数量不合法")
        private int quantity;

        // 构造器(演示用)
        Order(String orderId, String receiver, int quantity) {
            this.orderId = orderId;
            this.receiver = receiver;
            this.quantity = quantity;
        }
    }
}

```

```

public static void main(String[] args) throws Exception {
    Order good = new Order("A1001", "张三", 3);
    System.out.println("合法订单校验: " + validate(good));    // []

    Order bad = new Order("", " ", 0);
    System.out.println("非法订单校验: " + validate(bad));
    // [orderId: 字段不能为空, receiver: 收件人必须填写,
    //   quantity: 数量不合法(范围 1~9999)]
}
}

```

★ 重点:声明式校验的三层收益

① 规则与字段在一起:读 Order 类源码,一眼看到每个字段的约束,不用翻校验方法;② 校验逻辑只写一次:新类加注解即获校验能力,validate 一行不改;③ 规则可组合:一个字段可以同时挂 @NotEmpty + @Range + 自定义注解,框架按顺序执行。这就是"声明式"vs"命令式"的差别:你声明"是什么",框架负责"怎么做"。

§12.5 编译期处理与内置注解

12.5.1 两种消费时机

消费时机	工具	典型应用	是否进运行时
编译期	注解处理器 (APT,javac - processor)	Lombok 生成 getter、 DataBinding 生成代码、 Google AutoService	处理完即消失(或生成新源码)
运行时	反射(getAnnotation + 逻辑)	校验器(§12.4)、Spring 注 入、MyBatis 映射	必须 RetentionPolicy.RUNTIME

示例 12-5 编译期处理的最小形态(概念演示)

```

// 编译器看到 @Override 时做的"处理",是内置注解处理器的范例:
// 1. 扫描目标类的所有方法
// 2. 对每个标注 @Override 的方法,检查父类/接口是否有同签名方法
// 3. 没有 → 编译错误 "method does not override a method from its superclass"

// 同理 @Deprecated:
// 1. 扫描所有方法调用
// 2. 若目标方法带 @Deprecated 且调用点未同时标注
// 3. 生成警告,IDE 划线

// 自研 APT 的骨架(javax.annotation.processing,JDK 老 API):
// @SupportedAnnotationTypes("com.example.MyAnno")
// public class MyProcessor extends AbstractProcessor {
//     @Override
//     public boolean process(Set<? extends TypeElement> annotations,
//                          RoundEnvironment roundEnv) {
//         // 扫描带注解的元素,生成新 .java 文件或报错
//         return true;
//     }
// }

```

🚀 工程建议:什么时候用编译期,什么时候用运行时

需要"生成代码"或"编译期报错"→ APT(Lombok、Builder 生成);需要"读取对象运行时状态再决策"→ 反射(校验、注入、序列化)。APT 生成代码性能最好(没有反射开销),但开发体验重(要写处理器、处理增量编译);反射方案简单直接,代价是性能与可读性。多数业务项目的自定义注解走运行时,框架级工具才上 APT。

§12.6 注解工程模式:配置绑定

12.6.1 把 Properties 绑定到带注解的类

生产里的"配置类"模式:定义配置类,字段打上注解声明"从哪个配置键读取、默认值是什么",框架一行代码把 Properties 绑定成类型安全的配置对象——Spring 的 `@ConfigurationProperties` 就是这个模式的巅峰。

示例 12-6 注解驱动的配置绑定器

```

import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
import java.lang.reflect.Field;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.Properties;

public class ConfigBinder {
    // 配置键注解
    @Retention(RetentionPolicy.RUNTIME)
    @Target(ElementType.FIELD)
    public @interface Prop {
        String key();

        String def() default "";    // 默认值(字符串形态)
    }

    // 通用绑定:把 Properties 灌进任意"打了注解的类"
    static <T> T bind(Class<T> type, Properties props) throws Exception {
        T cfg = type.getDeclaredConstructor().newInstance();
        for (Field f : type.getDeclaredFields()) {
            Prop p = f.getAnnotation(Prop.class);
            if (p == null) {
                continue;
            }
            String raw = props.getProperty(p.key(), p.def());
            f.setAccessible(true);
            Class<?> ft = f.getType();
            if (ft == int.class || ft == Integer.class) {
                f.setInt(cfg, Integer.parseInt(raw.trim()));
            } else if (ft == boolean.class || ft == Boolean.class) {
                f.setBoolean(cfg, Boolean.parseBoolean(raw.trim()));
            } else {
                f.set(cfg, raw);
            }
        }
        return cfg;
    }

    // 配置类:声明"我要什么配置"
    static class ServerConfig {
        @Prop(key = "server.port", def = "8080")
        private int port;

        @Prop(key = "server.name", def = "unnamed")
        private String name;

        @Prop(key = "server.debug", def = "false")
        private boolean debug;

        @Override
        public String toString() {
            return "ServerConfig{port=" + port + ", name='" + name
                + "', debug=" + debug + "}";
        }
    }

    public static void main(String[] args) throws Exception {
        Properties props = new Properties();
        Path f = Path.of("server.properties");
        if (Files.exists(f)) {
            try (var in = Files.newBufferedReader(f, StandardCharsets.UTF_8)) {
                props.load(in);
            }
        }
    }
}

```

```
// 模拟配置文件内容:
props.setProperty("server.port", "9001");
props.setProperty("server.name", "主站");

ServerConfig cfg = bind(ServerConfig.class, props);
System.out.println(cfg);
// ServerConfig{port=9001, name='主站', debug=false}
// port 与 name 来自配置,debug 用默认值:绑定器正确合并
}
}
```

⚠ 易错点 3:注解默认值 vs 配置默认值,别混

两套默认值要分清: 注解元素的 default(如 def="8080")是"代码里写死的缺省";Properties 里没有该键时用它。真实配置有值则覆盖两者。生产还有第三层:环境变量覆盖配置(第7章)——绑定器的优先级链:环境变量 > 配置文件 > 注解默认值。设计配置系统时把这条链写清楚,是大型项目的必修课。

本章速查表

主题	要点
注解本质	结构化元数据;标签本身不做事,行为来自消费者(编译器/框架)
定义语法	public @interface X { 元素(); 元素() default 值; }
元素类型	基本类型/String/Class/枚举/注解/数组;不能存任意对象或 List
@Retention	SOURCE 编译即弃 / CLASS 进字节码 / RUNTIME 反射可读(自定义运行逻辑必选)
@Target	TYPE/FIELD/METHOD/PARAMETER/CONSTRUCTOR/ANNOTATION_TYPE/PACKAGE;多值用花括号
@Inherited	类注解可被子类继承;字段/方法注解不继承
读取	field.getAnnotation(X.class) → null 表示没有;Retention 不对就永远 null
运行时处理	getDeclaredFields → getAnnotation → 执行逻辑;校验器/注入/序列化
编译期处理	APT 注解处理器:生成源码/编译报错;Lombok 的原理
内置注解	@Override(编译检查)/ @Deprecated(警告)/ @SuppressWarnings(压警告)
工程模式	声明式校验、配置绑定;优先级链:环境变量 > 配置文件 > 注解默认值

最小必会清单

- 能解释"注解是给程序看的注释"与三种消费方式
- 能写出带默认值与必填元素的注解定义
- 能说出四个元注解的作用与各自的生产要点
- 能解释"Retention 不是 RUNTIME 时反射读不到"的排查顺序
- 能写出字段校验器的完整代码(扫描-读取-执行)
- 能说出 APT 与运行时处理的适用场景
- 能写出配置绑定器,并说出默认值优先级链

自测题

Q 1. 你定义了一个注解,反射却读不到。列出按顺序排查的三步。

✓ 参考答案

① Retention 是否为 RUNTIME(默认 CLASS 只进字节码,反射不可见);② Target 是否包含你要标注的元素类型(标到类上却 @Target(FIELD) 会编译错误,但标错"目标"是另一个坑:用了 @Target 未列出的位置会编译失败,说明 Target 限制了使用处);③ 读取 API 是否正确(getDeclaredAnnotation 管本类声明,getAnnotation 涉及继承规则)。

Q 2. 注解元素为什么不能是 List<String>?想要"多个值"怎么办?

✓ 参考答案

注解元素必须能在编译期常量折叠并写入字节码的"常量表",List 是运行时对象,装不进常量表。多值用数组: `String[] roles() default {"user"}`,使用 `@RequiresAuth(roles = {"admin", "ops"})`。枚举与注解也可以作元素类型,组合出嵌套结构。

Q 3. @Inherited 对方法上的注解生效吗?为什么?

✓ 参考答案

不生效。@Inherited 只对类级注解(标注在 TYPE 上的)生效:子类继承父类的类注解。方法被"重写"而非"继承"——重写后方法本体是新的,注解跟着方法声明走,所以方法注解不会自动传给重写版。需要"方法级继承"得自己处理:反射时沿父类找同名方法读注解。

Q 4. 校验器里,一个字段同时标了 @NotEmpty 与 @Range,执行顺序怎么保证?有什么坑?

✓ 参考答案

getDeclaredFields 的顺序不保证——两个注解的执行顺序在反射层面不可控(而且数组里多字段整体顺序也不保证)。坑:先执行 Range 时 value 为 null,Number 判断会跳过;先执行 NotEmpty 则正常。生产框架(Hibernate Validator)有明确的组与顺序机制;自研校验器要在文档里声明"顺序不保证",或按注解的显式 order 值排序。

Q 5. Lombok 的 @Getter 在编译期生成 getter——为什么不能靠运行时反射实现同样的效果?

✓ 参考答案

运行时反射可以"模拟读取",但调用方写的是 `user.getName()`——编译期就需要这个方法存在,反射无法让编译通过。Lombok 在编译期往字节码里真正生成 getName 方法,所以源码里不写、编译产物里有。这就是 APT 的典型价值:生成"编译期就要存在"的代码;运行时反射解决"编译期不知道类型"的问题。两者互补,选型看需求的时机。

Q 6. 配置绑定器里,int 字段 parse 失败(配置写成了 "abc"),程序会怎样?生产应该怎么做?

✓ 参考答案

NumberFormatException(RuntimeException)直接抛出,启动失败——这其实是"好的失败":配置错误必须在启动时暴露,而不是运行到一半才炸。生产增强:① 错误消息带键名("server.port 配置无法解析: abc");② 汇总所有字段的解析错误一次性报告(别修一个重启一次);③ 支持类型转换器注册表(第11章),把 String→LocalDate 等也纳管。

章末实验题:表单校验器 + 配置中心

实验 12:注解驱动的表单校验与配置绑定

实验目标:编写两部分程序:① 完整字段校验器(支持 @NotNull/@Range/@Regex 与链式错误汇总);② 配置绑定器(支持环境变量覆盖)。综合运用本章全部知识点。

实验要求:

- 任务 1:定义三个注解 @NotNull、@Range、@Regex(正则校验),均带 message 默认值,Retention 为 RUNTIME(覆盖 §12.2-12.3 定义与元注解)
- 任务 2:校验器 validate(Object):扫描全部字段(含父类),收集所有错误到 List,不抛异常(覆盖 §12.4 运行时处理)
- 任务 3:支持继承:父类字段也校验(覆盖 §12.4 继承链,第11章技巧)
- 任务 4:定义表单类 RegisterForm(用户名/年龄/邮箱),打上三个注解,演示合法与非法两组数据(覆盖 §12.4 业务)
- 任务 5:配置绑定器 bind(Class<T>, Properties):支持 int/boolean/String,缺失时用注解默认值(覆盖 §12.6 配置绑定)
- 任务 6:绑定器支持"环境变量覆盖":System.getenv("APP_PORT") 存在且非空时优先(覆盖 §12.6 优先级链)
- 任务 7:main 里演示:非法表单的错误清单、合法表单通过、配置绑定结果(覆盖全部)

实验环境:JDK 17+。

第一步 定义注解(不写处理逻辑),再写 validate 的骨架:遍历字段打印注解信息

第二步 任务 3 复用第11章的 while (c != null) 继承链遍历

第三步 任务 6 的环境变量:Windows 下先 set APP_PORT=9000 再运行(或代码里给默认路径)

参考答案要点


```

import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
import java.lang.reflect.Field;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.ArrayList;
import java.util.List;
import java.util.Properties;
import java.util.regex.Pattern;

/** 注解驱动表单校验器 + 配置绑定 — 第12章综合实验参考答案
 * 覆盖:注解定义、元注解、运行时处理、继承链、校验执行、配置绑定、优先级链
 */
public class AnnotationLab {

    // ---- 任务 1:三个校验注解 ----
    @Retention(RetentionPolicy.RUNTIME)
    @Target(ElementType.FIELD)
    public @interface NotNull {
        String message() default "不能为空";
    }

    @Retention(RetentionPolicy.RUNTIME)
    @Target(ElementType.FIELD)
    public @interface Range {
        int min() default Integer.MIN_VALUE;

        int max() default Integer.MAX_VALUE;

        String message() default "数值越界";
    }

    @Retention(RetentionPolicy.RUNTIME)
    @Target(ElementType.FIELD)
    public @interface Regex {
        String pattern();

        String message() default "格式不合法";
    }

    // ---- 任务 2+3:校验器(含继承链)----
    static List<String> validate(Object obj) throws IllegalAccessException {
        List<String> errors = new ArrayList<>();
        for (Class<?> c = obj.getClass(); c != null; c = c.getSuperclass()) {
            for (Field f : c.getDeclaredFields()) {
                f.setAccessible(true);
                Object value = f.get(obj);
                String where = c.getSimpleName() + "." + f.getName();

                NotNull nn = f.getAnnotation(NotNull.class);
                if (nn != null && value == null) {
                    errors.add(where + " " + nn.message());
                }

                Range rg = f.getAnnotation(Range.class);
                if (rg != null && value instanceof Number num
                    && (num.intValue() < rg.min() || num.intValue() > rg.max())) {
                    errors.add(where + " " + rg.message() + "[" + rg.min()
                        + "," + rg.max() + "]");
                }

                Regex rx = f.getAnnotation(Regex.class);
                if (rx != null && value != null
                    && !Pattern.matches(rx.pattern(), String.valueOf(value))) {
                    errors.add(where + " " + rx.message());
                }
            }
        }
    }
}

```

```

    }
    }
    return errors;
}

// ---- 任务 4:表单类 ----
static class RegisterForm {
    @NotNull(message = "用户名必须填写")
    @Regex(pattern = "[a-zA-Z0-9_]{4,16}", message = "用户名需 4-16 位字母数字")
    private String username;

    @NotNull
    @Range(min = 18, max = 60, message = "年龄需在 18-60")
    private Integer age;

    @NotNull(message = "邮箱必须填写")
    @Regex(pattern = "^[^@\\s]+@[^@\\s]+\\.^[^@\\s]+$", message = "邮箱格式不正确")
    private String email;
}

// ---- 任务 5+6:配置绑定器(含环境变量覆盖)----
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
public @interface Prop {
    String key();

    String def() default "";

    String env() default "";    // 可选的环境变量名(优先级最高)
}

static class AppConfig {
    @Prop(key = "app.port", def = "8080", env = "APP_PORT")
    private int port;

    @Prop(key = "app.name", def = "未命名应用", env = "APP_NAME")
    private String name;

    @Prop(key = "app.debug", def = "false", env = "APP_DEBUG")
    private boolean debug;
}

static <T> T bind(Class<T> type, Properties props) throws Exception {
    T cfg = type.getDeclaredConstructor().newInstance();
    for (Field f : type.getDeclaredFields()) {
        Prop p = f.getAnnotation(Prop.class);
        if (p == null) continue;
        String raw = null;
        // 优先级:环境变量 > 配置文件 > 注解默认值
        if (!p.env().isEmpty()) {
            String env = System.getenv(p.env());
            if (env != null && !env.isBlank()) {
                raw = env;
            }
        }
        if (raw == null) {
            raw = props.getProperty(p.key(), p.def());
        }
        f.setAccessible(true);
        Class<?> ft = f.getType();
        if (ft == int.class || ft == Integer.class) {
            f.setInt(cfg, Integer.parseInt(raw.trim()));
        } else if (ft == boolean.class || ft == Boolean.class) {
            f.setBoolean(cfg, Boolean.parseBoolean(raw.trim()));
        } else {
            f.set(cfg, raw);
        }
    }
}

```

```

    return cfg;
}

public static void main(String[] args) throws Exception {
    System.out.println("== 任务 7a:表单校验 ==");
    RegisterForm bad = new RegisterForm();
    bad.username = "a"; // 太短
    bad.age = 17; // 太小
    bad.email = "not-an-email"; // 格式错
    System.out.println("非法表单错误清单:");
    validate(bad).forEach(e -> System.out.println("  " + e));

    RegisterForm good = new RegisterForm();
    good.username = "zhangsan";
    good.age = 25;
    good.email = "zs@example.com";
    System.out.println("合法表单错误数: " + validate(good).size()); // 0

    System.out.println("== 任务 7b:配置绑定 ==");
    Properties props = new Properties();
    Path f = Path.of("app.properties");
    if (Files.exists(f)) {
        try (var in = Files.newBufferedReader(f, StandardCharsets.UTF_8)) {
            props.load(in);
        }
    }
    props.setProperty("app.name", "教务管理"); // 配置文件提供 name
    // 若设置了环境变量 APP_PORT,它会覆盖 props 里的值
    AppConfig cfg = bind(AppConfig.class, props);
    System.out.println("port=" + cfg.port + " name=" + cfg.name
        + " debug=" + cfg.debug);
    System.out.println("实验完成。");
}
}

```

设计说明:任务 2 的 validate 故意"收集错误不抛异常"——表单校验需要一次给用户列全所有问题,抛第一个就返回是差体验;任务 3 的继承链与第 11 章实验同源,但这里每个字段标了"出处类"便于定位;任务 6 的优先级链(环境变量 > 配置 > 默认值)是生产配置系统的标准语义,Spring 的 PropertySource 顺序就是它;正则里的 \s 转义在 Java 字符串里要写 \\s,实验源码已正确处理。扩展方向:校验器支持"注解组"(表单分步校验)、绑定器加类型转换器注册表、用 APT 把校验在编译期生成(性能敏感场景)。

下一章预告:第 13 章 模块系统与构建工具。JPMS 模块化、Maven 依赖管理、jar 的生成与运行——把多个类组织成可交付的制品。

第13章 模块系统与构建工具

建议学时:4-6 小时 | 难度:★★★☆☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 理解模块系统(JPMS)的动机:封装、显式依赖、可伸缩
- 掌握 module-info.java:requires / exports / opens 的语义
- 掌握 jar 的打包、运行与 classpath / module-path 的区别
- 掌握 Maven 的项目结构与核心命令
- 掌握 pom.xml 的依赖声明与依赖传递机制
- 理解依赖冲突的产生与解决策略
- 能完成"编写 → 构建 → 打包 → 运行"的完整流程

💡 从 C 到 Java:把"工程"组织起来

前 12 章都在写"单文件程序",生产代码不是这样:成百上千个类、几十个第三方库、明确的版本要求。C 有 makefile/CMake + 静态库 + 头文件;Java 有 Maven/Gradle + jar + 模块系统。对照关系:jar ≈ .a/.so 库;Maven 仓库 ≈ 系统包管理器(apt);pom.xml ≈ CMakeLists + 依赖清单;module-info.java ≈ 头文件里的"导出了什么"。

模块系统(JPMS,JDK9)回答三个问题。① 我的包向谁公开?(exports,像头文件的 #include 白名单);② 我需要谁的包?(requires,像链接时的 -lxxx);③ 谁可以反射我的包?(opens,第11章 setAccessible 的开关)。在模块化世界里,封装从"类级别"提升到"模块级别"——不想让外部碰的包,根本不导出,编译期就挡在门外。

Maven 是现代 Java 的门槛。不学 Maven,连 Spring Boot 项目都跑不起来。核心心智:"坐标"定位依赖(groupId:artifactId:version),中央仓库下载,约定优于配置的目录结构,pom.xml 声明一切。C 程序员刚接触时最不适应的:不是"怎么编译"(IDE 代劳),而是"依赖从哪来、版本怎么定、冲突怎么办"。

构建生命周期是流水线。compile → test → package → install,一步一个产物,像 makefile 的 target 依赖,但约定固定、顺序固定。CI/CD(第14章提)就是在这条流水线上加步骤。

学习顺序:模块系统(§13.1-13.3)建立"模块化思维",jar 打包(§13.3),Maven(§13.4-13.6),依赖管理(§13.7)。

📦 前置知识自查

- 第11-12章:反射与注解(opens 指令直接管反射权限)
- C 的编译组织:makefile、静态/动态库、头文件、链接过程
- 命令行基本操作:javac/java 已熟练(第1章)

🚀 本章学习路线

1. **第一阶段(约 1 小时):**§13.1-13.2 模块化的动机与 module-info 三指令
2. **第二阶段(约 1.5 小时):**§13.3 手工 jar 打包与 module-path 运行,体会"模块边界"
3. **第三阶段(约 2 小时):**§13.4-13.7 Maven 结构与依赖管理,完成章末实验
4. **验收标准:**能写出 module-info 三指令;能解释 classpath 与 module-path;能读懂 pom.xml 并说出依赖冲突解法

本章知识导图

- §13.1 从 classpath 到模块系统
- §13.2 module-info.java:requires / exports / opens
- §13.3 jar 打包与运行:classpath vs module-path
- §13.4 Maven 项目结构与约定
- §13.5 pom.xml:坐标、依赖、插件
- §13.6 构建生命周期与命令
- §13.7 依赖管理:传递依赖与冲突

§13.1 从 classpath 到模块系统

13.1.1 传统 classpath 的三个问题

JDK9 之前,Java 把 jar 都塞进 classpath: ① 没有封装边界 (public 类谁都能用); ② 依赖关系隐式 (谁 require 谁只有运行期 ClassNotFoundException 才知道); ③ 无法裁剪运行时 (JDK 全量加载)。模块系统(Project Jigsaw)从 JDK9 开始根治:每个模块显式声明 requires/exports,JDK 自身也模块化。

概念	C 的世界	Java 的世界
代码单元	.c/.h 文件	.java 文件(包组织)
编译单元	目标文件 .o	.class 字节码
库	.a / .so / .dll	jar(zip 打包的 class)
导出接口	头文件 #include 白名单	module-info 的 exports(模块化时)
链接依赖	-lxxx / ld 脚本	requires(模块化)/ Maven 依赖
包管理器	apt / brew / vcpkg	Maven Central(仓库)
构建脚本	Makefile / CMakeLists	pom.xml(Maven)/ build.gradle

★ 重点:模块化是"可选"的——但理解它必须

绝大多数业务项目至今跑在 classpath 上(模块化要求严格、第三方库参差),但模块系统是 JDK 的地基、是 Spring/框架"开箱即用"的保证,也是面试与排障的常识。学习策略:会写 module-info(小模块项目)、会解释三指令、知道反射需要 opens——生产大项目里模块化交给架构师决策。

§13.2 module-info.java:三个指令

13.2.1 最小模块声明

示例 13-1 module-info 三指令

```
// 文件位置:模块根目录下的 module-info.java(与包目录同级)
module com.example.students {
    // requires:本模块需要谁(相当于 C 的 #include + 链接)
    requires java.logging;

    // exports:把哪个包公开给外部(相当于头文件公开 API)
    exports com.example.students.api;

    // opens:允许哪个包被反射访问(第11章 setAccessible 的前提)
    // 模块化世界里,反射别人需要显式授权
    opens com.example.students.internal;

    // 需要反射 JDK 内部包时(极少见,通常意味着设计问题):
    // requires jdk.unsupported;
}
```

示例 13-2 模块化的小项目

```
// 目录结构:
// mods/
//   com.example.calc/
//     module-info.java
//     com/example/calc/Calculator.java      (public,导出)
//     com/example/calc/InternalCache.java   (public,但不导出:外部不可见)

// ---- module-info.java ----
module com.example.calc {
    exports com.example.calc;      // 只导出这个包
}

// ---- Calculator.java ----
package com.example.calc;

public class Calculator {
    // 公开 API:外部可调用
    public static int add(int a, int b) { return a + b; }

    public static void clearCache() {
        InternalCache.reset();      // 模块内部可自由访问
    }
}

// ---- InternalCache.java(不导出:外部 import 直接编译错误)----
package com.example.calc;

public class InternalCache {
    static void reset() { System.out.println("缓存已清空"); }
}
```

⚠ 易错点 1:模块化后"外部用不了"是编译错误,不是运行错误

模块化世界里,不导出的包被外部 import 时 编译直接失败 (package is not visible)——比 classpath 时代"运行期 ClassNotFoundException"早得多,这正是模块化的价值:封装在编译期兑现。同理,反射未 opens 的包,setAccessible 抛 InaccessibleObjectException(JDK9+),这也是第11章"安全裂缝"的官方补丁。

§13.3 jar 打包与运行

13.3.1 手工流程:编译 → 打包 → 运行

示例 13-3 手工 jar 全流程(命令行)

```
# 1. 普通编译(未模块化):classpath 方式
javac -d out src/com/example/app/*.java
# 2. 打 jar(相当于把 .o 归档成静态库)
jar --create --file app.jar -C out .
# 3. 运行(classpath 方式)
java -cp app.jar com.example.app.Main

# ---- 模块化流程 ----
# 1. 编译模块(multi-release 语法按需;这里普通编译)
javac --module-path mods -d out \
    src/com.example.calc/module-info.java \
    src/com.example.calc/com/example/calc/*.java
# 2. 打模块 jar(带 module-info 的 jar 就是模块)
jar --create --file mods/com.example.calc.jar -C out .
# 3. 用 module-path 运行(--module-path 替代 -cp,module 名.主类)
java --module-path mods --module com.example.calc/com.example.calc.Calculator

# 查看模块描述:
jar --describe-module --file mods/com.example.calc.jar
```

★ 重点:classpath 与 module-path 的区别

-cp(classpath):一堆 jar 平铺,无边界、无依赖声明,类找到了算数;--module-path:模块化 jar 的集合,依赖由 module-info 声明,JVM 启动时验证。判断一个 jar 是"模块"还是"普通 jar":jar 根目录有没有 module-info.class。绝大多第三方库是普通 jar(自动模块名 = 文件名去版本),这就是"模块化推进慢"的根源——但语法与原理不变。

§13.4 Maven 项目结构

13.4.1 约定优于配置

示例 13-4 Maven 标准目录

```
myapp/
├── pom.xml                # 项目描述:坐标/依赖/插件(唯一需要手写的)
├── src/
│   ├── main/
│   │   ├── java/         # 生产代码
│   │   │   └── com/example/app/Main.java
│   │   └── resources/    # 配置文件(properties 等,自动进 classpath)
│   └── test/
│       └── java/         # 测试代码(第14章)
├── target/               # 构建产物(自动生成,不提交 git)
│   ├── classes/         # 编译后的 class
│   └── myapp-1.0.jar     # 打包产物
```

问题	C 的回答	Maven 的回答
目录怎么排	自己定,CMake 里配	约定:src/main/java 等,零配置
依赖从哪来	apt 装系统库 / 手动放	pom.xml 声明,中央仓库自动下载
怎么编译	make / cmake --build	mvn compile
怎么打包	make install(拷 .so)	mvn package(产出 jar)
测试怎么跑	自己写脚本	mvn test(JUnit 集成)

§13.5 pom.xml:坐标、依赖、插件

示例 13-5 最小可用 pom.xml

```

<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>

  <!-- 坐标:仓库里唯一标识(类比 apt 包名 + 版本) -->
  <groupId>com.example</groupId>          <!-- 组织(反向域名) -->
  <artifactId>myapp</artifactId>          <!-- 项目名 -->
  <version>1.0.0</version>                <!-- 版本号 -->
  <packaging>jar</packaging>

  <!-- 属性:统一版本,避免散落 -->
  <properties>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
  </properties>

  <!-- 依赖:坐标三段 + 可选 scope -->
  <dependencies>
    <dependency>
      <groupId>com.fasterxml.jackson.core</groupId>
      <artifactId>jackson-databind</artifactId>
      <version>2.17.1</version>
    </dependency>
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter</artifactId>
      <version>5.10.2</version>
      <scope>test</scope>                  <!-- 只进测试 -->
    </dependency>
  </dependencies>
</project>

```

⚠ 易错点 2:scope 的作用域

scope 决定依赖"活到哪一步":compile(默认,打包进 jar)、test(仅测试)、provided(编译需要但运行时由容器提供,如 servlet-api)、runtime(运行需要编译不需要)。经典错误:把测试库(如 JUnit)声明成 compile,产出的 jar 膨胀几 MB,还可能在生产触发类冲突。记忆:依赖属于谁的使用期,就标准的 scope。

§13.6 构建生命周期与命令

13.6.1 一条命令链,逐级递进

示例 13-6 核心命令速查

```

mvn clean           # 删除 target/ 目录(干净的起点)
mvn compile         # 只编译 src/main/java → target/classes
mvn test            # 编译 + 运行测试(JUnit)
mvn package         # 编译 + 测试 + 打包 jar(到 target/)
mvn install         # package + 安装进本地仓库(~/.m2),供其他项目依赖
mvn dependency:tree # 查看依赖树(排查冲突的第一工具)
mvn help:effective-pom # 看"实际生效"的 pom(继承父 pom 之后)

# 跳过测试(不推荐,紧急情况下才用):
mvn package -DskipTests

# 指定 Java 版本:
mvn package -Dmaven.compiler.release=17

```


🚀 工程建议:Maven 的三个日常习惯

1. 每次提交前 `mvn test`:测试不过的代码不进仓库(CI 也会拦,第14章)
2. `target/` 不提交 `git .:gitignore` 写死,产物人人可重建
3. 依赖版本统一管理:多模块项目用 `dependencyManagement` 或 BOM(bill of materials)集中管版本,别让各模块各写各的

§13.7 依赖管理:传递依赖与冲突

13.7.1 传递依赖与"最近优先"

你的项目依赖 A,A 依赖 B——B 自动进入你的 classpath,这叫**传递依赖**(像 apt 自动装依赖包)。冲突:A 要 B 1.0,C 要 B 2.0,谁赢? **Maven 默认"最近优先 + 先声明优先"**:依赖树中离根最近的版本胜出;距离相同先声明者胜出。这个"自动决策"经常选错,所以 `mvn dependency:tree` 是每个 Java 工程师的日常工具。

示例 13-7 依赖冲突排查实录

```
# 症状:NoSuchMethodError / NoClassDefFoundError(编译正常、运行爆炸)
# 典型原因:classpath 上同时存在库的两个版本,方法签名对不上

# 1. 看依赖树,找出同一个库的不同版本:
mvn dependency:tree
# [INFO] com.example:myapp:jar:1.0.0
# +- com.example:lib-a:jar:1.0
# |   \- com.fasterxml.jackson.core:jackson-databind:jar:2.17.1
# \- com.example:lib-c:jar:1.0
#     \- com.fasterxml.jackson.core:jackson-databind:jar:2.16.0 ← 冲突!

# 2. 解法一:显式声明想要的版本(最近优先,你的声明必赢)
# 在 pom.xml 的 dependencies 里直接加:
# <dependency>
#   <groupId>com.fasterxml.jackson.core</groupId>
#   <artifactId>jackson-databind</artifactId>
#   <version>2.17.1</version>
# </dependency>

# 3. 解法二:排除某个传递依赖(明确不要它)
# <dependency>
#   <groupId>com.example</groupId>
#   <artifactId>lib-c</artifactId>
#   <version>1.0</version>
#   <exclusions>
#     <exclusion>
#       <groupId>com.fasterxml.jackson.core</groupId>
#       <artifactId>jackson-databind</artifactId>
#     </exclusion>
#   </exclusions>
# </dependency>
```

⚠ 易错点 3:NoSuchMethodError 是"版本冲突"的代名词

编译期一切正常,运行期抛 `NoSuchMethodError / NoClassDefFoundError / AbstractMethodError`:先别怀疑代码,十有八九是 classpath 上同一库的两个版本并存。排查顺序:① `mvn dependency:tree` 找重复;② 显式声明正确版本;③ 排除老版本来源;④ 再不行查 jar 内容 `jar tf`。C 的"链接错版本 .so"是同一个坑,只是 Java 的报错更隐晦(运行时才爆)。

本章速查表

主题	要点
模块化动机	封装边界 / 显式依赖 / 运行时可裁剪;JDK9+ 的 Jigsaw
requires	声明"我需要谁的模块";编译期检查,缺失立即报错
exports	公开哪些包给外部;没导出的包外部 import 编译失败
opens	允许哪些包被反射;反射未 opens 抛 InaccessibleObjectException
jar	zip 打包 class:jar --create --file x.jar -C out .;module-info 在根目录
classpath vs module-path	-cp 平铺无边界;--module-path 模块化,启动时验证依赖
Maven 目录	pom.xml + src/main/java + src/test/java + target/(产物)
坐标	groupId:artifactId:version 三段定位;仓库: Maven Central
scope	compile(默认)/ test / provided / runtime;选错 jar 膨胀或运行缺类
生命周期	clean → compile → test → package → install 逐级递进
传递依赖	A 依赖 B 自动带入;冲突"最近优先+先声明优先"
冲突排查	mvn dependency:tree → 显式版本 → 排除;NoSuchMethodError = 版本冲突信号

最小必会清单

- 能说出模块化解决的三个问题,并解释 exports/requires/opens
- 能写出 module-info.java 的最小声明并解释"编译期封装"
- 能用命令行完成 javac → jar → java 的完整流程
- 能说出 classpath 与 module-path 的区别
- 能画出 Maven 标准目录并解释每个目录的用途
- 能读懂 pom.xml:坐标、依赖、scope、properties
- 能说出 mvn 五个核心命令与 dependency:tree 的用途
- 能解释传递依赖与"最近优先"规则,会排查 NoSuchMethodError

自测题

Q 1. 模块 A 的 module-info 只写了 exports com.a.api,但代码里引用了 com.a.internal 的类,外部模块能编译通过吗?

✓ 参考答案

不能。外部模块 import com.a.internal 时编译报 "package is not visible"。即使两个类都是 public——模块边界比访问修饰符更严:public 决定"类内可见性上限",exports 决定"模块外可见性上限",取交集。这就是模块化时代的封装模型。

Q 2. 模块化项目里,反射代码访问另一个模块的类抛 InaccessibleObjectException,怎么解决?顺序是什么?

✓ 参考答案

① 最佳:找提供方,让它在自己模块的 module-info 加 opens com.example.internal; (或 exports,如果本来就该公开);② 次优:自己写"传递 opens"——被反射的包在提供方必须 open;③ 绕行:反射方的模块加 --add-opens JVM 参数 (临时方案,生产慎用)。永远先问"这个反射是否必要",再谈 opens——反射他人内部是最后手段。

Q 3. -cp 与 --module-path 的区别?普通 jar 能在 --module-path 上运行吗?

✓ 参考答案

-cp:平铺的 jar 集合,运行时按需查找类,无依赖声明。--module-path:模块 jar 集合,JVM 启动时解析 module-info 验证依赖关系。普通 jar(无 module-info)可以放 module-path——它被当成"自动模块",模块名取 jar 文件名(去版本号)。所以"模块化迁移"可以渐进:先把普通 jar 放 module-path 观察,再逐个补 module-info。

Q 4. 你的项目同时被 lib-a(依赖 Jackson 2.17)与 lib-c(依赖 Jackson 2.16)依赖,Maven 会选哪个版本?为什么这可能是错的?

✓ 参考答案

看依赖树深度:若两者都是直接传递一层,按声明顺序先声明者胜;若 lib-a 在更浅的层级,2.17 胜。自动决策可能错:比如 lib-c 只与 2.16 兼容,或更浅层级反而不是"最想要"的版本。所以冲突必须人工复核:显式声明目标版本,必要时排除,并跑完整测试验证兼容性。

Q 5. 运行期抛 NoSuchMethodError,代码却编译通过。为什么?第一步做什么?

✓ 参考答案

编译用了一版 API,运行 classpath 上却是另一版(旧版没有该方法,或签名不同)。这是依赖版本冲突的典型信号,不是你的代码 bug。第一步:mvn dependency:tree 找同 artifactId 的多版本;然后显式锁定正确版本,排除错误来源,重跑测试验证。

Q 6. 写一段文字解释 Maven 生命周期:为什么 mvn package 会自动编译、会自动跑测试?

✓ 参考答案

Maven 生命周期是固定流水线:validate → compile → test → package → verify → install,后一阶段自动包含前面所有阶段。mvn package = 执行到 package 为止,所以编译与测试是必经环节。这也解释了为什么"只想打包不想测试"要显式 -DskipTests——默认行为就是全流程。这与 makefile 的"目标依赖"等价,只是顺序被约定死、无法打乱。

章末实验题:模块化计算器 + Maven 构建

实验 13:模块化计算器与 Maven 构建

实验目标:完成两个递进任务:① 手工构建一个模块化计算器(module-info + jar + module-path 运行);② 用 Maven 构建一个带第三方依赖(Jackson 解析 JSON 配置)的项目。综合运用本章全部知识点。

实验要求:

- 任务 1:创建模块 com.example.calc,导出 api 包,内部实现包不导出(覆盖 §13.1-13.2 exports/requires)
- 任务 2:模块内用 java.util.logging 记录计算日志,module-info 加 requires java.logging(覆盖 §13.2 requires)
- 任务 3:手动执行 javac --module-path → jar --create → java --module-path 三条命令,输出计算器结果(覆盖 §13.3 打包运行)
- 任务 4:写一个"非法访问验证":在模块外 import 未导出包,观察编译错误(覆盖 §13.2 封装)
- 任务 5:创建 Maven 项目结构(手写 pom.xml 与目录),声明 jackson-databind 依赖,代码从 JSON 文件读配置(覆盖 §13.4-13.5)
- 任务 6:执行 mvn package 产出 jar,用 java -cp target/classes:依赖路径 或 mvn exec 运行,输出计算结果(覆盖 §13.6 构建)
- 任务 7:运行 mvn dependency:tree,解释传递依赖(覆盖 §13.7 依赖)

实验环境:JDK 17+(自带 jar 工具);Maven 3.8+(若未安装:任务 5-7 可用"手工下载 jar + javac -cp"替代,命令差异在参考答案注释中说明)。

第一步 做任务 1-4(纯命令行,无外部依赖),成功后再进 Maven 部分

第二步 务 3 的目录结构:mods/com.example.calc/{module-info.java, com/example/calc/...}

第三步 务 5 的 JSON 文件:config.json 放 src/main/resources,读类用 getResourceAsStream

参考答案要点

```
// ===== 任务 1-4:模块化计算器(手工命令行) =====
// 目录:mods/com.example.calc/
//  module-info.java
//  com/example/calc/Calculator.java
//  com/example/calc/LogUtil.java      (不导出)

// ---- module-info.java ----
module com.example.calc {
    requires java.logging;           // 用 JDK 的日志模块
    exports com.example.calc;       // 只导出公共包
}

// ---- Calculator.java ----
package com.example.calc;

import java.util.logging.Logger;

public class Calculator {
    private static final Logger LOG = Logger.getLogger("calc");

    public static int add(int a, int b) {
        LOG.info("计算: " + a + " + " + b);
        return a + b;
    }

    public static int sub(int a, int b) {
        LOG.info("计算: " + a + " - " + b);
        return a - b;
    }
}

// ---- LogUtil.java(不导出:包 com.example.calc 整体导出了?)
// 注意:exports com.example.calc 会导出该包全部 public 类,
// 想"部分隐藏"应拆包—此处 LogUtil 用包私有访问演示内部依赖:
package com.example.calc;

class LogUtil {                      // 包私有:模块内外都不可见
    static String tag(String s) { return "[calc] " + s; }
}

// ---- 主程序(模块外,只依赖导出 API)----
// 目录:mods/app/com/example/app/Main.java
package com.example.app;

import com.example.calc.Calculator;    // 合法:导出包
// import com.example.calc.LogUtil;    // 编译错误:不可见(任务 4 验证)

public class Main {
    public static void main(String[] args) {
        System.out.println("3 + 4 = " + Calculator.add(3, 4));
        System.out.println("9 - 2 = " + Calculator.sub(9, 2));
    }
}

// ---- 任务 3:命令行三条命令 ----
// javac --module-path mods -d out/calc mods/com.example.calc/module-info.java \
//      mods/com.example.calc/com/example/calc/*.java
// jar --create --file mods/com.example.calc.jar -C out/calc .
// javac --module-path mods -d out/app mods/app/com/example/app/Main.java
// java --module-path mods --module app/com.example.app.Main

// ===== 任务 5-7:Maven + Jackson(任务 5-6) =====
// pom.xml(见示例 13-5,加 jackson-databind 2.17.1)
// src/main/resources/config.json:
// { "a": 10, "b": 5 }

// ---- JsonConfigApp.java ----
package com.example.app;
```

```

import com.fasterxml.jackson.databind.JsonNode;
import com.fasterxml.jackson.databind.ObjectMapper;

import java.io.InputStream;

public class JsonConfigApp {
    public static void main(String[] args) throws Exception {
        // 从 classpath 读 resources 下的配置文件
        ObjectMapper mapper = new ObjectMapper();
        try (InputStream in = JsonConfigApp.class
            .getResourceAsStream("/config.json")) {
            JsonNode cfg = mapper.readTree(in);
            int a = cfg.get("a").asInt();
            int b = cfg.get("b").asInt();
            System.out.println(a + " + " + b + " = " + (a + b));
            System.out.println(a + " * " + b + " = " + (a * b));
        }
    }
}

// ---- 任务 6:构建与运行 ----
// mvn package
// java -cp target/classes:$HOME/.m2/repository/com/fasterxml/jackson/... \
//     com.example.app.JsonConfigApp
// (Windows 用分号分隔 classpath;Maven 的 exec 插件可简化:
// mvn exec:java -Dexec.mainClass=com.example.app.JsonConfigApp)

// ---- 任务 7:依赖树(节选) ----
// mvn dependency:tree
// jackson-databind → jackson-core + jackson-annotations(传递依赖)
// 若多个库各带不同版本 jackson,按"最近优先"规则解析,见 §13.7

```

设计说明:任务 2 的 `java.logging` 是 JDK 自带模块——requires 它演示了"依赖 JDK 模块"的标准写法;任务 4 用"取消注释即编译失败"让学生亲手验证模块封装;任务 5-6 的 Jackson 是生产最常用的 JSON 库,`getResourceAsStream` 读 `resources` 是标准姿势(注意 Windows 的 `classpath` 分隔符是分号)。没有 Maven 的环境,任务 5-7 可改为:从 Maven 中央仓库手动下载 `jackson-databind/core/annotations` 三个 jar,用 `javac -cp` 与 `java -cp` 完成等价流程。扩展方向:给模块加 `opens` 演示反射授权(与第11章衔接)、`pom` 加 `surefire` 插件跑 JUnit 测试(与第14章衔接)。

下一章预告:第14章 类型系统进阶。record、sealed 类、模式匹配、Optional——现代 Java 的类型安全武器库,也是全书的收官章。

第14章 类型系统进阶

建议学时:5-7 小时 | 难度:★★★★☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 掌握 record:不可变数据载体的定义、紧凑构造器、组件访问
- 掌握 sealed 类:受限继承与 permits 的语义
- 掌握 switch 表达式与模式匹配:类型模式、守卫、穷尽性
- 理解"代数数据类型":sealed + record + 模式匹配的组合拳
- 掌握 Optional 的核心方法与链式编程
- 能对照 C 的 union/enum/结构体,说出类型安全上的进步
- 能综合运用上述特性写出无强制转换、无 null 检查的健壮代码

💡 从 C 到 Java:让"类型"替你挡住错误

前 13 章教你怎么写 Java;这一章教你怎么用类型设计程序。C 的类型系统是"宽松"的:union 里存什么全凭自觉,enum 会被悄悄转成 int,void* 强转后编译器闭嘴。Java 的现代类型系统反过来:把"约定"写进类型,让编译器在编译期替你检查。

record 是"值"的声明。C 里定义"一对坐标",你得手写 struct、初始化函数、打印函数;Java 的 `record Point(int x, int y)` 一行生成全部:构造器、getter、equals、hashCode、toString——而且是不可变的。不可变 = 线程安全(第9章)、可作 Map 的键(第10章)、无别名陷阱(第5章)。

sealed 类把"可能的取值"写进声明。C 的 enum 告诉你"有几种值",但拿它做状态机时,switch 忘了一个 case 编译器不吭声。sealed + switch 表达式让 编译器强制穷尽所有分支——漏一个 case 编译失败,这是 C 永远给不了的安全感。

Optional 把"可能没有"写进类型。C 的返回 -1/0/NULL 是注释约定,传错了没人拦;Java 的 `Optional<T>` 把"可能缺失"变成类型的一部分,逼你处理不存在的情况。

为什么重要:这三个特性(record/sealed/模式匹配)合称"代数数据类型"(Algebraic Data Type,ADT)在 Java 17 的落地,是函数式编程思想进入主流 Java 的标志。看懂它们,Java 代码在你眼里就分成两种:处理"数据"的代码与处理"行为"的代码,结构清晰得多。

学习顺序:record(§14.1)→ sealed(§14.2)→ switch 表达式与模式匹配(§14.3)→ 组合成 ADT(§14.4)→ Optional(§14.5)。

📦 前置知识自查

- 第6章:继承、多态、抽象类(sealed 是继承的约束)
- 第10章:泛型(Optional<T>、record 泛型)
- 第9章:不可变对象与线程安全(record 的动机)
- C 语言:struct、union、enum、typedef 的习惯用法

🚀 本章学习路线

1. **第一阶段(约 1.5 小时):**§14.1 record,动手写 5 个 record,比较与手写 POJO 的差别
2. **第二阶段(约 1.5 小时):**§14.2 sealed + §14.3 switch 表达式与模式匹配,理解穷尽性检查
3. **第三阶段(约 2 小时):**§14.4 ADT 组合拳 + §14.5 Optional,完成章末实验(JSON 求值器)
4. **验收标准:**能用 sealed+record+switch 表达式写出无强制转换、无 null 判断、编译期穷尽的代码;能解释 Optional 链的每一步

本章知识导图

- §14.1 record:不可变数据载体
- §14.2 sealed 类:受限继承
- §14.3 switch 表达式与模式匹配
- §14.4 代数数据类型:sealed + record + 模式匹配
- §14.5 Optional:显式的"可能缺失"

§14.1 record:不可变数据载体

14.1.1 从冗长 POJO 到一行声明

C 程序员早已习惯写 struct + 一堆函数;Java 老式写法是 POJO(Plain Old Java Object):私有字段 + getter/setter + equals + hashCode + toString,六十行模板代码只为装两个数。record 把它压缩成一行,并且 免费送你正确性:字段 final、类 final、equals/hashCode/toString 按组件生成。

示例 14-1 record 基本用法

```
// 声明:Point 是一个 record,组件是 x 和 y
public record Point(int x, int y) {
    // 紧凑构造器:校验逻辑写在这里(不能给字段赋值)
    public Point {
        if (x < 0 || y < 0) {
            throw new IllegalArgumentException("坐标不能为负: " + x + ", " + y);
        }
    }

    // 额外方法照常加
    public double distance(Point o) {
        int dx = x - o.x, dy = y - o.y;
        return Math.sqrt(dx * dx + dy * dy);
    }
}

// 使用
public class RecordDemo {
    public static void main(String[] args) {
        Point p = new Point(3, 4);           // 全参构造器自动生成
        System.out.println(p.x());           // getter 不带 get 前缀:x()
        System.out.println(p);               // toString: Point[x=3, y=4]
        Point q = new Point(3, 4);
        System.out.println(p.equals(q));      // true:按组件比较
        System.out.println(p.hashCode() == q.hashCode()); // true
        // new Point(-1, 0) 抛 IllegalArgumentException(校验生效)
    }
}
```

示例 14-2 record 与类组合:局部 record


```
import java.util.ArrayList;
import java.util.List;

public class LocalRecordDemo {
    public static void main(String[] args) {
        // 局部 record:方法内部声明,立刻用
        record Price(int yuan, int fen) {
            int totalFen() { return yuan * 100 + fen; }
        }

        List<Price> prices = new ArrayList<>();
        prices.add(new Price(3, 50));
        prices.add(new Price(2, 99));

        int sum = 0;
        for (Price pr : prices) {
            sum += pr.totalFen();
        }
        System.out.println("总价: " + sum + " 分 = " + (sum / 100) + " 元 "
            + (sum % 100) + " 分");
    }
}
```

★ 重点:record 是不可变的"值",不是可变"实体"

record 适合"数据快照":坐标、金额、订单行、配置项。不适合需要演化状态的实体(计数器、连接池)——那种还是用普通类。判断标准: "这个对象换字段值是否代表另一个对象?"若是,用 record;若否,用类。C 的 struct 是可变副本,record 是 immutable value——多线程下免费安全(呼应第9章)。

⚠ 易错点 1:record 的字段是 final,没有 setter,组件不可重新赋值

record 隐含的"全字段构造器"是唯一的赋值通道;紧凑构造器里给 x 赋值会编译错误。想派生新值,用"with"方法返回新 record: `public Point withX(int nx) { return new Point(nx, y); }`——这正是不变式编程的姿势。另外注意:record 不能 extends 其他类(已隐式 extends `java.lang.Record`),但可以实现接口。

§14.2 sealed 类:受限继承

14.2.1 谁可以继承我,白纸黑字

普通继承是开放的:任何类都能 extends 你的类。sealed 把继承名单写进声明: "只有这些类能继承我,且它们必须同模块/同包,且必须标注 final/sealed/non-sealed"。C 语言里最接近的心智是 enum + 显式分发,但 sealed 是编译期强制的。

示例 14-3 sealed 类:形状体系

```
// 父类封闭:只允许 Circle、Rect、Triangle 继承
public sealed interface Shape
    permits Circle, Rect, Triangle {
    double area();
}

// 直接子类必须标注 final / sealed / non-sealed 之一
public final class Circle implements Shape {
    private final double r;
    public Circle(double r) { this.r = r; }
    public double area() { return Math.PI * r * r; }
}

public final class Rect implements Shape {
    private final double w, h;
    public Rect(double w, double h) { this.w = w; this.h = h; }
    public double area() { return w * h; }
}

// non-sealed:继续开放给第三方
public non-sealed class Triangle implements Shape {
    private final double base, height;
    public Triangle(double base, double height) {
        this.base = base; this.height = height;
    }
    public double area() { return base * height / 2; }
}
```

示例 14-4 精简版:sealed class 语法与“同一文件”技巧

```
// sealed class 与 interface 语法一致;permits 列表可省略(同文件时)
// 同一源文件中声明时,permits 可以省:
public sealed class Animal {
    // ...
}

final class Dog extends Animal { }           // 同文件:自动认可
final class Cat extends Animal { }

// 若子类在别的包,必须写 permits 且子类必须能访问父类:
// public sealed class Animal permits zoo.Dog, zoo.Cat { }
```

⚠ 易错点 2:sealed 的三个强制性条件

① 子类必须出现在 permits 列表(或与 sealed 类同文件);② 每个直接子类必须标 final / sealed / non-sealed 之一——忘记标注直接编译错误;③ 子类与父类须同模块,或同包(跨包需 permits 且子类可访问)。**non-sealed 是“开个口子”给第三方扩展,一开口,穷尽性检查就失效——生产代码谨慎使用。**

§14.3 switch 表达式与模式匹配

14.3.1 switch 表达式:有值、无 fall-through

老式 switch 是语句(C 程序员的老朋友,还有 break 陷阱);switch 表达式是“有返回值的表达式”,**每个分支自然结束、无需 break、类型统一,漏分支编译错误。**

示例 14-5 switch 表达式基础

```

public class SwitchExprDemo {
    static String dayName(int day) {
        // 表达式形式:整个 switch 有值;箭头分支无需 break
        return switch (day) {
            case 1 -> "星期一";
            case 2 -> "星期二";
            case 3 -> "星期三";
            case 4 -> "星期四";
            case 5 -> "星期五";
            case 6, 7 -> "周末";
            default -> "非法日期: " + day;    // 必须给出兜底
        };
    }

    // yield:多语句分支返回
    static String describe(int day) {
        return switch (day) {
            case 1 -> {
                String s = "一";
                yield "周" + s;                // yield 带值返回
            }
            default -> "其他";
        };
    }

    public static void main(String[] args) {
        System.out.println(dayName(3));    // 星期三
        System.out.println(dayName(7));    // 周末
        System.out.println(describe(1));    // 周一
    }
}

```

14.3.2 模式匹配:对类型做 switch,而不是对值

示例 14-6 instanceof 模式匹配与 switch 模式标签

```

// instanceof 模式匹配:类型判断 + 自动强转,一步到位
public static String describe(Object o) {
    if (o instanceof String s) {
        return "字符串,长度 " + s.length();
    } else if (o instanceof Integer i) {
        return "整数 " + i;
    } else {
        return "其他类型";
    }
}

// switch 模式标签:对"类型"分支
public static String classify(Object o) {
    return switch (o) {
        case String s -> "字符串: " + s;
        case Integer i -> "整数: " + i;
        case Long l -> "长整: " + l;
        case null -> "是 null";                // 显式处理 null
        default -> "未知类型";
    };
}

// 守卫(guard):模式 + 条件过滤
public static String scoreLevel(Object o) {
    return switch (o) {
        case Integer i when i >= 90 -> "优秀";
        case Integer i when i >= 60 -> "及格";
        case Integer i -> "不及格";
        default -> "不是成绩";
    };
}

```

★ 重点:模式匹配消灭"强转 + null 判断"

老代码 `if (o instanceof String) { String s = (String) o; ... }` 三步;新代码一步 `case String s`。且: `switch` 表达式对 `sealed` 类型穷尽时,可以省略 `default`——编译器知道你列全了。C 的强转是"我说了算",Java 的模式匹配是"编译器验证我说得对"。

§14.4 代数数据类型:sealed + record + 模式匹配

14.4.1 组合拳:安全的"有穷取值"建模

三个特性合起来,表达力质变: `sealed` 限定取值集合(枚举维度),`record` 承载每个取值的载荷(积维度),`switch` 表达式穷尽分发(处理维度)。C 里这个模型是 `union + enum + 手写 switch`——漏 `case`、转错字段都是运行时炸弹;Java 版本全部编译期检查。

示例 14-7 表达式树:ADT 的经典例子

```
// 表达式:数字 / 加法 / 乘法,三种"变体",每个带自己的数据
public sealed interface Expr permits Num, Add, Mul {
}

public record Num(int value) implements Expr { }

public record Add(Expr left, Expr right) implements Expr { }

public record Mul(Expr left, Expr right) implements Expr { }

// 求值器:穷尽分发,编译器保证不漏分支
public final class Evaluator {
    public static int eval(Expr e) {
        return switch (e) {
            case Num n -> n.value(); // 拆 record
            case Add a -> eval(a.left()) + eval(a.right());
            case Mul m -> eval(m.left()) * eval(m.right());
            // 没有 default!编译器知道 sealed 只有三种
        };
    }
}

public static void main(String[] args) {
    // (1 + 2) * 3
    Expr e = new Mul(new Add(new Num(1), new Num(2)), new Num(3));
    System.out.println(eval(e)); // 9
}
```

维度	C 的写法	Java 17 的写法	安全差异
变体	enum + union	sealed interface	漏变体:编译报错
载荷	union 成员(自觉)	record 组件(强制)	读错字段:编译报错
分发	switch + 强转	switch 模式匹配	类型不符:分支不进,不是崩溃
缺失值	NULL / -1 约定	Optional<T>	不处理则编译警告/必须处理

§14.5 Optional:显式的"可能缺失"

14.5.1 为什么消灭 null

C 的 NULL 是"约定";Java 的 null 是十亿美金错误(第5章 NPE)。 `Optional<T>` 是一个包装盒:里面要么有 T,要么空——"可能没有"从注释变成类型。注意:Optional 是给"返回值"用的,不是给字段和参数用的。

示例 14-8 Optional 核心链式操作

```
import java.util.Optional;

public class OptionalDemo {
    // 数据库风格的查找:可能没有用户
    static Optional<String> findNickname(int id) {
        if (id == 1) {
            return Optional.of("小明");
        }
        return Optional.empty();
    }

    public static void main(String[] args) {
        Optional<String> nick = findNickname(1);

        // orElse:没有就给默认值
        System.out.println(nick.orElse("匿名用户"));

        // orElseGet:延迟求值(推荐,不计算就不用)
        System.out.println(findNickname(2).orElseGet(() -> "游客" + System.nanoTime()));

        // orElseThrow:没有就抛异常(语义最清晰)
        String name = findNickname(1)
            .orElseThrow(() -> new IllegalStateException("用户不存在"));

        // map:有值才转换,没有还是空(链式)
        Optional<Integer> len = nick.map(String::length);

        // filter:有值且满足条件才保留
        Optional<String> longName = nick.filter(s -> s.length() > 3);

        // flatMap:转换结果本身是 Optional,避免嵌套
        Optional<String> upper = nick.flatMap(
            s -> Optional.of(s.toUpperCase()));

        // ifPresent:有值才做副作用
        nick.ifPresent(s -> System.out.println("欢迎回来, " + s));

        // 三件套终端操作:map/filter/flatMap 是中间,orElse* 是终端
        String result = findNickname(99)
            .map(String::toUpperCase)
            .filter(s -> !s.isEmpty())
            .orElse("N/A");
        System.out.println(result);           // N/A
    }
}
```

⚠ 易错点 3:Optional 的四个使用红线

① 不做字段(序列化、性能、语义都说不通);② 不做方法参数(调用方仍然可能传 null,还多一层包装——不如直接判空);③ 不用 get() 裸取(空盒子 get 抛 NoSuchElementException,与 NPE 无异,用 orElse/orElseThrow);④ 不把集合包进 Optional(空集合本身就能表达"没有",Optional.of(List.of()) 是双重包装)。记住:Optional 解决的是"单个值可能缺失",不是通用 null 替代品。

🚀 工程建议:现代 Java 代码风格速览

1. 数据用 record,行为用类方法:DTO、返回值、事件载荷一律 record
2. 状态机与协议用 sealed + switch 表达式:漏分支编译不过,重构时编译器当哨兵
3. 查找类 API 返回 Optional<T>:调用方自然写出 orElse 链
4. 警惕 non-sealed:开了口子,穷尽性就没了,审慎

本章速查表

主题	要点
record	不可变数据载体;一行生成构造器/getter(x())/equals/hashCode/toString
紧凑构造器	校验逻辑所在;不能给组件赋值
record 限制	final、不可继承(隐式 extends Record)、可实现接口、可泛型
sealed	permit 列表限定继承;子类须 final/sealed/non-sealed
non-sealed	重新开放继承;代价是穷尽性失效
switch 表达式	箭头分支、无 break、有值;default 或穷尽时省略
yield	多语句分支用 yield 返回值
模式匹配	instanceof 模式 / switch 模式标签;自动绑定变量;免强转
守卫	case X x when 条件 -> 结果
ADT 组合	sealed(变体) + record(载荷) + switch(分发) = 编译期穷尽
Optional 中间操作	map / filter / flatMap;有值才继续,空则保持空
Optional 终端操作	orElse / orElseGet / orElseThrow / ifPresent
Optional 红线	不做字段、不做参数、不 get()、不包集合

最小必会清单

- 能写 record 声明、紧凑构造器、with 方法
- 能写 sealed interface + 三类子类标注
- 能写 switch 表达式与模式标签、守卫、yield
- 能说出"漏分支编译错误"依赖的两个前提:sealed + 无 non-sealed
- 能用 sealed+record+switch 写出表达式树求值器
- 能写出 Optional 的 map/filter/flatMap 链与三种 orElse 终端
- 能对照 C 的 union/enum/NULL 说明类型安全差异

自测题

Q 1. record 与"手写 POJO + 手写 equals/hashCode"相比,除了代码量,最本质的差异是什么?

✅ 参考答案

不可变性。record 字段全 final,类全 final,没有 setter,equals/hashCode 由编译器按组件生成且必然一致。手写 POJO 很容易写出"字段改了但 equals 没更新""hashCode 用了可变字段"这类一致性 bug(第10章说过的哈希契约),record 从机制上消灭了整类错误。不可变还带来线程安全与可作 Map 键的免费红利。

Q 2. 下面的代码哪里错了?

```
public sealed class A { }  
public class B extends A { }
```

✅ 参考答案

两处错误:① A 的 permits 未列出 B(且 B 不在同一源文件),编译器不认 B 是合法子类;② 即使列出,直接子类必须标注 final/sealed/non-sealed,B 什么都没标,直接编译失败。修正:同文件声明可省 permits;跨文件则 `sealed class A`

permits B 且 final class B extends A。

Q 3. 为什么"sealed + switch 表达式"可以不写 default?什么情况下又必须写?

✓ 参考答案

switch 表达式要求穷尽。当 switch 的目标类型是 sealed 且 permits 列表完备、所有直接子类都被模式覆盖、且没有 non-sealed 子类时,编译器能静态证明穷尽,于是允许省略 default。一旦有任何 non-sealed 分支(或目标不是 sealed,如 Object),编译器无法证明穷尽,必须写 default(或 null 分支)。这是"类型系统穷尽性"的直观体现。

Q 4. Optional 的空盒子上调用 map 会发生什么?flatMap 与 map 的区别?

✓ 参考答案

空盒子上调用 map:不执行函数,直接返回空 Optional(短路,链式安全)。map 的函数返回普通值,Optional 自动包装;flatMap 的函数返回 Optional 本身,用于"函数内部已是 Optional 查找"的场景,避免产生 Optional<Optional<T>> 嵌套。举例:getUser(id).flatMap(u -> findOrder(u.orderId()))——findOrder 返回 Optional,若用 map 会得到嵌套。

Q 5. 用 ADT 设计一个"支付结果":成功(带金额)、失败(带原因)。写出声明与处理代码。

✓ 参考答案

```
public sealed interface PayResult permits Ok, Fail { }

public record Ok(int amount) implements PayResult { }

public record Fail(String reason) implements PayResult { }

public static void handle(PayResult r) {
    switch (r) {
        case Ok ok -> System.out.println("支付成功: " + ok.amount() + " 元");
        case Fail f -> System.out.println("支付失败: " + f.reason());
    }
}
```

关键点:sealed 保证只有两种结果;switch 无 default 但编译通过(穷尽);record 给每个结果配载荷;新增第三种结果时,编译器强制所有 switch 补分支——"修改一处,编译器帮你找全所有处理点"。

Q 6. 一个方法要从配置里读端口号,可能没有配置、可能非法。用 Optional 怎么设计?

✓ 参考答案

```
public Optional<Integer> portFrom(String cfg) {
    if (cfg == null) {
        return Optional.empty(); // 没有配置
    }
    try {
        int p = Integer.parseInt(cfg);
        return (p >= 1 && p <= 65535) ? Optional.of(p) : Optional.empty();
    } catch (NumberFormatException e) {
        return Optional.empty(); // 非法配置
    }
}

// 调用方:
int port = portFrom(config)
    .orElse(8080); // 兜底默认端口
```

要点:用 Optional 表达"可能没有",调用方三选一——默认值(orElse)、抛异常(orElseThrow)、忽略(ifPresent)。不要返回 null,不要抛异常表达"没有"(那是异常,不是缺失)。

章末实验题:JSON 迷你求值器与结算系统

实验 14:用 ADT 实现迷你表达式求值器 + 订单结算

实验目标:综合运用本章全部特性,实现两个递进任务:① 迷你表达式求值器(支持加减乘除与取负),要求扩展一种新运算后,编译器强制所有处理点更新;② 订单结算系统,用 record + Optional 处理"优惠券可能不存在/已过期"。

实验要求:

- 任务 1:用 sealed interface Expr 建模表达式,至少包含 Num / Neg / Add / Mul 四种变体(覆盖 §14.2 sealed)
- 任务 2:每个变体用 record 携带载荷(覆盖 §14.1 record)
- 任务 3:用 switch 表达式 + 模式匹配写 eval,不得使用 default(覆盖 §14.3 穷尽性)
- 任务 4:再增加 Div 变体(除零抛异常),观察所有 switch 的编译错误并逐个修复(覆盖 §14.4 编译器哨兵)
- 任务 5:实现 print(Expr) 输出中缀表达式(覆盖 §14.3 yield 多语句分支)
- 任务 6:结算系统:Order 是 record;findCoupon 返回 Optional<Coupon>;优惠券有 rate(折扣率)与 expiry;过期视为不存在(覆盖 §14.5 Optional 链)
- 任务 7:结算方法用 map/filter/orElse 组合,总价计算正确(覆盖 §14.5 链式)
- 任务 8:测试用例覆盖:合法表达式、除零、负号、无优惠券、已过期优惠券(覆盖全章验收)

实验环境:JDK 17+ 即可,单文件 javac/java 运行。

第一步 完成任务 1-4:声明 + 求值器,务必观察到"加 Div 后编译器报错"的过程

第二步 任务 5 的 print 用 yield 组装字符串

第三步 任务 6-7:Coupon 是 record(rate, expiry);findCoupon 返回 Optional;结算 = 原价 * 折扣率

参考答案要点


```

import java.time.LocalDate;
import java.util.Optional;

// ===== 任务 1-5:表达式求值器 =====
// sealed:只允许四种变体;record:每种变体自带载荷
public sealed interface Expr permits Num, Neg, Add, Mul, Div {
}

public record Num(int value) implements Expr { }

public record Neg(Expr inner) implements Expr { }

public record Add(Expr left, Expr right) implements Expr { }

public record Mul(Expr left, Expr right) implements Expr { }

// 任务 4:新增的变体—编译器会让所有 switch 报"not exhaustive"
public record Div(Expr left, Expr right) implements Expr { }

public final class ExprUtil {
    // 任务 3/4:穷尽分发,无 default;加 Div 后此处必须先修
    public static int eval(Expr e) {
        return switch (e) {
            case Num n -> n.value();
            case Neg g -> -eval(g.inner());
            case Add a -> eval(a.left()) + eval(a.right());
            case Mul m -> eval(m.left()) * eval(m.right());
            case Div d -> {
                int r = eval(d.right());
                if (r == 0) {
                    throw new ArithmeticException("除零错误");
                }
                yield eval(d.left()) / r; // 任务 5:多语句用 yield
            }
        };
    }

    // 任务 5:输出中缀表达式
    public static String print(Expr e) {
        return switch (e) {
            case Num n -> String.valueOf(n.value());
            case Neg g -> "-" + print(g.inner()) + "";
            case Add a -> "(" + print(a.left()) + " + " + print(a.right()) + "";
            case Mul m -> "(" + print(m.left()) + " * " + print(m.right()) + "";
            case Div d -> "(" + print(d.left()) + " / " + print(d.right()) + "";
        };
    }

    public static void main(String[] args) {
        // (1 + 2) * 3 / (-2) = -4 ... 1.5 取整 -4?看运算顺序:
        // ((1+2)*3)/Neg(2) -> 9 / -2 = -4(整数除法)
        Expr e = new Div(new Mul(new Add(new Num(1), new Num(2)),
            new Num(3)), new Neg(new Num(2)));
        System.out.println(print(e) + " = " + eval(e));
    }
}

// ===== 任务 6-7:订单结算系统 =====
record Coupon(double rate, LocalDate expiry) {
    boolean validOn(LocalDate day) {
        return !expiry.isBefore(day); // 未过期
    }
}

record Order(int amount, String couponCode) { }

public final class Checkout {
    // 模拟数据库:按优惠码查券,可能不存在

```

```

static Optional<Coupon> findCoupon(String code) {
    if (code == null || code.isBlank()) {
        return Optional.empty();
    }
    if (code.equals("SAVE20")) {
        return Optional.of(new Coupon(0.8, LocalDate.of(2026, 12, 31)));
    }
    if (code.equals("OLD10")) {
        return Optional.of(new Coupon(0.9, LocalDate.of(2025, 1, 1))); // 已过期
    }
    return Optional.empty();
}

// 结算:无券/过期券 → 原价;有效券 → 按折扣率
static int settle(Order order, LocalDate today) {
    return findCoupon(order.couponCode())
        .filter(c -> c.validOn(today)) // 过期即视为不存在
        .map(c -> (int) (order.amount() * c.rate()))
        .orElse(order.amount()); // 无券:原价
}

public static void main(String[] args) {
    LocalDate today = LocalDate.of(2026, 8, 26);
    System.out.println(settle(new Order(100, "SAVE20"), today)); // 80
    System.out.println(settle(new Order(100, "OLD10"), today)); // 100:过期
    System.out.println(settle(new Order(100, null), today)); // 100:无券
    System.out.println(settle(new Order(100, "NOPE"), today)); // 100:未知码
}
}

```

设计说明:任务 4 是本章最有教育意义的时刻——给 sealed 体系新增一个变体,编译器把所有 switch 的遗漏点全部找出来,这正是"类型驱动的重构"。任务 6-7 的 Optional 链(filter 过滤过期券 → map 计算折扣价 → orElse 兜底原价)是生产中最常见的三件套组合。扩展方向:给 Expr 增加"化简"方法(把 Add(Num 0, x) 化简为 x),再体验一次编译器哨兵;结算可加"满减券"变体,体会 record 组合扩展。

全书写完。14 章从数据类型到类型系统进阶,你已具备生产级 Java 能力。建议回顾:第5章内存、第6章面向对象、第9章并发、第14章类型系统——这四章是 Java 与 C 思维差异最大的地方。